UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 01 resuelta

Evaluación de Sistemas Heredados: Auditoría de Línea Base e Inventario de Deuda Técnica

Asignatura	Ingeniería de Software II
Corte	Segundo Corte
Duración	Semana 1
Proyecto integrador	Fase 17: Auditoría de línea base y diagnóstico de deuda técnica del MVP heredado de Ingeniería de Software I
Herramientas	Linter estático (SonarQube/SonarCloud), Git/GitHub, ISO/IEC/IEEE 12207:2026 como referencia + IA generativa auditada
Formato de entrega	Documento con taller resuelto y Ficha de Aplicación al Proyecto Integrador (FPI-17) diligenciada
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

Fase 17 del Proyecto Integrador — Evolución del MVP

En Ingeniería de Software I aprendiste a construir un software correcto: pasaste de una necesidad real de una MiPyme a un MVP funcional, cerrando el semestre con la síntesis final registrada en tu FPI-16. En Ingeniería de Software II el eje de gravedad se desplaza de la construcción a la industrialización, operación, seguridad, confiabilidad y evolución de ese mismo software a escala. En el mundo profesional, un ingeniero rara vez enfrenta una página en blanco: la práctica real consiste en heredar bases de código existentes, comprender sus límites, medir sus riesgos y evolucionarlas sin interrumpir la operación del negocio.

Esta guía retoma directamente el MVP entregado en la FPI-16: tu propio código, ahora, es el "código heredado" que debes auditar con rigor antes de tocarlo. El diagnóstico de línea base y el inventario de deuda técnica que construyas aquí se registrarán en la FPI-17 y guiarán la evolución del sistema durante todo este segundo semestre.

Resultados de aprendizaje

- Evaluar una arquitectura de software existente (MVP heredado) analizando de forma cualitativa y cuantitativa su deuda técnica, límites operativos y riesgos técnicos asociados.
- Establecer la línea base de configuración y el diagnóstico inicial de un sistema de software heredado bajo los lineamientos de la ISO/IEC/IEEE 12207:2026.
- Clasificar la deuda técnica según su tipo (código, diseño, pruebas, seguridad) y según el cuadrante de decisión de Martin Fowler.

- Aplicar heurísticas de auditoría técnica (complejidad ciclomática, acoplamiento y cohesión) sobre código real.
- Auditar y validar críticamente las sugerencias de IA generativa aplicadas al análisis de código legado, de forma ética e independiente.

Objetivos

General

Establecer la línea base técnica del MVP heredado de Ingeniería de Software I, mediante un inventario riguroso de su deuda técnica, como punto de partida formal para su industrialización y evolución.

Específicos

- Explicar por qué el ciclo de vida ISO/IEC/IEEE 12207:2026 exige establecer una línea base antes de cualquier mantenimiento.
- Diferenciar los cuatro tipos de deuda técnica (código, diseño, pruebas, seguridad e infraestructura).
- Clasificar hallazgos reales de deuda técnica según el cuadrante de Martin Fowler.
- Auditar un fragmento de código legado detectando vulnerabilidades y proponiendo una refactorización SOLID validada.
- Registrar el diagnóstico de línea base y el inventario de deuda técnica del proyecto integrador en la FPI-17.

Saberes previos

Antes de iniciar, responde (no se califica):

- ¿Recuerdas los principios SOLID y los code smells que trabajaste en Ingeniería de Software I?

Respuesta del estudiante: Sí: responsabilidad única, abierto/cerrado, sustitución, segregación e inversión de dependencias. En Horizonte reviso vistas amplias y duplicación de controles.

- ¿Alguna vez recibiste un código escrito por otra persona (o por ti mismo meses atrás) que te costó entender?

Respuesta del estudiante: El propio código de distintas etapas exige recuperar contexto. Lo documento por versiones y pruebas; no invento participación de otro desarrollador.

- ¿Qué tan honesto crees que puede ser tu equipo auditando las propias decisiones que tomó el semestre pasado bajo presión de tiempo?

Respuesta del estudiante: La autorrevisión individual puede ocultar sesgos. Uso evidencia de archivos y pruebas y dejo pendiente revisión externa cuando no ocurrió.

Concepto 1 — De "software como producto" a "software como sistema operable"

El SWEBOK v4 organiza la disciplina en áreas de conocimiento. En Ingeniería de Software I trabajaste principalmente Software Requirements y Software Design; en este nivel entran en juego Software Architecture, Software Engineering Operations y Software Security como pilares para lograr la confiabilidad de un sistema en producción.

La ISO/IEC/IEEE 12207:2026 exige establecer formalmente una Línea Base (Baseline) antes de emprender cualquier mantenimiento adaptativo, correctivo o perfectivo: una especificación o producto de trabajo

aprobado que sirve como punto de partida medible para la evolución del software, y que solo se modifica mediante control de cambios formal. Esta semana, esa línea base es el diagnóstico exhaustivo de calidad, cobertura de pruebas, seguridad y arquitectura de tu propio MVP heredado.

Concepto 2 — La metáfora de la deuda técnica (Ward Cunningham, 1992)

La deuda técnica describe el costo diferido de tomar atajos de diseño o implementación en el corto plazo para acelerar una entrega. Como la deuda financiera, no es intrínsecamente mala si se adquiere de forma consciente; pero si no se paga de forma planificada, el "interés" acumulado (tiempo extra, bugs en cascada, parálisis arquitectónica) puede hacer inviable el proyecto.

Tipo de deuda	Descripción
Deuda de código	Micro-arquitectura: variables sin intención reveladora, métodos gigantescos, acoplamiento estrecho entre clases (Code Smells).
Deuda de diseño / arquitectónica	Decisiones estructurales deficientes que impiden el crecimiento modular (ej. la interfaz accede directamente a la base de datos sin capa de abstracción).
Deuda de pruebas	Ausencia o debilidad de pruebas automatizadas: sin ellas no se puede refactorizar con seguridad, porque no hay forma de verificar que el comportamiento no cambió.
Deuda de seguridad e infraestructura	Credenciales en duro, librerías desactualizadas con CVE conocidos, o ausencia de entornos reproducibles (Docker).

Concepto 3 — El cuadrante de decisión de la deuda técnica (Martin Fowler)

- **Deliberada + Prudente:** el equipo sabe que toma un atajo, lo documenta y planifica su pago inmediato. Es la única deuda aceptable en ingeniería.
- **Deliberada + Imprudente:** el equipo escribe código rápido y sucio a sabiendas del desastre, sin documentar ni planificar el pago. Debe evitarse categóricamente.
- **Inadvertida + Imprudente/Prudente:** ocurre por ignorancia de principios de diseño, o porque el negocio evoluciona y el diseño original queda obsoleto de forma natural. Se mitiga con auditorías sistemáticas.

Regla de oro de esta guía: *"Toda deuda técnica que tu equipo decida asumir este semestre debe ser deliberada y prudente: documentada, justificada y con un plan de pago explícito — nunca oculta ni improvisada."*

Concepto 4 — Heurísticas de auditoría técnica de código heredado

- **Complejidad ciclomática (McCabe):** mide la cantidad de caminos lógicos independientes de un método. Un valor mayor a 10 indica un método propenso a errores, difícil de probar y de mantener.
- **Acoplamiento y cohesión (métricas CK):** cohesión mediante LCOM (Lack of Cohesion in Methods) y acoplamiento mediante CBO (Coupling Between Object Classes). Se busca siempre alta cohesión funcional y bajo acoplamiento de datos.
- **Análisis de la cadena de suministro:** revisión de las librerías de terceros (requirements.txt, package.json) para identificar licencias restrictivas (ej. GPL v3) o vulnerabilidades conocidas.

Analogía: un transportador que hereda un camión de carga antiguo para cruzar el Puente Internacional Simón Bolívar no arranca a máxima velocidad sin antes revisar frenos, aceite, SOAT y documentos. Arrancar con los frenos desgastados para entregar la carga más rápido es exactamente lo que hace un equipo que ignora la deuda técnica de seguridad de su propio código por salir rápido a producción.

Ejemplo resuelto: cálculo de complejidad ciclomática

Antes de que calcules la complejidad ciclomática de tu propio código en el taller, observa este ejemplo ilustrativo (no lo copies: usa tu propio método real).

```
def validar_descuento(cliente, monto, es_temporada_alta):
    if cliente.es_frecuente:              # +1
        if monto > 100000:                # +1
            descuento = 0.15
        else:
            descuento = 0.05
    elif es_temporada_alta:               # +1
        descuento = 0.02
    else:
        descuento = 0.0
    return descuento
```

Este método tiene 3 decisiones (condicionales), por lo que V(G) = 3 + 1 = 4. Eso significa que se necesitan mínimo 4 casos de prueba distintos para cubrir todos sus caminos lógicos: cliente frecuente con monto alto, cliente frecuente con monto bajo, temporada alta sin ser cliente frecuente, y ningún caso especial.

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, por qué la ISO/IEC/IEEE 12207:2026 exige establecer una línea base antes de iniciar el mantenimiento de un sistema heredado.

Respuesta del estudiante: La línea base registra versiones, comportamiento, pruebas y deuda antes de cambiar el sistema. Permite comparar resultados, rastrear cambios y recuperar una versión conocida. En Horizonte incluye commit local, dependencias, migraciones y límites del modelo. La ficha ISO confirma el marco de ciclo de vida; no atribuyo a su resumen público una cláusula textual específica no consultada.

Ejercicio 2

Da un ejemplo propio (real o hipotético) de cada uno de los cuatro tipos de deuda técnica (código, diseño, pruebas, seguridad).

Respuesta del estudiante: Código: controlador de dashboard combina consultas y composición. Diseño: aislamiento repetido exige disciplina en cada API. Pruebas: falta aceptación transversal y rendimiento representativo. Seguridad/infraestructura: Docker backend sin USER explícito y revisión ASVS incompleta. Son hallazgos y riesgos, no prueba de incidentes ocurridos.

Nivel Intermedio (auditoría de código legado + IA)

Ejercicio 3

Analiza el siguiente fragmento de un sistema de facturación e inventario. Identifica al menos tres problemas graves (pista: inyección SQL, lógica fiscal en duro, credenciales expuestas, acoplamiento por variable global) y clasifícalos según el cuadrante de Fowler:

Respuesta del estudiante: El SELECT y el INSERT interpolan entradas y permiten inyección; api_key queda expuesta y se imprime; CONEXION_GLOBAL acopla persistencia; IVA fijo oculta política; se registra venta antes de confirmar pago y no hay control de stock ni rollback. Sin conocer intención no se puede inferir deliberación: clasifico riesgo imprudente y registro intención no demostrada. El IVA fijo puede ser deuda deliberada prudente solo si se limita y documenta su contexto.

```
CONEXION_GLOBAL = None

def procesar_transaccion_venta(cliente_nombre, cliente_cedula, referencia_zapato, cantidad,
metodo_pago):
    global CONEXION_GLOBAL
    cursor = CONEXION_GLOBAL.cursor()

    # Inyección SQL directa
    consulta = f"SELECT id, precio, stock FROM zapatos WHERE referencia =
'{referencia_zapato}'"
    cursor.execute(consulta)
    zapato_id, precio, stock = cursor.fetchone()

    iva = 0.19  # IVA en duro
    total = (cantidad * precio) * (1 + iva)

    cursor.execute(f"""
        INSERT INTO ventas (cliente_nombre, cliente_cedula, zapato_id, cantidad,
total_pagado)
        VALUES ('{cliente_nombre}', '{cliente_cedula}', {zapato_id}, {cantidad}, {total})
    """)

    if metodo_pago == "PAYU":
        api_key = "PROD_SECRET_KEY_EXPOSED_12345"  # Secreto expuesto
        print(f"Llamando a PayU con {api_key}...")
```

Ejercicio 4

Calcula la complejidad ciclomática V(G) del método procesar_transaccion_venta del Ejercicio 3 (cuenta las decisiones: if, elif). Indica cuántos casos de prueba mínimos necesitaría para cubrir todos sus caminos lógicos.

Respuesta del estudiante: El fragmento contiene un único if (metodo_pago == PAYU): V(G)=1+1=2. Hay dos caminos de control mínimos, PAYU y otro. Dos tests no garantizan seguridad ni cubren entradas inválidas; agregaría producto inexistente, cantidad negativa, stock insuficiente, error de pago e inyección como dato.

Ejercicio 5

Pide a una IA generativa que refactorice el fragmento del Ejercicio 3 aplicando SOLID (Inversión de Dependencias) para desacoplar la base de datos y la pasarela de pago. Documenta el prompt y la respuesta completa.

Respuesta del estudiante: Prompt de esta entrega: refactoriza el ejemplo de venta con dependencias inyectadas, Decimal y transacción, sin secretos ni bibliotecas externas. Respuesta completa: ver el listado Refactor didáctico de venta incluido al final del taller. Es un ejemplo aislado, no una función de Horizonte: el SaaS no cobra ni mueve dinero.

Ejercicio 6

Audita la refactorización de la IA respondiendo, con evidencia concreta, las cuatro preguntas de la matriz de auditoría que usarás en la FPI-17: ¿detectó bien las vulnerabilidades?, ¿el código compila lógicamente?, ¿qué licencias tienen las librerías sugeridas?, ¿cómo se protegería la privacidad si los datos fueran reales? Guarda tus respuestas: las trasladarás directamente a tu ficha.

Respuesta del estudiante: Auditoría: SQL parametrizado pertenece al adaptador, no al núcleo; no hay credencial en argumentos ni logs; se compila el ejemplo con ast.parse. Solo usa Decimal y Protocol de la biblioteca estándar de Python; no añade paquetes. Se usan nombres ficticios. Para producción, tokenizar pago, garantizar idempotencia y diseñar compensación entre pago remoto y transacción local; no afirmar atomicidad distribuida.

Artefacto académico completo — propuesta o ejemplo según respuesta anterior

```
from decimal import Decimal
from typing import Protocol
class Inventory(Protocol):
    def quote(self, sku: str, quantity: int): ...
    def record_sale(self, sku: str, quantity: int, total: Decimal): ...
class Payment(Protocol):
    def authorize(self, total: Decimal, key: str): ...
def sell(repo: Inventory, payment: Payment, sku: str, quantity: int,
         tax: Decimal, key: str, unit_of_work):
    if quantity <= 0:
        raise ValueError("Cantidad inválida")
    price = repo.quote(sku, quantity)
    total = (price * quantity * (Decimal("1") + tax)).quantize(Decimal("0.01"))
    authorization = payment.authorize(total, key)
    with unit_of_work():
        repo.record_sale(sku, quantity, total)
    return authorization
# Los adaptadores implementan SQL parametrizado, stock con bloqueo,
# secretos por entorno y compensación del pago si falla la persistencia.
```

Lista de autoevaluación antes de entregar

- Distingo los cuatro tipos de deuda técnica y los identifico con ejemplos reales de mi propio MVP.
- Clasifiqué al menos una deuda real de mi proyecto según el cuadrante de Fowler.
- Analicé un fragmento de código legado detectando vulnerabilidades reales (no genéricas).
- Audité si la refactorización de la IA realmente resolvió el acoplamiento o solo lo disimuló.
- Diligencié completamente la FPI-17, recuperando y validando lo cerrado en la FPI-16.

Foro de reflexión

Título: *"El dilema del código legado: ¿refactorizar hasta la perfección o convivir pragmáticamente con la deuda?"*

Redacte una reflexión de 250–300 palabras respondiendo: desde el punto de vista del ROI de una MiPyme, ¿es justificable detener nuevas funcionalidades para refactorizar a fondo durante varias semanas, o es aceptable convivir con la deuda mientras el sistema cumpla sus requisitos básicos? ¿Cómo afecta la ausencia de pruebas automatizadas al costo real de pagar esa deuda? Apoye su respuesta con evidencia de su propio proyecto.

Reflexión del estudiante (283 palabras)
En Horizonte, considero que conservar línea base verificable y priorizar deuda por riesgo, sin reescribir todo el SaaS. La línea base registra versiones, comportamiento, pruebas y deuda antes de cambiar el sistema. Permite comparar resultados, rastrear cambios y recuperar una versión conocida. En Horizonte incluye commit local, dependencias, migraciones y límites del modelo. La ficha ISO confirma el marco de ciclo de vida; no atribuyo a su resumen público una cláusula textual específica no consultada. El caso concreto que orienta mi análisis es el siguiente: El SELECT y el INSERT interpolan entradas y permiten inyección; api_key queda expuesta y se imprime; CONEXION_GLOBAL acopla persistencia; IVA fijo oculta política; se registra venta antes de confirmar pago y no hay control de stock ni rollback. Sin conocer intención no se puede inferir deliberación: clasifico riesgo imprudente y registro intención no demostrada. El IVA fijo puede ser deuda deliberada prudente solo si se limita y documenta su contexto. Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia. La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados.

Cuestionario

Seleccione una única respuesta.

1. Un sistema carece de pruebas automatizadas y cualquier cambio en el cálculo del IVA rompe el módulo de inventarios. Según el cuadrante de Fowler, esto es:

- A. Deuda técnica deliberada y prudente.
- B. Deuda técnica inadvertida e imprudente, por falta de pruebas de regresión y diseño modular. ✅
- C. Un diseño evolutivo óptimo.
- D. No es deuda técnica porque el sistema sigue facturando.

2. El propósito primordial de establecer una línea base técnica al heredar un MVP es:

- A. Reescribir el software completo con el framework de moda.
- B. Registrar formalmente el estado actual (calidad, dependencias, cobertura, vulnerabilidades) para medir objetivamente la evolución del sistema. ✅
- C. Eliminar la necesidad de revisiones de código por pares.
- D. Desplegar automáticamente en producción sin pruebas previas.

3. Reemplazar toda la base de datos relacional por NoSQL sin analizar atributos de calidad ni escribir pruebas, solo porque "es lo moderno", es un ejemplo de:

- A. Aplicación correcta del principio Abierto/Cerrado.
- B. Hype-Driven Development: complejidad y riesgo introducidos sin justificación en drivers arquitectónicos reales. ✅
- C. Reducción óptima del tamaño del software.
- D. Aplicación estricta del modelo de madurez CMMI.

4. Una dependencia desactualizada con una vulnerabilidad crítica (CVE) reportada pertenece a la categoría de deuda de:

- A. Documentación; se soluciona con un comentario en el código.
- B. Seguridad y cadena de suministro; se mitiga con escaneo SCA y actualización a versión parchada. ✅
- C. Proceso; se corrige eliminando el archivo de dependencias.
- D. Arquitectura; se mitiga migrando a microservicios.

5. Subir un esquema de base de datos con nombres, cédulas y teléfonos reales de clientes a una IA pública para pedir una sugerencia de refactorización:

- A. Es válido porque el código SQL no es información confidencial.
- B. Viola la Ley 1581 de 2012 y el principio ético de confidencialidad, al transferir datos personales reales sin autorización. ✅
- C. Es legal si se declara el uso del LLM en el manual técnico.
- D. Está exento de regulación por tratarse de un prompt de IA.

6. Un método con complejidad ciclomática (McCabe) mayor a 10 se considera:

- A. Óptimo y de fácil mantenimiento sin importar su tamaño.
- B. Altamente propenso a errores, difícil de probar y de mantener. ✅
- C. Imposible de medir sin herramientas comerciales de pago.
- D. Un indicador exclusivamente de rendimiento en producción.

7. Un equipo asume conscientemente un atajo técnico, lo documenta y planifica pagarlo en la siguiente iteración. Según Fowler, esta es:

- A. Deuda inadvertida e imprudente.
- B. La única deuda aceptable en ingeniería: deliberada y prudente. ✅
- C. Una violación grave de la ISO 12207.
- D. Deuda que nunca debe registrarse en el backlog.

Cuestionario resuelto: 1. B; 2. B; 3. B; 4. B; 5. B; 6. B; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-17

Auditoría de Línea Base e Inventario de Deuda Técnica del MVP Heredado

Esta ficha continúa directamente la FPI-16, que cerró Ingeniería de Software I. El MVP entregado allí es ahora el sistema heredado que tu equipo debe auditar con rigor antes de evolucionarlo.

A. Identificación de la ficha

Código de la ficha	FPI-17
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	Evaluación de sistemas heredados: línea base y deuda técnica

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Recuperar y validar la FPI-16

Síntesis del cierre de Ingeniería de Software I (recuperada de la FPI-16)

Copia aquí el resumen del MVP, la deuda técnica pendiente y las lecciones aprendidas registradas al cierre de la FPI-16. Respuesta: FPI-16 describía escenario contractual y deuda de pruebas/documentación. El corte actual incorpora captura manual, mensualidades, CSV/XLSX/XML, Mock Bank, modelos experimentales y QA local. No se conserva como actual el conteo antiguo de22 tests.

¿Algo cambió desde la ficha anterior?

Qué cambió	Por qué cambió	Impacto en el proyecto
Estado del producto	Las16 guías anteriores eran un corte antiguo	Se usa matriz vigente y se conserva historial
Tipo de beneficiario	SaaS multiempresa según estudiante	No se simula una empresa aliada única

Si nada cambió, escribe "Sin cambios" en la primera fila y deja la tabla así.

C. Aplicación específica — Línea base y deuda técnica

1. Ficha del MVP heredado

Nombre del MVP	Horizonte SaaS de inteligencia de liquidez para microempresas

MiPyme aliada	Segmento SaaS de microempresas; sin aliada única
Lenguajes y stack de desarrollo	Python/Django/DRF; React/TS; PostgreSQL; Redis/Celery
URL del repositorio de GitHub	https://github.com/elcreado/Horizonte

2. Inventario de deuda técnica (mínimo 3 ítems)

Ejemplo resuelto (ilustrativo — no lo copies, completa la tabla con hallazgos reales de tu propio MVP):

Ítem de deuda	Categoría	Ubicación	Impacto en calidad	Plan de pago tentativo
DT01 Contenedor backend	Seguridad/infraestructura	backend/Dockerfile	Root implícito y servidor desarrollo	Sprint propuesto1: build no root y regresión
DT02 Aceptación y carga	Pruebas	docs/acceptance; pruebas de carga	Sin evidencia integral p95/UX	Sprint propuesto2: protocolo reproducible
DT03 Observabilidad/SLO	Diseño/operación	Operación y telemetría	Diagnóstico y disponibilidad no acreditados	Sprint propuesto3: señales mínimas y ensayo

Ítem de deuda	Categoría (código/diseño/pruebas/seguridad)	Ubicación (archivo/módulo)	Impacto en calidad	Plan de pago tentativo
DT01 Contenedor backend	Seguridad/infraestructura	backend/Dockerfile	Root implícito y servidor desarrollo	Sprint propuesto1: build no root y regresión
DT02 Aceptación y carga	Pruebas	docs/acceptance; pruebas de carga	Sin evidencia integral p95/UX	Sprint propuesto2: protocolo reproducible
DT03 Observabilidad/SLO	Diseño/operación	Operación y telemetría	Diagnóstico y disponibilidad no acreditados	Sprint propuesto3: señales mínimas y ensayo

3. Registro de Decisión Arquitectónica — ADR N.° 01

A partir de esta guía, cada ADR de tu proyecto seguirá siempre esta misma plantilla estructurada de 5 campos.

Título	ADR-01 Línea base y deuda
Estado	Aceptado como registro académico; implementación y brechas descritas por separado

Contexto	SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar. Línea base y deuda
Decisión	Conservar línea base verificable y priorizar deuda por riesgo, sin reescribir todo el SaaS.
Consecuencias (positivas y negativas)	Menor complejidad y límites explícitos. Requiere medición, aceptación y revisión externa para completar evidencia.

4. Auditoría a la IA en la refactorización del código legado

Elemento auditado	Hallazgo y corrección del equipo
¿La IA detectó con exactitud las vulnerabilidades (inyección SQL, secretos expuestos)?	Auditoría: SQL parametrizado pertenece al adaptador, no al núcleo; no hay credencial en argumentos ni logs; se compila el ejemplo con ast.parse. Solo usa Decimal y Protocol de la biblioteca estándar de Python; no añade paquetes. Se usan nombres ficticios. Para producción, tokenizar pago, garantizar idempotencia y diseñar compensación entre pago remoto y transacción local; no afirmar atomicidad distribuida.
¿El código refactorizado compila lógicamente sin errores de sintaxis o imports inexistentes?	Se contrasta contra código y resultados, no contra confianza del LLM. Conservar línea base verificable y priorizar deuda por riesgo, sin reescribir todo el SaaS.
¿Qué licencias aplican a las librerías que sugirió importar la IA?	Se contrasta contra código y resultados, no contra confianza del LLM. Conservar línea base verificable y priorizar deuda por riesgo, sin reescribir todo el SaaS.
¿Cómo se protegió la privacidad de los datos reales (Ley 1581) al redactar los prompts?	Se contrasta contra código y resultados, no contra confianza del LLM. Conservar línea base verificable y priorizar deuda por riesgo, sin reescribir todo el SaaS.

D. Reflexión y trazabilidad

¿Qué aprendimos?

Resume lo aprendido sobre línea base, deuda técnica y el cuadrante de Fowler.

Respuesta: La línea base registra versiones, comportamiento, pruebas y deuda antes de cambiar el sistema. Permite comparar resultados, rastrear cambios y recuperar una versión conocida. En Horizonte incluye commit local, dependencias, migraciones y límites del modelo. La ficha ISO confirma el marco de ciclo de vida; no atribuyo a su resumen público una cláusula textual específica no consultada. Código: controlador de dashboard combina consultas y composición. Diseño: aislamiento repetido exige disciplina en cada API. Pruebas: falta aceptación transversal y rendimiento representativo. Seguridad/infraestructura: Docker backend sin USER explícito y revisión ASVS incompleta. Son hallazgos y riesgos, no prueba de incidentes ocurridos.

¿Cómo se aplica al proyecto?

Explica cómo este diagnóstico honesto cambia la forma en que el equipo evolucionará el MVP este semestre.

Respuesta: Conservar línea base verificable y priorizar deuda por riesgo, sin reescribir todo el SaaS.

Decisiones tomadas

Decisión	Justificación
Conservar línea base verificable y priorizar deuda por riesgo, sin reescribir todo el SaaS.	Alineación con riesgo financiero y capacidad individual
Separar evidencia y propuesta	Evitar certificar despliegues, revisiones o mediciones no realizados

Evidencia generada

Estado documentado: Ficha del MVP heredado diligenciada. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Inventario de deuda técnica con plan de pago. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: ADR N.° 01 registrado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Código SOLID refactorizado y validado por el equipo. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Dificultades o ajustes identificados

¿Fue difícil ser honestos sobre la deuda propia? ¿Qué generó más discusión en el equipo?

Respuesta: Trabajo solo: el riesgo es sesgo de autorrevisión, no desacuerdo de un equipo ficticio. Contrasté Dockerfile y código con las afirmaciones de la ficha.

Próximo paso

¿Qué debe hacer el equipo antes de la siguiente guía?

Respuesta: Versionar diagnóstico y priorizar DT01–DT03 antes de elegir nuevos componentes.

Conexión con la siguiente guía

El inventario de deuda técnica y el ADR N.° 01 construidos en esta FPI-17 son la base directa de la siguiente guía, donde el equipo decidirá y justificará el estilo arquitectónico (capas, hexagonal, monolito modular) hacia el cual evolucionará el MVP. Ninguna deuda identificada aquí se resuelve en silencio: cada refactorización futura deberá referenciar el ítem de deuda correspondiente de esta ficha.

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5

Rigor de la auditoría de línea base (20%)	No define el estado actual del MVP	Descripción general y superficial	Analiza calidad, dependencias y seguridad con algo de detalle	Análisis detallado con métricas y evidencia	Línea base exhaustiva con métricas automatizadas y ADR sólido
Calidad del inventario de deuda técnica (20%)	No identifica deudas o son subjetivas	Identifica 1-2 ítems sin rigor metodológico	Identifica 3 deudas con categorización adecuada	Deudas bien categorizadas con impacto y plan de pago	Inventario riguroso, con ubicación exacta y plan de pago con hitos
Análisis del código legado y cuadrante de Fowler (15%)	No identifica vulnerabilidades reales	Identifica errores triviales sin clasificar	Identifica vulnerabilidades y clasifica correctamente	Análisis riguroso y bien argumentado	Análisis ejemplar, con clasificación precisa de Fowler
Rigor de la auditoría a la IA (20%)	Copia ciegamente la respuesta de la IA	Auditoría superficial	Detecta errores básicos de la refactorización	Auditoría robusta, valida licencias y compilación	Auditoría forense excepcional, incluye privacidad Ley 1581
Participación en el foro (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada
Trazabilidad respecto a la FPI-16 (15%)	No recupera lo cerrado en Software I	Copia sin verificar vigencia	Valida lo esencial	Valida y actualiza correctamente	Trazabilidad ejemplar con causa e impacto de cambios

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%).

Recursos académicos y bibliografía

- Fowler, Martin. TechnicalDebtQuadrant (martinfowler.com).
- Cunningham, Ward. The WyCash Portfolio Management System.
- Pressman, Roger. Ingeniería del Software: Un Enfoque Práctico (Reingeniería y Evolución).
- ISO/IEC/IEEE 12207:2026. Systems and software engineering — Software life cycle processes.
- McCabe, Thomas J. A Complexity Measure.

UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 02 resuelta

Drivers Arquitectónicos: Atributos de Calidad (ISO/IEC 25010) y Escenarios Verificables

Asignatura	Ingeniería de Software II
Corte	Primer Corte
Duración	Semana 2
Proyecto integrador	Fase 18: Definición de drivers arquitectónicos y escenarios de calidad verificables del MVP
Herramientas	ISO/IEC 25010:2023, plantilla de escenarios del SEI (Software Engineering Institute) + IA generativa auditada
Formato de entrega	Documento con taller resuelto y Ficha de Aplicación al Proyecto Integrador (FPI-18) diligenciada
Fecha límite	Domingo de la Semana 2, 11:59 p.m. (ver fecha exacta en el aula virtual)
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar.
Corte documental 30/10/2026. Se contrastaron código, compose.yaml, .github/workflows/ci.yml y docs/IMPLEMENTATION_STATUS.md. El registro del 18/10 informa 235 pruebas PostgreSQL: 233 aprobadas y 2 omitidas; es evidencia histórica, no una nueva ejecución de esta entrega. No se certifica producción actual, ASVS, p95 representativo ni validación con empresarios. Presupuesto inicial 0 USD y respaldo manual por decisión del proyecto.

Fase 18 del Proyecto Integrador — Evolución del MVP

En la FPI-17 diagnosticaste la deuda técnica de tu MVP heredado. Antes de decidir cómo corregirla, necesitas responder una pregunta más fundamental: ¿qué significa "calidad" para tu sistema, en términos medibles? Construir un software que "funcione" es solo el primer paso; el verdadero reto de la ingeniería es garantizar que sea seguro, escalable, disponible y mantenible bajo restricciones reales de presupuesto e infraestructura. Estas características, junto con las restricciones del negocio, son los drivers arquitectónicos: las fuerzas que determinarán cada decisión de diseño del resto del semestre.

Esta guía retoma el inventario de deuda técnica de la FPI-17: varios de esos hallazgos son, en realidad, síntomas de que un atributo de calidad nunca se definió de forma explícita y verificable. Aquí corregirás eso, registrando los drivers de tu proyecto en la FPI-18.

Resultados de aprendizaje

- Diferenciar un requisito funcional de un atributo de calidad (requisito no funcional) según ISO/IEC 25010:2023.
- Redactar escenarios de atributos de calidad verificables usando la plantilla del SEI (Estímulo/Fuente/Entorno/Artefacto/Respuesta/Medida).
- Identificar trade-offs (compromisos) entre atributos de calidad en conflicto.
- Priorizar los drivers arquitectónicos más críticos del MVP según el contexto real de la MiPyme.
- Auditar críticamente escenarios de calidad propuestos por IA generativa, detectando vaguedad o falta de medida cuantitativa.

Objetivos

General

Definir y priorizar los drivers arquitectónicos del proyecto integrador mediante escenarios de calidad verificables, como base para las decisiones de diseño del semestre.

Específicos

- Explicar las diferencias entre las características de ISO/IEC 25010:2023 (eficiencia de desempeño, seguridad, mantenibilidad, fiabilidad, etc.).
- Redactar al menos tres escenarios de calidad con la plantilla de seis partes del SEI.
- Identificar al menos un trade-off real entre dos atributos de calidad de su proyecto.
- Justificar la priorización de sus tres drivers más críticos.
- Registrar los drivers y escenarios del proyecto en la FPI-18.

Saberes previos

Antes de iniciar, responde (no se califica):

- ¿Recuerdas la diferencia entre un requisito funcional y uno no funcional (Guía 5 de Software I)?

Respuesta del estudiante: RF indica qué hace el SaaS; RNF fija calidad bajo condiciones, como latencia o aislamiento.

- ¿Alguna vez un producto (no de software) fue muy seguro pero incómodo de usar, o muy rápido pero poco confiable? ¿Qué compromiso se hizo?

Respuesta del estudiante: Como ejemplo hipotético, más verificación de acceso puede costar latencia. Priorizo privacidad y optimizo después de medir.

- ¿Qué característica de tu MVP crees que la organización beneficiaria valora más: que nunca se caiga, que sea rápido, o que sea fácil de modificar?

Respuesta del estudiante: Para clientes potenciales priorizo integridad y aislamiento. La preferencia real se validará con empresarios; no atribuyo una opinión a una empresa ficticia.

Concepto 1 — Atributos de calidad según ISO/IEC 25010:2023

La norma ISO/IEC 25010:2023 (modelo SQuaRE) organiza la calidad de un producto de software en características medibles, más allá de si el sistema "funciona". Algunas de las más relevantes para un MVP en evolución:

Característica	Qué mide
Eficiencia de desempeño	Tiempo de respuesta, uso de recursos (CPU, memoria) bajo carga.
Fiabilidad (Reliability)	Capacidad de mantenerse operando correctamente durante un periodo, incluyendo tolerancia a fallos.
Seguridad	Protección de datos y funciones contra accesos no autorizados.
Mantenibilidad	Facilidad para modificar, corregir o extender el sistema sin introducir defectos.
Compatibilidad	Capacidad de intercambiar información con otros sistemas y coexistir sin conflictos.
Usabilidad	Facilidad con la que usuarios específicos logran sus objetivos con efectividad y satisfacción.

Un requisito funcional dice qué hace el sistema ("debe permitir registrar una venta"); un atributo de calidad dice qué tan bien lo hace bajo condiciones específicas ("debe registrar la venta en menos de 1 segundo con 20 usuarios concurrentes").

Concepto 2 — Escenarios de atributos de calidad verificables (plantilla del SEI)

Decir "el sistema debe ser seguro" no es un driver arquitectónico: es una aspiración vaga. El Software Engineering Institute (SEI) propone estructurar cada atributo de calidad como un escenario con seis partes obligatorias, siguiendo el mismo espíritu de verificabilidad que ya aplicaste a los requisitos no funcionales en Software I:

Parte	Pregunta que responde
Fuente del estímulo	¿Quién o qué genera el evento? (usuario, sistema externo, pico de carga, atacante).
Estímulo	¿Qué condición o evento ocurre? (ej. 50 usuarios consultan simultáneamente).
Artefacto	¿Qué parte del sistema se ve afectada? (ej. el servicio de facturación).
Entorno	¿En qué condiciones ocurre? (ej. en horario pico, con conexión inestable).
Respuesta	¿Qué debe hacer el sistema? (ej. procesar la consulta sin caerse).
Medida de la respuesta	¿Cómo se verifica cuantitativamente? (ej. en menos de 2 segundos, con 0% de errores).

Ejemplo resuelto: "Cuando 30 clientes (fuente) consultan el catálogo de productos simultáneamente (estímulo) sobre el módulo de inventario (artefacto) durante el horario de mayor venta (entorno), el sistema debe responder cada consulta (respuesta) en menos de 1.5 segundos con una tasa de error menor al 1% (medida)."

Concepto 3 — Trade-offs entre atributos de calidad

Los atributos de calidad no son gratuitos: mejorar uno con frecuencia empeora otro. Aumentar la seguridad (cifrado, validaciones) suele reducir el desempeño; aumentar la disponibilidad (redundancia) suele aumentar el costo; aumentar la mantenibilidad (más abstracciones) puede aumentar la complejidad inicial. Un arquitecto de software no elimina estos compromisos: los negocia explícitamente y los documenta, en vez de optimizar un atributo a ciegas mientras degrada otro sin darse cuenta.

Regla de oro de esta guía: *"Ningún atributo de calidad se declara sin su escenario verificable de seis partes; y ningún escenario se aprueba sin analizar qué otro atributo podría verse sacrificado."*

Concepto 4 — Priorización de drivers según el contexto real

No todos los atributos de calidad importan igual para toda organización. Un sistema de facturación de una MiPyme con conectividad inestable en una zona rural de Norte de Santander probablemente priorice la fiabilidad y la operación offline por encima de un tiempo de respuesta ultrarrápido que solo importaría con miles de usuarios simultáneos que ese negocio nunca tendrá. Priorizar drivers es, otra vez, un ejercicio de contexto real — no de lo que "suena importante" en un libro de arquitectura.

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, la diferencia entre un requisito funcional y un atributo de calidad, con un ejemplo propio.

Respuesta del estudiante: RF describe una función, como importar CSV; un atributo de calidad describe bajo qué condiciones funciona, por ejemplo sin leer otra empresa. En Horizonte la integridad e idempotencia importan tanto como recibir el archivo. Una meta cuantitativa no equivale a resultado medido.

Ejercicio 2

Para un caso ficticio (una farmacia que digitaliza sus pedidos), redacta un escenario de calidad completo (las 6 partes del SEI) para el atributo de disponibilidad.

Respuesta del estudiante: Farmacia ficticia. Fuente: dependiente. Estímulo: consulta de pedido. Artefacto: API de pedidos. Entorno: horario comercial y carga acordada. Respuesta: atender consulta o devolver error controlado. Medida: 99% de solicitudes válidas con respuesta correcta en ventana de 30 días; registrar excluidos y caídas. Es meta propuesta.

Nivel Intermedio (trade-offs + auditoría a la IA)

Ejercicio 3

Identifica un trade-off real entre dos atributos de calidad para el caso de la farmacia (ej. seguridad vs. desempeño) y explica cómo lo resolverías, justificando qué atributo prioriza el contexto del negocio.

Respuesta del estudiante: Consultar autenticación y permisos añade trabajo, pero protege información de pacientes y pedidos. Priorizaría seguridad sobre unos milisegundos; optimizaría consultas e índices y mediría p95. No eliminaría controles para cumplir un tiempo arbitrario.

Ejercicio 4

Redacta un escenario de calidad para el atributo de mantenibilidad de tu propio proyecto, usando la plantilla de seis partes.

Respuesta del estudiante: Fuente: Adrian; estímulo: nueva regla de proyección; artefacto: servicio forecast; entorno: desarrollo con fixtures sintéticos; respuesta: cambio localizado y regresión íntegra; medida propuesta: modificar un servicio y sus pruebas sin cambiar el contrato público, revisión en una jornada. No es medición histórica.

Ejercicio 5

Pide a una IA generativa que redacte tres escenarios de calidad para tu MVP. Audita: ¿cada escenario tiene una medida cuantitativa real, o son declaraciones vagas tipo "debe ser rápido y seguro"? Reescribe los que fallen. Documenta el prompt, la respuesta y tu corrección.

Respuesta del estudiante: Prompt: redacta tres escenarios medibles de Horizonte sin afirmar mediciones. Respuesta: seguridad, cero accesos financieros cruzados en matriz tenant/rol; rendimiento, p95 menor de 2 s con 20 usuarios y 10.000 movimientos por empresa en entorno documentado; mantenibilidad, cambio de regla aislado con regresiones. Auditoría: cero fallos solo aplica a esa matriz; p95 y jornada son metas, no resultados. Se exige indicar fuente, estímulo, artefacto, entorno, respuesta y medida.

Lista de autoevaluación antes de entregar

- Diferencio con claridad un requisito funcional de un atributo de calidad.
- Mis escenarios de calidad tienen las seis partes del SEI, incluyendo una medida cuantitativa.
- Identifiqué al menos un trade-off real y justifiqué cómo lo resolvería mi equipo.
- Audité si una IA generó escenarios verificables o solo declaraciones vagas.
- Diligencié completamente la FPI-18, priorizando drivers reales de mi proyecto.

Foro de reflexión

Título: *"¿Puede un sistema ser excelente en todos los atributos de calidad a la vez?"*

Redacte una reflexión de 250–300 palabras respondiendo: ¿por qué intentar maximizar todos los atributos de calidad simultáneamente (máxima seguridad, máximo desempeño, máxima disponibilidad, mínimo costo) es una meta poco realista? ¿Cómo debería un equipo decidir, con criterio y no por capricho, qué atributos sacrificar parcialmente en su MVP? Apoye su respuesta con un ejemplo concreto de su propio proyecto.

Reflexión del estudiante (276 palabras)
En Horizonte, considero que priorizar aislamiento, integridad monetaria y mantenibilidad; medir desempeño antes de prometerlo. RF describe una función, como importar CSV; un atributo de calidad describe bajo qué condiciones funciona, por ejemplo sin leer otra empresa. En Horizonte la integridad e idempotencia importan tanto como recibir el archivo. Una meta cuantitativa no equivale a resultado medido.

El caso concreto que orienta mi análisis es el siguiente: Consultar autenticación y permisos añade trabajo, pero protege información de pacientes y pedidos. Priorizaría seguridad sobre unos milisegundos; optimizaría consultas e índices y mediría p95. No eliminaría controles para cumplir un tiempo arbitrario.

Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia.

La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados.

Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones. Si falta medición, registro el protocolo y la brecha; no transformo una propuesta en un éxito histórico. Así puedo defender el trabajo y decidir el siguiente incremento con evidencia.

Cuestionario

Seleccione una única respuesta.

1. La diferencia entre un requisito funcional y un atributo de calidad es que:

- A. Son exactamente lo mismo con nombres distintos.
- B. El funcional define qué hace el sistema; el atributo de calidad define qué tan bien lo hace bajo condiciones específicas y medibles. ✅
- C. Los atributos de calidad no pueden medirse cuantitativamente.
- D. Los requisitos funcionales solo aplican a interfaces de usuario.

2. En la plantilla de escenarios del SEI, la "Medida de la respuesta" corresponde a:

- A. Quién genera el evento que dispara el escenario.
- B. El criterio cuantitativo que permite verificar si el sistema cumplió la respuesta esperada. ✅
- C. El nombre del desarrollador responsable del módulo.
- D. El entorno físico donde se aloja el servidor.

3. "El sistema debe ser rápido y seguro" no es un escenario de calidad válido porque:

- A. Es demasiado corto para ser un escenario.
- B. Le faltan las seis partes estructuradas del SEI y una medida cuantitativa verificable. ✅
- C. Los escenarios de calidad no pueden mencionar seguridad.
- D. Debe redactarse siempre en inglés técnico.

4. Un trade-off entre atributos de calidad significa que:

- A. Mejorar un atributo no tiene ningún efecto sobre los demás.
- B. Mejorar un atributo (ej. seguridad) con frecuencia implica un costo en otro (ej. desempeño), y debe negociarse explícitamente. ✅
- C. Solo existe un atributo de calidad válido por proyecto.
- D. Los trade-offs solo ocurren en sistemas distribuidos.

5. Para una MiPyme con conectividad inestable en una zona rural, priorizar la fiabilidad y la operación offline por encima de la latencia ultrabaja es un ejemplo de:

- A. Un error de ingeniería, porque el desempeño siempre debe ser la prioridad máxima.
- B. Priorización de drivers arquitectónicos según el contexto real del negocio, no según preferencias genéricas. ✅
- C. Una violación de la norma ISO/IEC 25010:2023.
- D. Una decisión que no requiere justificación técnica.

6. Según la regla de oro de esta guía, un escenario de calidad se aprueba solo cuando:

- A. Suena importante y ambicioso para el negocio.
- B. Tiene sus seis partes completas y se analizó qué otro atributo podría verse sacrificado. ✅
- C. Un solo integrante del equipo lo redactó sin revisión.
- D. La IA lo generó automáticamente sin revisión humana.

7. La mantenibilidad, según ISO/IEC 25010:2023, se refiere a:

- A. La velocidad de respuesta del sistema bajo carga.
- B. La facilidad para modificar, corregir o extender el sistema sin introducir defectos. ✅
- C. La cantidad de servidores necesarios para desplegar el sistema.
- D. La compatibilidad con navegadores web antiguos exclusivamente.

Cuestionario resuelto: 1. B; 2. B; 3. B; 4. B; 5. B; 6. B; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-18

Drivers Arquitectónicos y Escenarios de Calidad del Proyecto Integrador

Esta ficha continúa la FPI-17: convierte varios hallazgos de deuda técnica en atributos de calidad explícitos y verificables.

A. Identificación de la ficha

Código de la ficha	FPI-18
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	Drivers arquitectónicos: atributos de calidad y escenarios verificables

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Recuperar y validar la FPI-17

Deuda técnica relevante (recuperada de la FPI-17)

¿Qué ítems del inventario de deuda técnica son, en realidad, síntomas de un atributo de calidad nunca definido?

Respuesta: DT02 revela metas p95 y aceptación sin protocolo representativo; DT03 revela ausencia de SLO medido. Convertir esos riesgos en escenarios, no en promesas.

¿Algo cambió desde la ficha anterior?

Qué cambió	Por qué cambió	Impacto en el proyecto
Estado del producto	Las16 guías anteriores eran un corte antiguo	Se usa matriz vigente y se conserva historial
Tipo de beneficiario	SaaS multiempresa según estudiante	No se simula una empresa aliada única

Si nada cambió, escribe "Sin cambios" en la primera fila y deja la tabla así.

C. Aplicación específica — Drivers y escenarios

1. Escenarios de calidad verificables (mínimo 3, atributos distintos)

Atributo	Fuente/Estímulo	Artefacto	Entorno	Respuesta	Medida
Seguridad	Usuario solicita empresa ajena	API financiera	Sesión válida de otra empresa	Rechazo sin datos	0 fugas en matriz tenant/rol definida

Rendimiento	20 usuarios consultan	Dashboard	10.000 movimientos/empresa; entorno fijado	Respuesta útil	p95<2s y error<1%; meta sin medir
Mantenibilidad	Adrian cambia regla	Servicio forecast	Desarrollo con fixtures	Regresión conserva contrato	Cambio acotado y tests; plazo meta1jornada

2. Trade-offs identificados

Atributo A vs. Atributo B	Decisión de compromiso	Justificación según el contexto
Seguridad vs latencia	Conservar permisos; optimizar consultas	Datos financieros entre suscriptores
Disponibilidad vs costo	Ensayo y límites explícitos, sin SLA gratis	Presupuesto0USD

3. Priorización de los tres drivers más críticos del MVP

Lista y justifica, en orden, los tres atributos de calidad más importantes para tu MiPyme aliada.

Respuesta: Orden: 1 aislamiento de empresas y confidencialidad; 2 integridad monetaria e idempotencia; 3 mantenibilidad. Clientes potenciales regionales, sin aliado único.

Registro de Decisión Arquitectónica — ADR N.° 02

Título	ADR-02 Drivers de calidad
Estado	Aceptado como registro académico; implementación y brechas descritas por separado
Contexto	SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar. Drivers de calidad
Decisión	Priorizar aislamiento, integridad monetaria y mantenibilidad; medir desempeño antes de prometerlo.
Consecuencias (positivas y negativas)	Menor complejidad y límites explícitos. Requiere medición, aceptación y revisión externa para completar evidencia.

D. Reflexión y trazabilidad

¿Qué aprendimos?

Resume lo aprendido sobre atributos de calidad, escenarios SEI y trade-offs.

Respuesta: RF describe una función, como importar CSV; un atributo de calidad describe bajo qué condiciones

funciona, por ejemplo sin leer otra empresa. En Horizonte la integridad e idempotencia importan tanto como recibir el archivo. Una meta cuantitativa no equivale a resultado medido.

¿Cómo se aplica al proyecto?

Explica cómo estos drivers orientarán las decisiones arquitectónicas de las próximas guías.

Respuesta: Priorizar aislamiento, integridad monetaria y mantenibilidad; medir desempeño antes de prometerlo.

Decisiones tomadas

Decisión	Justificación
Priorizar aislamiento, integridad monetaria y mantenibilidad; medir desempeño antes de prometerlo.	Alineación con riesgo financiero y capacidad individual
Separar evidencia y propuesta	Evitar certificar despliegues, revisiones o mediciones no realizados

Evidencia generada

Estado documentado: Escenarios de calidad de seis partes diligenciados. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Tabla de trade-offs con decisiones justificadas. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Priorización de drivers documentada. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: ADR N.° 02 registrado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Dificultades o ajustes identificados

¿Qué trade-off generó más desacuerdo en el equipo?

Respuesta: El compromiso es p95 frente a seguridad. Adrian conserva controles y mide, sin simular votación de un equipo.

Próximo paso

¿Qué debe hacer el equipo antes de la siguiente guía?

Respuesta: Usar los tres drivers para comparar monolito y distribución en FPI-19.

Conexión con la siguiente guía

Los drivers y escenarios priorizados en esta FPI-18 son el insumo directo de la siguiente guía, donde formalizarás el proceso de toma de decisiones arquitectónicas y consolidarás la plantilla completa de ADR que usarás durante todo el semestre. Si un escenario de calidad resulta impreciso al contrastarlo con una decisión real de arquitectura, deberá corregirse registrando el cambio, no reescribirse en silencio.

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5
Comprensión conceptual (15%)	Confunde requisito funcional con atributo de calidad	Comprende conceptos básicos	Comprende lo esencial	Aplica correctamente ISO 25010	Explica y relaciona los conceptos con profundidad
Calidad de los escenarios SEI (25%)	Escenarios vagos sin medida	Escenarios básicos, medida imprecisa	Escenarios aceptables y verificables	Escenarios completos y bien redactados	Escenarios ejemplares, trazables a la deuda técnica
Análisis de trade-offs (20%)	No identifica trade-offs reales	Identificación superficial	Identifica y justifica un trade-off básico	Justificación sólida con criterio de contexto	Análisis riguroso y bien argumentado
Rigor de la auditoría a la IA (15%)	No audita o copia sin procesar	Críticas superficiales	Detecta vaguedad básica	Auditoría robusta con reescritura correcta	Auditoría excepcional y bien argumentada
Participación en el foro (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada
Trazabilidad respecto a la FPI-17 (15%)	No recupera lo construido antes	Copia sin verificar vigencia	Valida lo esencial	Valida y actualiza correctamente	Trazabilidad ejemplar con causa e impacto de cambios

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%).

Recursos académicos y bibliografía

- ISO/IEC 25010:2023. Systems and software Quality Requirements and Evaluation (SQuaRE).
- Bass, Len; Clements, Paul; Kazman, Rick. Software Architecture in Practice (Software Engineering Institute).
- IEEE Computer Society. SWEBOK Guide, Version 4.0 (Software Architecture).

UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 03 resuelta

Toma de Decisiones Arquitectónicas: Trade-offs y Redacción de ADRs

Asignatura	Ingeniería de Software II
Corte	Primer Corte
Duración	Semana 3
Proyecto integrador	Fase 19: Análisis formal de alternativas arquitectónicas y consolidación de la disciplina de ADRs
Herramientas	Plantilla estándar de ADR, Mermaid/PlantUML + IA generativa auditada
Formato de entrega	Documento con taller resuelto y Ficha de Aplicación al Proyecto Integrador (FPI-19) diligenciada
Fecha límite	Domingo de la Semana 3, 11:59 p.m. (ver fecha exacta en el aula virtual)
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar.
Corte documental 30/10/2026. Se contrastaron código, compose.yaml, .github/workflows/ci.yml y docs/IMPLEMENTATION_STATUS.md. El registro del 18/10 informa 235 pruebas PostgreSQL: 233 aprobadas y 2 omitidas; es evidencia histórica, no una nueva ejecución de esta entrega. No se certifica producción actual, ASVS, p95 representativo ni validación con empresarios. Presupuesto inicial 0 USD y respaldo manual por decisión del proyecto.

Fase 19 del Proyecto Integrador — Evolución del MVP

La arquitectura de software no es un plano inmutable dibujado al inicio del proyecto bajo la fantasía del "Gran Diseño Previo". Se define, de forma más precisa, como el conjunto de decisiones de diseño técnicamente justificadas y registradas que dan forma al sistema. El valor de un arquitecto no está en trazar diagramas que nadie lee, sino en su capacidad de negociar compensaciones entre atributos de calidad en conflicto y registrar esas decisiones de forma ligera para que el sistema pueda evolucionar sin colapsar ante la rotación del equipo o el cambio de prioridades del negocio.

Esta guía retoma los drivers y trade-offs de la FPI-18: ahora los convertirás en decisiones formales, comparando alternativas concretas y registrándolas con la plantilla completa de ADR que usarás durante el resto del semestre.

Resultados de aprendizaje

- Evaluar y comparar alternativas de diseño arquitectónico contrastando trade-offs entre atributos de calidad.
- Documentar decisiones estructurales usando el estándar de Registro de Decisión de Arquitectura (ADR).
- Distinguir un ADR bien redactado (con alternativas consideradas y consecuencias) de uno superficial.
- Auditar críticamente patrones arquitectónicos sugeridos por IA, detectando alucinaciones estructurales y dependencias costosas.

Objetivos

General

Formalizar el proceso de toma de decisiones arquitectónicas del proyecto integrador mediante el análisis comparado de alternativas y su registro riguroso en ADRs.

Específicos

- Explicar por qué un ADR es un registro histórico e inmutable, no un documento que se reescribe.
- Comparar al menos dos alternativas de diseño para una misma decisión, con sus trade-offs.
- Redactar un ADR completo (título, estado, contexto, decisión, alternativas consideradas, consecuencias).
- Auditar una recomendación arquitectónica de IA identificando si está justificada en drivers reales o es una moda tecnológica.

Saberes previos

Antes de iniciar, responde (no se califica):

- ¿Alguna vez tu equipo tomó una decisión técnica sin dejar registro de por qué, y meses después nadie recordaba la razón?

Respuesta del estudiante: Un registro incompleto dificulta recordar motivos; no afirmo una reunión histórica. Los ADR de esta entrega conservan contexto del corte actual.

- En tu FPI-18, ¿qué trade-off te pareció más difícil de resolver?

Respuesta del estudiante: El compromiso costo/disponibilidad: presupuesto cero limita servicio siempre activo. Se registran metas de ensayo y límites.

- ¿Sabes qué significa que una decisión sea "deliberada y prudente" (Guía 1)? Los ADR documentan exactamente ese tipo de decisiones.

Respuesta del estudiante: Sí: deuda consciente con razón, riesgo, responsable y criterio de pago. El ADR conserva también alternativas y consecuencias.

Concepto 1 — El ADR como registro histórico e inmutable

Un Architecture Decision Record (ADR) es un documento corto que captura una decisión estructural importante junto con su contexto y consecuencias. A diferencia de un manual que se actualiza constantemente, un ADR es inmutable una vez aceptado: si la decisión cambia después, no se edita el ADR original, se crea uno nuevo que lo reemplaza y referencia (estado "Reemplazado por ADR-0X"). Esto preserva la

trazabilidad histórica: cualquier persona que se una al equipo puede entender no solo qué se decidió, sino por qué, y qué alternativas se descartaron.

Concepto 2 — Estructura completa de un ADR profesional

Campo	Contenido
Título	Frase corta y descriptiva de la decisión (no del problema).
Estado	Propuesto, Aceptado, Rechazado o Reemplazado.
Contexto	La situación y las fuerzas en juego (drivers arquitectónicos, restricciones de la MiPyme, deuda técnica relevante).
Alternativas consideradas	Al menos dos opciones evaluadas, con sus ventajas y desventajas respecto a los atributos de calidad.
Decisión	Qué se decidió, en una frase clara y directa.
Consecuencias	Efectos positivos y negativos esperados, incluyendo qué trade-off se aceptó.

Un ADR sin alternativas consideradas es solo una afirmación de autoridad ("lo decidimos así porque sí"), no una decisión de ingeniería. La comparación explícita de opciones es lo que convierte un ADR en evidencia de rigor técnico.

Concepto 3 — Comparación de alternativas mediante matriz de decisión

Antes de redactar el ADR, es útil construir una matriz simple que compare cada alternativa contra los drivers de calidad priorizados en la FPI-18: para cada opción, se valora (alto/medio/bajo) su desempeño frente a cada atributo relevante. Esto hace explícito el trade-off y facilita defender la decisión ante el equipo o un evaluador externo.

Concepto 4 — Detectar alucinaciones estructurales de la IA

Al pedir a una IA que sugiera o compare patrones arquitectónicos, es frecuente que proponga soluciones populares en la industria global (microservicios, serverless, colas de mensajería distribuidas) sin considerar el contexto real de una MiPyme pequeña. Esto no es un error de sintaxis: es una alucinación estructural, una recomendación técnicamente plausible pero desconectada del contexto real del proyecto.

Regla de oro de esta guía: *"Ninguna decisión arquitectónica se acepta sin al menos dos alternativas comparadas explícitamente contra los drivers de calidad reales del proyecto."*

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, por qué un ADR debe tratarse como un registro inmutable y no como un documento que se edita libremente.

Respuesta del estudiante: Un ADR aceptado conserva el contexto que justificó una decisión. Si cambia la decisión, creo un ADR sucesor y marco el anterior reemplazado, conservando enlaces. Los borradores sí pueden corregirse; inmutabilidad histórica no significa mantener errores tipográficos como si fueran decisiones.

Ejercicio 2

Para un caso ficticio (elegir entre almacenamiento local en archivo o una base de datos relacional para una tienda pequeña), redacta un ADR completo con los 6 campos del Concepto 2.

Respuesta del estudiante: ADR ficticio: persistencia de pedidos. Estado: propuesto. Contexto: tienda necesita consultas, concurrencia e integridad. Decisión: SQLite inicial con migración prevista a PostgreSQL. Consecuencias: transacciones y consultas sencillas, límites de concurrencia. Alternativas: JSON descartado por actualización concurrente e integridad; servidor gestionado aplazado por costo. Seis campos completos; no es decisión productiva de Horizonte.

Nivel Intermedio (matriz de decisión + auditoría a la IA)

Ejercicio 3

Construye una matriz de decisión (alto/medio/bajo) comparando dos alternativas arquitectónicas reales de tu proyecto contra tus tres drivers priorizados en la FPI-18.

Respuesta del estudiante: Matriz Horizonte, en orden seguridad/integridad, mantenibilidad y costo operativo: monolito modular = alto/alto/alto; microservicios por dominio = medio/medio/bajo para un estudiante. Alto significa mejor adecuación, no cantidad de tecnología. La red y transacciones distribuidas incrementan superficie y operación.

Ejercicio 4

Pide a una IA generativa que recomiende una arquitectura para tu MVP sin darle el contexto de que es una MiPyme pequeña con recursos limitados. Audita: ¿propuso una solución sobredimensionada (microservicios, colas distribuidas, nube multiregión) sin justificarla en tus drivers reales? Documenta el prompt, la respuesta y tu corrección.

Respuesta del estudiante: Prompt: recomienda arquitectura para una plataforma financiera sin información de escala. Respuesta de esta IA: faltan usuarios, carga, presupuesto y responsables; usaría provisionalmente monolito modular y cola para importaciones. Auditoría: no hubo recomendación de microservicios que rechazar. Completo contexto: SaaS académico, presupuesto cero y un estudiante; ratifico módulos Django y servicios puros. No se fabrica una alucinación para cumplir el ejercicio.

Lista de autoevaluación antes de entregar

- Entiendo por qué un ADR es un registro inmutable, no un documento editable.
- Mi ADR compara al menos dos alternativas reales, no solo justifica una decisión ya tomada.
- Construí una matriz de decisión trazable a mis drivers de la FPI-18.
- Detecté si una IA propuso una alucinación estructural (arquitectura sobredimensionada) para mi contexto.
- Diligencié completamente la FPI-19 con mi ADR formal.

Foro de reflexión

Título: *"¿Documentar decisiones es burocracia o es protección del proyecto?"*

Redacte una reflexión de 250–300 palabras respondiendo: ¿qué riesgos concretos corre un proyecto cuando las decisiones arquitectónicas importantes se toman verbalmente y nunca se documentan? ¿Cómo un ADR bien escrito protege al equipo cuando cambia de integrantes o cuando el negocio cuestiona una decisión meses después? Apoye su respuesta con un ejemplo de su propio proyecto o del taller.

Reflexión del estudiante (282 palabras)
En Horizonte, considero que registrar monolito modular y comparar alternativas mediante drivers, con ADR sucesor para cambios. Un ADR aceptado conserva el contexto que justificó una decisión. Si cambia la decisión, creo un ADR sucesor y marco el anterior reemplazado, conservando enlaces. Los borradores sí pueden corregirse; inmutabilidad histórica no significa mantener errores tipográficos como si fueran decisiones.

El caso concreto que orienta mi análisis es el siguiente: Matriz Horizonte, en orden seguridad/integridad, mantenibilidad y costo operativo: monolito modular = alto/alto/alto; microservicios por dominio = medio/medio/bajo para un estudiante. Alto significa mejor adecuación, no cantidad de tecnología. La red y transacciones distribuidas incrementan superficie y operación.

Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia.

La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados.

Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones. Si falta medición, registro el protocolo y la brecha; no transformo una propuesta en un éxito histórico. Así puedo defender el trabajo y decidir el siguiente incremento con evidencia.

Cuestionario

Seleccione una única respuesta.

1. Un ADR se considera un registro:

- A. Editable libremente en cualquier momento sin dejar rastro.
- B. Histórico e inmutable; si la decisión cambia, se crea un nuevo ADR que reemplaza al anterior. ✅
- C. Que solo puede redactar el profesor del curso.
- D. Opcional, sin relación con la calidad del proyecto.

2. Un ADR sin alternativas consideradas se asemeja a:

- A. Una decisión de ingeniería rigurosa y completa.
- B. Una afirmación de autoridad sin evidencia de análisis comparativo real. ✅
- C. El formato obligatorio exigido por ISO 25010.
- D. Un requisito funcional del sistema.

3. El campo "Consecuencias" de un ADR debe incluir:

- A. Solo los efectos positivos de la decisión.
- B. Los efectos positivos y negativos esperados, incluyendo el trade-off aceptado. ✅
- C. El nombre del cliente que aprobó la decisión.
- D. El código fuente completo de la implementación.

4. Que una IA recomiende microservicios y colas distribuidas para el MVP de una MiPyme pequeña sin justificarlo en drivers reales es un ejemplo de:

- A. Una recomendación técnica impecable que debe aceptarse sin más análisis.
- B. Una alucinación estructural: una sugerencia plausible pero desconectada del contexto real del proyecto. ✅
- C. Una violación de la Ley 1581 de 2012.
- D. El único patrón arquitectónico válido en la industria actual.

5. Una matriz de decisión que compara alternativas contra los drivers de calidad sirve principalmente para:

- A. Sustituir la necesidad de escribir el ADR.
- B. Hacer explícito el trade-off entre opciones antes de tomar y documentar la decisión final. ✅
- C. Decidir el nombre del repositorio de GitHub.
- D. Eliminar la necesidad de considerar el contexto de la MiPyme.

6. Según la regla de oro de esta guía, una decisión arquitectónica se acepta solo cuando:

- A. El integrante más experimentado del equipo la propone sin más análisis.
- B. Se compararon explícitamente al menos dos alternativas contra los drivers de calidad reales. ✅
- C. La IA la sugirió primero.
- D. No existe ninguna alternativa posible.

7. El estado "Reemplazado" en un ADR indica que:

- A. El ADR se eliminó del repositorio.
- B. La decisión original ya no aplica y fue sustituida por un nuevo ADR que la referencia. ✅
- C. El ADR nunca fue aprobado formalmente.
- D. El ADR corresponde a una decisión de código, no de arquitectura.

Cuestionario resuelto: 1. B; 2. B; 3. B; 4. B; 5. B; 6. B; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-19

Análisis de Alternativas y ADR Formal del Proyecto Integrador

Esta ficha continúa la FPI-18: convierte los drivers priorizados en una decisión arquitectónica formal, comparada y documentada.

A. Identificación de la ficha

Código de la ficha	FPI-19
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	Toma de decisiones arquitectónicas: trade-offs y ADRs

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Recuperar y validar la FPI-18

Drivers priorizados (recuperados de la FPI-18)

Copia aquí los tres drivers de calidad priorizados en la FPI-18. Respuesta: Aislamiento e integridad; mantenibilidad; costo operativo compatible con presupuesto cero y trabajo individual.

¿Algo cambió desde la ficha anterior?

Qué cambió	Por qué cambió	Impacto en el proyecto
Estado del producto	Las16 guías anteriores eran un corte antiguo	Se usa matriz vigente y se conserva historial
Tipo de beneficiario	SaaS multiempresa según estudiante	No se simula una empresa aliada única

Si nada cambió, escribe "Sin cambios" en la primera fila y deja la tabla así.

C. Aplicación específica — Matriz de decisión y ADR

1. Matriz de decisión (mínimo 2 alternativas)

Alternativa	Driver 1 (alto/medio/bajo)	Driver 2 (alto/medio/bajo)	Driver 3 (alto/medio/bajo)
Monolito modular	Alto	Alto	Alto

Microservicios	Medio	Medio	Bajo

Registro de Decisión Arquitectónica — ADR N.° 03

Título	ADR-03 Registro ADR
Estado	Aceptado como registro académico; implementación y brechas descritas por separado
Contexto	SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar. Registro ADR
Decisión	Registrar monolito modular y comparar alternativas mediante drivers, con ADR sucesor para cambios.
Consecuencias (positivas y negativas)	Menor complejidad y límites explícitos. Requiere medición, aceptación y revisión externa para completar evidencia.
Alternativas consideradas y por qué se descartaron	Se descarta introducir tecnología o certificación no justificada; se conserva solución más simple que satisface drivers. Matriz Horizonte, en orden seguridad/integridad, mantenibilidad y costo operativo: monolito modular = alto/alto/alto; microservicios por dominio = medio/medio/bajo para un estudiante. Alto significa mejor adecuación, no cantidad de tecnología. La red y transacciones distribuidas incrementan superficie y operación.

D. Reflexión y trazabilidad

¿Qué aprendimos?

Resume lo aprendido sobre la estructura de un ADR y el análisis de alternativas.

Respuesta: Un ADR aceptado conserva el contexto que justificó una decisión. Si cambia la decisión, creo un ADR sucesor y marco el anterior reemplazado, conservando enlaces. Los borradores sí pueden corregirse; inmutabilidad histórica no significa mantener errores tipográficos como si fueran decisiones.

¿Cómo se aplica al proyecto?

Explica qué decisión arquitectónica concreta tomó el equipo y cómo la defendería ante un evaluador externo.

Respuesta: Registrar monolito modular y comparar alternativas mediante drivers, con ADR sucesor para cambios.

Decisiones tomadas

Decisión	Justificación
Registrar monolito modular y comparar alternativas mediante drivers, con ADR sucesor para cambios.	Alineación con riesgo financiero y capacidad individual

Separar evidencia y propuesta	Evitar certificar despliegues, revisiones o mediciones no realizados

Evidencia generada

Estado documentado: Matriz de decisión diligenciada. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: ADR N.° 03 completo, con alternativas consideradas. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Dificultades o ajustes identificados

¿Qué alternativa fue más difícil de descartar? ¿Por qué?

Respuesta: La opción más atractiva pero costosa fue microservicios; faltan carga y equipo que justifiquen su operación.

Próximo paso

¿Qué debe hacer el equipo antes de la siguiente guía?

Respuesta: Aplicar el ADR al límite de cálculo y persistencia en FPI-20.

Conexión con la siguiente guía

La decisión registrada en el ADR N.° 03 de esta FPI-19 es la antesala directa de la siguiente guía, donde el equipo elegirá formalmente su estilo arquitectónico (capas, hexagonal, monolito modular) y comenzará a refactorizar el código real del MVP hacia esa estructura. Si el estilo elegido contradice alguna alternativa ya descartada aquí, el equipo deberá explicar qué cambió.

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5
Comprensión conceptual del ADR (15%)	Confunde el ADR con documentación general	Comprende conceptos básicos	Comprende lo esencial	Aplica correctamente la estructura de 6 campos	Explica y relaciona los conceptos con profundidad
Rigor de la matriz de decisión (25%)	No compara alternativas reales	Comparación superficial	Comparación aceptable con criterios básicos	Comparación rigurosa trazable a los drivers	Matriz ejemplar y bien argumentada

Calidad del ADR redactado (25%)	ADR incompleto o sin alternativas	ADR básico, alternativas débiles	ADR aceptable con los 6 campos	ADR completo, claro y bien justificado	ADR de nivel profesional
Rigor de la auditoría a la IA (15%)	No audita o copia sin procesar	Críticas superficiales	Detecta alucinaciones básicas	Auditoría robusta con corrección concreta	Auditoría excepcional y bien argumentada
Participación en el foro (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada
Trazabilidad respecto a la FPI-18 (10%)	No recupera lo construido antes	Copia sin verificar vigencia	Valida lo esencial	Valida y actualiza correctamente	Trazabilidad ejemplar con causa e impacto de cambios

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%).

Recursos académicos y bibliografía

- Nygard, Michael. Documenting Architecture Decisions (origen del formato ADR).
- Bass, Len; Clements, Paul; Kazman, Rick. Software Architecture in Practice.
- IEEE Computer Society. SWEBOK Guide, Version 4.0 (Software Architecture).

UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 04 resuelta

Estilos Arquitectónicos: Capas, Arquitectura Hexagonal, Clean Architecture y Monolito Modular

Asignatura	Ingeniería de Software II
Corte	Primer Corte
Duración	Semana 4
Proyecto integrador	Fase 20: Refactorización estructural del MVP hacia un estilo arquitectónico justificado
Herramientas	Mermaid/PlantUML, IDE, Git + IA generativa auditada
Formato de entrega	Documento con taller resuelto, Pull Request de refactorización y Ficha de Aplicación al Proyecto Integrador (FPI-20) diligenciada
Fecha límite	Domingo de la Semana 4, 11:59 p.m. (ver fecha exacta en el aula virtual)
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar.
Corte documental 30/10/2026. Se contrastaron código, compose.yaml, .github/workflows/ci.yml y docs/IMPLEMENTATION_STATUS.md. El registro del 18/10 informa 235 pruebas PostgreSQL: 233 aprobadas y 2 omitidas; es evidencia histórica, no una nueva ejecución de esta entrega. No se certifica producción actual, ASVS, p95 representativo ni validación con empresarios. Presupuesto inicial 0 USD y respaldo manual por decisión del proyecto.

Fase 20 del Proyecto Integrador — Evolución del MVP

El ADR N.° 03 de la FPI-19 formalizó cómo tu equipo toma decisiones estructurales. Ahora aplicarás esa disciplina a la decisión arquitectónica más importante del semestre: qué estilo le dará forma al núcleo de tu MVP. La arquitectura no es una colección fortuita de código, sino un conjunto de decisiones dirigidas a satisfacer los atributos de calidad priorizados en la FPI-18, bajo restricciones reales de costo y tiempo.

Esta guía retoma el inventario de deuda técnica de la FPI-17 (probablemente marcado con alto acoplamiento) y los drivers de la FPI-18: elegirás y justificarás un estilo arquitectónico, y refactorizarás al menos un módulo real de tu MVP hacia esa estructura, registrando el resultado en la FPI-20.

Resultados de aprendizaje

- Diferenciar la Arquitectura por Capas, la Arquitectura Hexagonal (Puertos y Adaptadores) y el Monolito Modular.
- Aplicar el patrón de Puertos y Adaptadores para desacoplar el núcleo de negocio de la infraestructura externa.
- Justificar la elección de un estilo arquitectónico según los drivers de calidad del proyecto, evitando la sobre-ingeniería.
- Refactorizar un módulo real del MVP hacia el estilo elegido, validando que el núcleo de negocio quede libre de dependencias de infraestructura.
- Auditar críticamente una refactorización arquitectónica sugerida por IA generativa.

Objetivos

General

Refactorizar estructuralmente un módulo crítico del MVP hacia un estilo arquitectónico justificado, desacoplando el núcleo de negocio de la infraestructura externa.

Específicos

- Explicar la regla de dependencia unidireccional de la Arquitectura por Capas y su principal limitación.
- Explicar el concepto de Puerto (interfaz del núcleo) y Adaptador (implementación externa) en Arquitectura Hexagonal.
- Justificar cuándo el Monolito Modular es preferible a fragmentar el sistema prematuramente.
- Aplicar principios SOLID (Guía 11 de Software I) durante la refactorización hexagonal.
- Registrar la decisión de estilo arquitectónico y su evidencia de refactorización en la FPI-20.

Saberes previos

Antes de iniciar, responde (no se califica):

- ¿Recuerdas los principios SOLID, en particular la Inversión de Dependencias (Guía 11 de Software I)?

Respuesta del estudiante: Sí: el núcleo no debe depender de detalles reemplazables de infraestructura.

- ¿Tu controlador HTTP actual llama directamente a la base de datos, o pasa por alguna capa intermedia?

Respuesta del estudiante: Las vistas usan ORM y llaman servicios. project_obligations es puro; otras composiciones de dashboard tienen acceso ORM.

- ¿Cuántos archivos o clases estarías dispuesto a agregar a tu proyecto para ganar mantenibilidad, antes de sentir que es demasiado?

Respuesta del estudiante: Solo añado abstracciones si reducen variación, riesgo o costo de prueba. No hay número universal de clases aceptable.

Concepto 1 — Arquitectura por Capas

Organiza el sistema en niveles jerárquicos (Presentación, Lógica de Negocio, Acceso a Datos), bajo la regla de dependencia unidireccional: una capa superior solo depende de las inferiores, nunca al revés. Su principal limitación es el alto acoplamiento hacia la base de datos: si el modelo de persistencia cambia, el efecto suele propagarse hacia arriba.

Concepto 2 — Arquitectura Hexagonal (Puertos y Adaptadores)

Diseñada por Alistair Cockburn, sitúa las reglas de negocio en el centro. El núcleo define interfaces abstractas (Puertos) para la entrada (ej. casos de uso) y la salida (ej. interfaces de persistencia). Los componentes externos (Adaptadores: controladores REST, adaptadores de base de datos SQL/NoSQL) implementan o consumen esos puertos, aislando el núcleo de la infraestructura.

> *Analogía territorial: una máquina automatizada de mezclado de arcilla en una ladrillera de Villa del Rosario tiene el motor y el algoritmo de mezcla como núcleo de negocio. Para recibir energía, cuenta con un puerto universal (enchufe estándar). Si la conectamos a la red eléctrica de la CENS o a una planta diésel (adaptadores), el núcleo de mezcla funciona exactamente igual: es inmune a las variaciones del proveedor de energía.*

Concepto 3 — Monolito Modular y Clean Architecture

El Monolito Modular agrupa módulos de negocio bien delimitados dentro de un único proceso de ejecución. Es una alternativa altamente recomendada frente a microservicios prematuros, pues reduce la complejidad operativa de red y latencia manteniendo alta cohesión y bajo acoplamiento. Clean Architecture (Robert C. Martin) extiende la lógica hexagonal en círculos concéntricos: si una regla de negocio de una Entidad cambia, el efecto puede propagarse a los Casos de Uso y potencialmente a las capas externas, pero nunca al revés.

Concepto 4 — Evitar la sobre-ingeniería arquitectónica

Implementar puertos y adaptadores para un MVP con tres funcionalidades básicas puede duplicar el número de archivos y clases sin necesidad real. La decisión de estilo arquitectónico debe evaluarse con el mismo criterio de la Guía 3: comparando alternativas contra los drivers de calidad priorizados, no adoptando el patrón "correcto según el libro" sin medir el costo-beneficio real para la MiPyme.

Regla de oro de esta guía: *"El estilo arquitectónico se elige por sus atributos de calidad ganados frente a su sobrecosto de archivos y complejidad, nunca por moda."*

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, la diferencia entre un Puerto y un Adaptador en Arquitectura Hexagonal.

Respuesta del estudiante: Puerto es el contrato que expresa una capacidad del dominio; adaptador realiza ese contrato usando infraestructura. En Horizonte una fuente de movimientos podría ser puerto y CSV o Mock Bank sus adaptadores. No hace falta una interfaz por cada modelo ORM.

Ejercicio 2

Para un caso ficticio (un controlador Flask que hace SELECT directos a SQL Server), identifica qué principio SOLID se viola y qué puerto/adaptador propondrías para corregirlo.

Respuesta del estudiante: El controlador depende de detalles SQL y mezcla transporte, consulta y negocio; se afecta inversión de dependencias y responsabilidad única. Propongo puerto obtener_pedido y adaptador SQL parametrizado, más servicio de aplicación; la vista solo valida HTTP y serializa.

Nivel Intermedio (refactorización real + auditoría a la IA)

Ejercicio 3

Toma el fragmento de código legado que auditaste en la FPI-17 (o uno similar de tu MVP real) y diseña los puertos (interfaces) necesarios para separar el núcleo de negocio de la persistencia.

Respuesta del estudiante: Para cálculo contractual basta una función pura con balance, corte, horizonte y obligaciones. Las consultas Company/Obligation quedan fuera. El servicio existente project_obligations ya usa esa separación. Para importación variable propongo un puerto leer_movimientos y adaptadores CSV/XLSX; no añado repositorio ficticio para cada entidad.

Ejercicio 4

Pide a una IA generativa que refactorice ese fragmento hacia Arquitectura Hexagonal. Audita: ¿las interfaces son estrictamente necesarias o introducen sobre-ingeniería para un caso tan simple? ¿el núcleo quedó realmente libre de importaciones de infraestructura (ej. excepciones de SQLAlchemy)? Documenta el prompt, la respuesta y tu corrección.

Respuesta del estudiante: Prompt: simplifica project_obligations sin ORM y conserva Decimal y contrato. Respuesta: sustituir el if de inclusión por guard clause y mantener agregación por fecha. Auditoría: usa únicamente timedelta y Decimal, no importa Django ni excepciones SQL. La versión académica se conserva como propuesta; no se edita el backend con esta solicitud documental.

Ejercicio 5

Ejecuta pruebas unitarias sobre el núcleo de negocio refactorizado sin necesidad de levantar la base de datos real. Documenta el resultado.

Respuesta del estudiante: Se ejecuta una comprobación aislada de equivalencia del cálculo contractual, sin BD, con horizontes 30/60/90 y entradas vencidas/futuras. Resultado disponible en evidencia_academica.json. Verifica la variante didáctica y el servicio importado; no equivale a integración de un PR ni aceptación del producto.

Lista de autoevaluación antes de entregar

- Diferencio con claridad Capas, Hexagonal y Monolito Modular.
- Mi refactorización desacopla realmente el núcleo de negocio de la infraestructura.
- Verifiqué que las interfaces creadas sean necesarias y no sobre-ingeniería.
- Audité si la IA dejó fugas de infraestructura dentro del núcleo.
- Diligencié completamente la FPI-20 con evidencia real de un Pull Request de refactorización.

Foro de reflexión

Título: *"La paradoja de la flexibilidad arquitectónica: ¿es la Arquitectura Hexagonal un suicidio económico para una MiPyme?"*

Redacte una reflexión de 250–300 palabras respondiendo: para un negocio tradicional que factura con un MVP básico, ¿justifican los atributos de mantenibilidad y testabilidad el incremento en archivos y tiempo de desarrollo que implementa puertos y adaptadores? Argumente con base en el cuadrante de deuda técnica de Fowler (Guía 1) y en los drivers de su propio proyecto (FPI-18).

Reflexión del estudiante (287 palabras)
En Horizonte, considero que mantener servicios de cálculo puros y aislar infraestructura en límites con variación real. Puerto es el contrato que expresa una capacidad del dominio; adaptador realiza ese contrato usando infraestructura. En Horizonte una fuente de movimientos podría ser puerto y CSV o Mock Bank sus adaptadores. No hace falta una interfaz por cada modelo ORM.

El caso concreto que orienta mi análisis es el siguiente: Para cálculo contractual basta una función pura con balance, corte, horizonte y obligaciones. Las consultas Company/Obligation quedan fuera. El servicio existente project_obligations ya usa esa separación. Para importación variable propongo un puerto leer_movimientos y adaptadores CSV/XLSX; no añado repositorio ficticio para cada entidad.

Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia.

La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados.

Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones. Si falta medición, registro el protocolo y la brecha; no transformo una propuesta en un éxito histórico. Así puedo defender el trabajo y decidir el siguiente incremento con evidencia.

Cuestionario

Seleccione una única respuesta.

1. La diferencia fundamental entre un Puerto y un Adaptador en Arquitectura Hexagonal es que:

- A. El Puerto maneja protocolos de red y el Adaptador define lógica de base de datos.
- B. El Puerto es una interfaz abstracta que pertenece al núcleo; el Adaptador implementa o consume dicha interfaz desde la infraestructura. ✅
- C. El Puerto es un componente visual y el Adaptador uno de persistencia.
- D. No hay diferencia técnica entre ambos términos.

2. La principal limitación de la Arquitectura por Capas tradicional es:

- A. No permite el uso de bases de datos relacionales.
- B. El alto acoplamiento hacia la base de datos, que propaga cambios hacia las capas superiores. ✅
- C. Prohíbe el uso de controladores REST.
- D. Solo puede usarse en aplicaciones móviles.

3. El Monolito Modular es preferible a los microservicios prematuros porque:

- A. Elimina por completo la necesidad de pruebas.
- B. Reduce la complejidad operativa de red y latencia manteniendo alta cohesión y bajo acoplamiento. ✅
- C. No requiere control de versiones.

- D. Es obligatorio según ISO/IEC 25010.

4. En Clean Architecture, si una regla de negocio de una Entidad cambia, el efecto:

- A. Nunca se propaga a ninguna otra capa.
- B. Puede propagarse hacia los Casos de Uso y potencialmente las capas externas, pero nunca al revés. ✅
- C. Solo afecta a la base de datos.
- D. Elimina automáticamente los adaptadores existentes.

5. Implementar puertos y adaptadores para un MVP de tres funcionalidades básicas sin justificación real de crecimiento futuro es un ejemplo de:

- A. Buena práctica obligatoria en todo proyecto.
- B. Posible sobre-ingeniería arquitectónica, según el criterio de la Guía 3. ✅
- C. Una violación de la Ley 1581 de 2012.
- D. La única forma válida de aplicar SOLID.

6. Después de una refactorización hexagonal correcta, el núcleo de negocio debería:

- A. Importar directamente excepciones y clases del ORM de base de datos.
- B. Estar completamente libre de dependencias de infraestructura, comunicándose solo a través de sus puertos. ✅
- C. Depender del framework web utilizado.
- D. Ejecutarse únicamente dentro de un contenedor Docker.

7. Según la regla de oro de esta guía, un estilo arquitectónico se elige por:

- A. Ser el más popular en la industria en el momento.
- B. Sus atributos de calidad ganados frente a su sobrecosto de archivos y complejidad. ✅
- C. La preferencia personal del integrante más experimentado, sin más análisis.
- D. Ser el único mencionado en el SWEBOK v4.

Cuestionario resuelto: 1. B; 2. B; 3. B; 4. B; 5. B; 6. B; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-20

Refactorización Estructural del Proyecto Integrador

Esta ficha continúa la FPI-19: convierte la decisión del ADR N.° 03 en una refactorización real del código del MVP.

A. Identificación de la ficha

Código de la ficha	FPI-20
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	Estilos arquitectónicos: Capas, Hexagonal, Clean Architecture y Monolito Modular

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Recuperar y validar la FPI-19

Decisión arquitectónica (recuperada del ADR N.° 03 de la FPI-19)

Copia aquí el estilo arquitectónico decidido y su justificación resumida. Respuesta: Registrar monolito modular y comparar alternativas mediante drivers, con ADR sucesor para cambios.

¿Algo cambió desde la ficha anterior?

Qué cambió	Por qué cambió	Impacto en el proyecto
Estado del producto	Las16 guías anteriores eran un corte antiguo	Se usa matriz vigente y se conserva historial
Tipo de beneficiario	SaaS multiempresa según estudiante	No se simula una empresa aliada única

Si nada cambió, escribe "Sin cambios" en la primera fila y deja la tabla así.

C. Aplicación específica — Refactorización

1. Diagrama de contenedores y componentes (núcleo, puertos y adaptadores)

Describe o adjunta (Mermaid) el diagrama del módulo refactorizado. Respuesta: React -> API Django -> servicios de aplicación -> ORM PostgreSQL; project_obligations recibe

valores sin ORM. Puerto futuro BankSource -> adaptadores CSV/XLSX/Mock. Las consultas tenant permanecen en backend.

2. Evidencia del Pull Request de refactorización

Enlace al PR en GitHub con el código refactorizado.

Respuesta: Propuesta aislada en esta guía; no hay PR integrado generado por esta tarea. Repositorio conocido: https://github.com/elcreado/Horizonte .

3. Auditoría a la IA en la refactorización

Elemento auditado	Hallazgo y corrección del equipo
¿Las interfaces creadas son estrictamente necesarias o hay sobre-ingeniería?	Se ejecuta una comprobación aislada de equivalencia del cálculo contractual, sin BD, con horizontes 30/60/90 y entradas vencidas/futuras. Resultado disponible en evidencia_academica.json. Verifica la variante didáctica y el servicio importado; no equivale a integración de un PR ni aceptación del producto.
¿El núcleo quedó libre de importaciones de infraestructura?	Se contrasta contra código y resultados, no contra confianza del LLM. Mantener servicios de cálculo puros y aislar infraestructura en límites con variación real.

Registro de Decisión Arquitectónica — ADR N.° 04

Título	ADR-04 Monolito modular
Estado	Aceptado como registro académico; implementación y brechas descritas por separado
Contexto	SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar. Monolito modular
Decisión	Mantener servicios de cálculo puros y aislar infraestructura en límites con variación real.
Consecuencias (positivas y negativas)	Menor complejidad y límites explícitos. Requiere medición, aceptación y revisión externa para completar evidencia.

D. Reflexión y trazabilidad

¿Qué aprendimos?

Resume lo aprendido sobre los estilos arquitectónicos trabajados.

Respuesta: Puerto es el contrato que expresa una capacidad del dominio; adaptador realiza ese contrato usando

infraestructura. En Horizonte una fuente de movimientos podría ser puerto y CSV o Mock Bank sus adaptadores. No hace falta una interfaz por cada modelo ORM.

¿Cómo se aplica al proyecto?

Explica qué mantenibilidad ganó el MVP tras la refactorización.

Respuesta: El cálculo puro existente permite pruebas sin BD. La variante académica demuestra equivalencia; no se atribuye mejora productiva no integrada.

Decisiones tomadas

Decisión	Justificación
Mantener servicios de cálculo puros y aislar infraestructura en límites con variación real.	Alineación con riesgo financiero y capacidad individual
Separar evidencia y propuesta	Evitar certificar despliegues, revisiones o mediciones no realizados

Evidencia generada

Estado documentado: Diagrama de componentes. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Pull Request de refactorización fusionado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: ADR N.° 04 registrado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Dificultades o ajustes identificados

¿Qué parte del código fue más difícil de desacoplar?

Respuesta: Separar autorización y persistencia de vistas amplias sin omitir filtro por empresa.

Próximo paso

¿Qué debe hacer el equipo antes de la siguiente guía?

Respuesta: Delimitar contextos y contratos antes de distribuir.

Conexión con la siguiente guía

El núcleo desacoplado en esta FPI-20 es la base para evaluar, en la siguiente guía, si algún módulo de tu sistema justifica distribuirse en un servicio independiente o si debe permanecer en el monolito modular.

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5
Comprensión conceptual (15%)	Confunde los estilos entre sí	Comprende conceptos básicos	Comprende lo esencial	Diferencia correctamente los estilos	Explica y relaciona con profundidad
Calidad de la refactorización real (30%)	No refactoriza o rompe el sistema	Refactorización parcial con fugas de infraestructura	Refactorización aceptable, núcleo mayormente aislado	Núcleo completamente aislado y probado	Refactorización ejemplar, sin fugas y con pruebas verdes
Justificación del estilo elegido (15%)	No justifica o copia sin análisis	Justificación superficial	Justificación aceptable con drivers básicos	Justificación sólida trazable a drivers	Justificación rigurosa y bien argumentada
Rigor de la auditoría a la IA (15%)	No audita o copia sin procesar	Críticas superficiales	Detecta sobre-ingeniería básica	Auditoría robusta con corrección concreta	Auditoría excepcional y bien argumentada
Participación en el foro (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada
Trazabilidad respecto a la FPI-19 (15%)	No recupera lo construido antes	Copia sin verificar vigencia	Valida lo esencial	Valida y actualiza correctamente	Trazabilidad ejemplar con causa e impacto de cambios

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%).

Recursos académicos y bibliografía

- Cockburn, Alistair. Hexagonal Architecture (Ports and Adapters).
- Martin, Robert C. Clean Architecture: A Craftsman's Guide to Software Structure and Design.
- IEEE Computer Society. SWEBOK Guide, Version 4.0 (Software Architecture).

Comprobaciones de esta entrega
30/10/2026:26 comprobaciones del cálculo contractual aprobaron (18 equivalencias y8 rechazos de horizonte), sin BD. La prueba real tests.test_application.ApplicationTests.test_company_isolation también pasó sobre BD temporal. La variante académica no se integró al backend. Evidencias: _trabajo/evidencia_academica.json y prueba_aislamiento.log.

UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 05 resuelta

Distribución y Acoplamiento: Microservicios, APIs y el Debate de la Distribución Prematura

Asignatura	Ingeniería de Software II
Corte	Primer Corte (Cierre de Corte)
Duración	Semana 5
Proyecto integrador	Fase 21: Análisis de viabilidad de distribución y mapa de Bounded Contexts del MVP
Herramientas	C4 Model, Mermaid (Context Map) + IA generativa auditada
Formato de entrega	Documento con taller resuelto y Ficha de Aplicación al Proyecto Integrador (FPI-21) diligenciada
Fecha límite	Domingo de la Semana 5, 11:59 p.m. (ver fecha exacta en el aula virtual)
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar.
Corte documental 30/10/2026. Se contrastaron código, compose.yaml, .github/workflows/ci.yml y docs/IMPLEMENTATION_STATUS.md. El registro del 18/10 informa 235 pruebas PostgreSQL: 233 aprobadas y 2 omitidas; es evidencia histórica, no una nueva ejecución de esta entrega. No se certifica producción actual, ASVS, p95 representativo ni validación con empresarios. Presupuesto inicial 0 USD y respaldo manual por decisión del proyecto.

Fase 21 del Proyecto Integrador — Evolución del MVP

En la Guía 4 elegiste un estilo arquitectónico y desacoplaste el núcleo de negocio de la infraestructura dentro de un mismo proceso. La pregunta de esta semana es distinta y mucho más riesgosa: ¿deberías ir un paso más allá y fragmentar tu sistema en servicios distribuidos físicamente? El patrón de microservicios domina el discurso de la industria, pero la evidencia demuestra que distribuir bases de datos e infraestructura antes de alcanzar una escala adecuada es uno de los mayores generadores de fracaso técnico y financiero en las empresas pequeñas.

Esta guía retoma el monolito modular refactorizado en la FPI-20: en vez de fragmentarlo a ciegas, identificarás sus límites de negocio (Bounded Contexts) y decidirás, con evidencia y no con moda, qué módulos —si acaso alguno— justifican distribuirse. El resultado se registra en la FPI-21.

Resultados de aprendizaje

- Explicar el Principio de Distribución Prematura de Fowler y Newman.
- Identificar Bounded Contexts (límites de negocio) mediante Diseño Guiado por el Dominio (DDD).
- Analizar las consecuencias técnicas del patrón Database-per-Service.
- Evaluar la viabilidad financiera y de latencia real de una propuesta de arquitectura distribuida para una MiPyme.
- Auditar críticamente una arquitectura de microservicios sugerida por IA, detectando sobrecostos y riesgos de red.

Objetivos

General

Analizar con rigor científico la viabilidad de distribuir el MVP, identificando sus Bounded Contexts antes de tomar cualquier decisión de partición física.

Específicos

- Explicar por qué la distribución física introduce latencia, consistencia eventual y fallos parciales de red.
- Delimitar al menos tres Bounded Contexts del dominio del MVP.
- Calcular el costo real de infraestructura distribuida frente al margen operativo de la MiPyme.
- Justificar, para cada módulo, si permanece en el monolito o se distribuye, con un atributo de calidad concreto.
- Registrar el mapa de contextos y la decisión de distribución en la FPI-21.

Saberes previos

Antes de iniciar, responde (no se califica):

- ¿Has escuchado hablar de "microservicios"? ¿Qué crees que gana y qué crees que pierde un sistema al dividirse así?

Respuesta del estudiante: Microservicios permiten despliegue separado pero agregan red, consistencia y observación. Horizonte no necesita repartir cada módulo en un servidor.

- ¿Cuántos módulos de negocio distintos tiene tu MVP actualmente (ej. facturación, inventario, clientes)?

Respuesta del estudiante: Cinco módulos Django principales: accounts, banking, classify, invoices y forecast; no son cinco microservicios.

- ¿Qué tan estable es la conexión a internet de la organización beneficiaria de tu proyecto?

Respuesta del estudiante: No hay empresa aliada única ni medición de conexión del segmento. Debo considerar conectividad variable y reintentos sin duplicado.

Concepto 1 — El Principio de Distribución Prematura

Martin Fowler y Sam Newman definen que la primera ley de la distribución de objetos es: "no distribuyas tus objetos". La distribución física introduce de forma ineludible problemas de latencia, consistencia eventual,

fallos parciales de red y la necesidad de transacciones distribuidas o sagas de compensación. Ninguno de estos problemas existe en un monolito modular bien diseñado.

> *Analogía territorial: una distribuidora de calzado en Cúcuta debe enviar 100 cajas de zapatos escolares a Venezuela por la aduana de La Parada. La opción monolítica es cargar todo en un solo camión articulado: un viaje, una aduana, cero coordinación en la vía. La opción distribuida es enviar a 100 motorizados independientes, uno por caja: si una moto pincha, las otras 99 llegan, pero coordinar a 100 operarios, pagar 100 peajes y conciliar 100 trámites aduaneros introduce un caos logístico insostenible para un negocio pequeño de calzado.*

Concepto 2 — Diseño Orientado al Dominio (DDD) y Bounded Contexts

DDD define que un sistema debe dividirse según límites del lenguaje de negocio y de los equipos de desarrollo (Bounded Contexts). Cada contexto delimitado posee su propio modelo de datos exclusivo y, en principio, no debe cruzar claves foráneas físicas directas con otros esquemas. Identificar estos límites —incluso si el sistema sigue siendo un monolito— es la preparación necesaria para cualquier distribución futura ordenada.

Concepto 3 — El patrón Database-per-Service y sus consecuencias

Si un módulo se distribuye con su propia base de datos, se pierde la integridad referencial y las transacciones ACID directas entre tablas de diferentes servicios, requiriendo consistencia eventual y mecanismos de compensación (sagas). Este costo técnico debe compararse honestamente contra el beneficio real que la distribución aportaría — no asumirse como gratuito.

Concepto 4 — Arquitecturas Orientadas a Eventos (EDA) como puente

Si en el futuro se justifica distribuir un contexto, las Arquitecturas Orientadas a Eventos permiten el acoplamiento asíncrono y la consistencia eventual entre contextos delimitados mediante la emisión de Eventos de Dominio (hechos inmutables del pasado del negocio), evitando el acoplamiento síncrono directo entre servicios.

Regla de oro de esta guía: *"Ningún módulo se distribuye solo porque 'así se hace en la industria'; se distribuye únicamente cuando el costo de mantenerlo en el monolito supera, con evidencia real, el costo operativo de la distribución."*

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, qué es un Bounded Context y por qué es útil identificarlo incluso si el sistema sigue siendo un monolito.

Respuesta del estudiante: Un Bounded Context delimita reglas y vocabulario coherentes. En un monolito permite evitar que factura signifique automáticamente dinero cobrado. Horizonte separa bancos, facturación, clasificación y proyección, aunque se desplieguen juntos.

Ejercicio 2

Para un caso ficticio (una distribuidora con módulos de Ventas, Inventario y Envíos), identifica los límites de cada Bounded Context y qué términos de negocio son exclusivos de cada uno.

Respuesta del estudiante: Ventas: pedido, precio pactado y cliente comprador. Inventario: existencia, reserva y ubicación. Envíos: guía, entrega y destinatario. Una venta confirmada solicita reserva; una reserva satisfecha habilita envío. Cada contexto publica identificadores y eventos, no comparte tablas sin contrato.

Nivel Intermedio (viabilidad financiera + auditoría a la IA)

Ejercicio 3

Calcula (de forma aproximada) el costo mensual de una infraestructura distribuida en la nube (ej. funciones serverless + una base de datos por servicio) y compáralo contra el margen operativo típico de una MiPyme con ventas de 500 USD mensuales.

Respuesta del estudiante: Supuesto didáctico, no cotización: 3 servicios a 7 USD y 3 BD a 7 USD más 10 USD de operación = 52 USD/mes. Con ventas 500 USD y margen supuesto 20%, utilidad previa 100 USD; 52 consume 52% de esa utilidad. Monolito supuesto 17 USD consume 17%. El margen real es desconocido; ventas no son utilidad. Horizonte mantiene presupuesto 0 USD y acepta límites de capas gratuitas.

Ejercicio 4

Pide a una IA generativa que diseñe una arquitectura de microservicios distribuidos en la nube para un negocio pequeño con bajo volumen de transacciones. Audita: ¿consideró el costo real de la infraestructura propuesta? ¿asumió una red de internet local perfectamente confiable? Documenta el prompt, la respuesta y tu evaluación.

Respuesta del estudiante: Prompt: diseña distribución para un negocio de bajo volumen y explicita costo y red. Respuesta: mantener monolito y procesamiento asíncrono; separar solo ante necesidad demostrada. Auditoría: no asumir red estable, usar reintentos idempotentes; no inventar tarifas actuales ni SLA gratis. La comparación anterior es análisis de sensibilidad con supuestos.

Ejercicio 5

Diseña un diagrama Mermaid de Context Map para tu propio proyecto, mostrando los Bounded Contexts identificados y sus relaciones.

Respuesta del estudiante: Context Map textual y Mermaid: graph LR; Acceso-->Banca; Banca-->Clasificacion; Facturacion-->Obligaciones; Obligaciones-->Proyeccion; Banca-->Proyeccion. Cada flecha significa datos mediante servicios internos. Factura pendiente y movimiento liquidado se concilian para evitar doble conteo. No hay servicio que transfiera fondos.

Artefacto académico completo — propuesta o ejemplo según respuesta anterior

```
graph LR
  Acceso --> Banca
  Banca --> Clasificacion
  Banca --> Proyeccion
  Facturacion --> Obligaciones
  Obligaciones --> Proyeccion
```

Lista de autoevaluación antes de entregar

- Identifiqué al menos tres Bounded Contexts reales de mi proyecto, no inventados.
- Calculé, de forma aproximada, el costo real de una alternativa distribuida para mi MiPyme.
- Entiendo las consecuencias del patrón Database-per-Service (pérdida de transacciones ACID directas).

- Audité si una IA propuso una arquitectura distribuida sin justificar el costo ni la latencia real.
- Diligencié completamente la FPI-21 con una decisión de distribución justificada módulo por módulo.

Foro de reflexión

Título: *"El Hype-Driven Architecture (HDA): ¿es el abuso de microservicios el mayor destructor de MiPymes en Latinoamérica?"*

Redacte una reflexión de 250–300 palabras respondiendo: ¿por qué muchos equipos eligen microservicios distribuidos para proyectos pequeños a pesar de que la sobrecarga de red y la complejidad operativa suelen destruir el ROI del negocio? Fundamente su postura con evidencia de su propio análisis de costos (Ejercicio 3).

Reflexión del estudiante (272 palabras)
En Horizonte, considero que conservar cuentas, clasificación, facturas y proyección en el monolito; workers no equivalen a microservicios. Un Bounded Context delimita reglas y vocabulario coherentes. En un monolito permite evitar que factura signifique automáticamente dinero cobrado. Horizonte separa bancos, facturación, clasificación y proyección, aunque se desplieguen juntos. El caso concreto que orienta mi análisis es el siguiente: Supuesto didáctico, no cotización: 3 servicios a 7 USD y 3 BD a 7 USD más 10 USD de operación = 52 USD/mes. Con ventas 500 USD y margen supuesto 20%, utilidad previa 100 USD; 52 consume 52% de esa utilidad. Monolito supuesto 17 USD consume 17%. El margen real es desconocido; ventas no son utilidad. Horizonte mantiene presupuesto 0 USD y acepta límites de capas gratuitas. Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia. La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados. Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones.

Cuestionario

Seleccione una única respuesta.

1. La primera ley de la distribución de objetos, según Fowler y Newman, es:

- A. "Distribuye siempre que puedas para ganar escalabilidad."
- B. "No distribuyas tus objetos": la distribución introduce latencia, consistencia eventual y fallos parciales de red. ✅
- C. "Toda base de datos debe ser independiente desde el primer día."
- D. "Los microservicios siempre reducen la complejidad operativa."

2. Un Bounded Context, en DDD, se define como:

- A. Un límite geográfico de red configurado en el firewall corporativo.
- B. El espacio conceptual y lingüístico donde un modelo de dominio y sus términos tienen un significado consistente. ✅
- C. Un tipo de contenedor Docker con límites estrictos de CPU.
- D. El total de líneas de código permitidas por clase.

3. La consecuencia técnica ineludible del patrón Database-per-Service es:

- A. Las consultas JOIN tradicionales se aceleran automáticamente.
- B. Se pierde la integridad referencial y las transacciones ACID directas entre servicios, requiriendo consistencia eventual. ✅
- C. Las bases de datos se sincronizan solas sin ningún broker.
- D. Todos los servicios deben usar el mismo motor de persistencia.

4. Las Arquitecturas Orientadas a Eventos (EDA) permiten:

- A. Eliminar por completo la necesidad de bases de datos.
- B. El acoplamiento asíncrono y la consistencia eventual entre contextos mediante eventos de dominio inmutables. ✅
- C. Sustituir la necesidad de un modelo de dominio.
- D. Garantizar consistencia ACID entre microservicios distribuidos.

5. Proponer una arquitectura serverless de $150 USD mensuales para un negocio que factura $500 USD totales es un ejemplo de:

- A. Una recomendación técnica impecable sin más análisis necesario.
- B. Una alucinación de viabilidad financiera que debe auditarse antes de aceptarla. ✅
- C. El estándar recomendado por ISO/IEC 25010.
- D. Una arquitectura obligatoria para cualquier MVP.

6. Identificar los Bounded Contexts de un sistema es útil incluso si permanece como monolito porque:

- A. No tiene ninguna utilidad si no se distribuye.
- B. Prepara ordenadamente al sistema para una eventual distribución futura, delimitando modelos y lenguaje de negocio. ✅
- C. Es un requisito exclusivo de arquitecturas serverless.
- D. Elimina la necesidad de un catálogo de requisitos.

7. Según la regla de oro de esta guía, un módulo se distribuye solo cuando:

- A. Es la tendencia más popular del momento en la industria.
- B. El costo de mantenerlo en el monolito supera, con evidencia real, el costo operativo de distribuirlo. ✅
- C. El equipo quiere aprender una tecnología nueva.
- D. El profesor lo exige sin más justificación.

Cuestionario resuelto: 1. B; 2. B; 3. B; 4. B; 5. B; 6. B; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-21

Mapa de Bounded Contexts y Decisión de Distribución del Proyecto Integrador

Esta ficha continúa la FPI-20: analiza el monolito modular refactorizado e identifica sus límites de negocio antes de considerar cualquier distribución física.

A. Identificación de la ficha

Código de la ficha	FPI-21
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	Distribución y acoplamiento: microservicios y Bounded Contexts

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Recuperar y validar la FPI-20

Estructura refactorizada (recuperada de la FPI-20)

Resume el estilo arquitectónico y los módulos principales del monolito modular actual.

Respuesta: Monolito Django con accounts, banking, classify, invoices y forecast; React como cliente y Celery para trabajo asíncrono.

¿Algo cambió desde la ficha anterior?

Qué cambió	Por qué cambió	Impacto en el proyecto
Estado del producto	Las16 guías anteriores eran un corte antiguo	Se usa matriz vigente y se conserva historial
Tipo de beneficiario	SaaS multiempresa según estudiante	No se simula una empresa aliada única

Si nada cambió, escribe "Sin cambios" en la primera fila y deja la tabla así.

C. Aplicación específica — Distribución

1. Diagrama de Bounded Contexts (Context Map)

Adjunta o describe el diagrama Mermaid con los contextos delimitados del proyecto.

Respuesta: Context Map textual y Mermaid: graph LR; Acceso-->Banca; Banca-->Clasificacion; Facturacion-->Obligaciones; Obligaciones-->Proyeccion; Banca-->Proyeccion. Cada flecha significa datos mediante servicios

internos. Factura pendiente y movimiento liquidado se concilian para evitar doble conteo. No hay servicio que transfiera fondos.

2. Tabla de decisiones de distribución

Módulo de dominio	¿Sigue en el monolito? (Sí/No)	Atributo de calidad que lo justifica
Acceso	Sí	Integridad y menor operación; límites por servicios
Banca	Sí	Integridad y menor operación; límites por servicios
Clasificación	Sí	Integridad y menor operación; límites por servicios
Facturación	Sí	Integridad y menor operación; límites por servicios
Proyección	Sí	Integridad y menor operación; límites por servicios

Registro de Decisión Arquitectónica — ADR N.° 05

Título	ADR-05 Contextos de dominio
Estado	Aceptado como registro académico; implementación y brechas descritas por separado
Contexto	SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar. Contextos de dominio
Decisión	Conservar cuentas, clasificación, facturas y proyección en el monolito; workers no equivalen a microservicios.
Consecuencias (positivas y negativas)	Menor complejidad y límites explícitos. Requiere medición, aceptación y revisión externa para completar evidencia.
Análisis de costo operativo local	Conservar cuentas, clasificación, facturas y proyección en el monolito; workers no equivalen a microservicios.

D. Reflexión y trazabilidad

¿Qué aprendimos?

Resume lo aprendido sobre distribución prematura y Bounded Contexts.

Respuesta: Un Bounded Context delimita reglas y vocabulario coherentes. En un monolito permite evitar que factura signifique automáticamente dinero cobrado. Horizonte separa bancos, facturación, clasificación y proyección, aunque se desplieguen juntos.

¿Cómo se aplica al proyecto?

Explica si el equipo decidió distribuir algún módulo y por qué. Respuesta: Conservar cuentas, clasificación, facturas y proyección en el monolito; workers no equivalen a microservicios.

Decisiones tomadas

Decisión	Justificación
Conservar cuentas, clasificación, facturas y proyección en el monolito; workers no equivalen a microservicios.	Alineación con riesgo financiero y capacidad individual
Separar evidencia y propuesta	Evitar certificar despliegues, revisiones o mediciones no realizados

Evidencia generada

Estado documentado: Diagrama de Bounded Contexts. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Tabla de decisiones de distribución. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: ADR N.° 05 registrado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Dificultades o ajustes identificados

¿Qué módulo fue más difícil de delimitar? Respuesta: Facturas y obligaciones: una factura pendiente no equivale a entrada de caja; conciliación evita duplicados.

Próximo paso

¿Qué debe hacer el equipo antes de la siguiente guía? Respuesta: Inventariar contratos y errores de API en FPI-22.

Conexión con la siguiente guía

Los límites de contexto delimitados aquí definen las fronteras de tus futuras APIs: en la siguiente guía diseñarás el contrato formal (OpenAPI) de las interfaces entre estos contextos, sea que permanezcan en el monolito o eventualmente se distribuyan.

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5
Comprensión conceptual (15%)	Confunde monolito con distribución sin criterio	Comprende conceptos básicos	Comprende lo esencial	Aplica correctamente DDD y distribución	Explica y relaciona con profundidad
Calidad del mapa de Bounded Contexts (25%)	No delimita contextos reales	Delimitación superficial	Delimitación aceptable con límites básicos	Mapa claro y coherente con el dominio	Mapa ejemplar y riguroso
Rigor del análisis de viabilidad financiera (20%)	No calcula costos reales	Cálculo superficial sin comparación	Cálculo aceptable con comparación básica	Análisis sólido trazable al presupuesto real	Análisis riguroso y bien argumentado
Rigor de la auditoría a la IA (15%)	No audita o copia sin procesar	Críticas superficiales	Detecta riesgos básicos	Auditoría robusta con corrección concreta	Auditoría excepcional y bien argumentada
Participación en el foro (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada
Trazabilidad respecto a la FPI-20 (15%)	No recupera lo construido antes	Copia sin verificar vigencia	Valida lo esencial	Valida y actualiza correctamente	Trazabilidad ejemplar con causa e impacto de cambios

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%).

Recursos académicos y bibliografía

- Newman, Sam. Building Microservices.
- Evans, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software.
- Fowler, Martin. MonolithFirst (martinfowler.com).
- Forsgren, Nicole; Humble, Jez; Kim, Gene. Accelerate.

UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 06 resuelta

Ingeniería de APIs: Diseño de Contratos (OpenAPI), REST Profesional y Seguridad con JWT

Asignatura	Ingeniería de Software II
Corte	Segundo Corte
Duración	Semana 6
Proyecto integrador	Fase 22: Diseño del contrato OpenAPI y esquema de seguridad JWT de la API del MVP
Herramientas	Swagger Editor, Postman, JWT.io + IA generativa auditada
Formato de entrega	Documento con taller resuelto y Ficha de Aplicación al Proyecto Integrador (FPI-22) diligenciada
Fecha límite	Domingo de la Semana 6, 11:59 p.m. (ver fecha exacta en el aula virtual)
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar.
Corte documental 30/10/2026. Se contrastaron código, compose.yaml, .github/workflows/ci.yml y docs/IMPLEMENTATION_STATUS.md. El registro del 18/10 informa 235 pruebas PostgreSQL: 233 aprobadas y 2 omitidas; es evidencia histórica, no una nueva ejecución de esta entrega. No se certifica producción actual, ASVS, p95 representativo ni validación con empresarios. Presupuesto inicial 0 USD y respaldo manual por decisión del proyecto.

Fase 22 del Proyecto Integrador — Evolución del MVP

En las guías anteriores decidiste la estructura interna de tu sistema y sus límites de contexto. Ahora formalizarás cómo ese sistema se comunica con el mundo exterior: su API. Un mal diseño de API, sin contratos estandarizados ni seguridad robusta, no solo bloquea el trabajo colaborativo con otros equipos (frontend, integraciones externas), sino que expone el sistema a riesgos catastróficos de seguridad.

Esta guía retoma los Bounded Contexts de la FPI-21: diseñarás el contrato formal de al menos un endpoint por contexto relevante, antes de escribir el código del controlador, siguiendo el enfoque API Design-First. El resultado se registra en la FPI-22.

Resultados de aprendizaje

- Aplicar el enfoque API Design-First definiendo el contrato antes del código.
- Diseñar especificaciones OpenAPI 3.0 completas y validadas sin advertencias.
- Aplicar el Modelo de Madurez de Richardson para diseñar APIs REST de nivel 2 (verbos y códigos de estado correctos).
- Configurar un esquema de seguridad con JSON Web Tokens (JWT).
- Auditar críticamente un esquema OpenAPI generado por IA, detectando fugas de privacidad y errores de semántica HTTP.

Objetivos

General

Diseñar y asegurar el contrato de API del MVP siguiendo el enfoque API Design-First, el Modelo de Madurez de Richardson y autenticación JWT.

Específicos

- Explicar los cuatro niveles del Modelo de Madurez de Richardson.
- Diseñar al menos tres endpoints en OpenAPI 3.0 con esquemas de datos completos.
- Configurar el esquema de seguridad global JWT en la especificación.
- Diseñar un esquema de respuesta pública que excluya datos personales sensibles.
- Registrar el contrato de API y el ADR de seguridad en la FPI-22.

Saberes previos

Antes de iniciar, responde (no se califica):

- ¿Recuerdas los verbos HTTP (GET, POST, PUT, DELETE) y qué código de estado corresponde a un error del cliente vs. del servidor?

Respuesta del estudiante: GET consulta; POST crea/ejecuta operación; PUT reemplaza; DELETE elimina según contrato.4xx refiere solicitud/autorización y5xx fallo servidor.

- ¿Has usado Postman o una herramienta similar para probar una API?

Respuesta del estudiante: El proyecto tiene pruebas con APIClient y recorridos HTTP documentados. No atribuyo uso de una colección Postman no disponible.

- ¿Tu API actual devuelve datos sensibles del cliente (cédula, teléfono) sin ningún filtro?

Respuesta del estudiante: No se auditó exhaustivamente toda respuesta para certificar ausencia de PII. Se restringe acceso por empresa y se minimizan campos; falta revisión transversal.

Concepto 1 — API Design-First

El contrato de la interfaz (archivo OpenAPI en YAML/JSON) se define, revisa y aprueba antes de escribir una sola línea de código en el backend. Esto permite el desarrollo paralelo de los equipos frontend y de pruebas, que pueden mockear las respuestas y programar sin esperar a que el backend esté terminado.

> *Analogía territorial: cuando un transportador de calzado llega al puesto de control fronterizo en La Parada, el agente de aduana no improvisa los requisitos en el momento. Existe una planilla estandarizada (el Contrato OpenAPI) que define exactamente qué campos son obligatorios: número de lote, peso, origen, NIT. Si la planilla viene mal diligenciada, el sistema rechaza el camión de inmediato con un mensaje claro (HTTP 400 Bad Request) antes de inspeccionar la carga física.*

Concepto 2 — Modelo de Madurez de Richardson

Nivel	Descripción
Nivel 0	HTTP usado solo como túnel de transporte (ej. un único POST para todo, al estilo RPC).
Nivel 1	Mapeo de entidades a URIs específicas (ej. /zapatos/1, /despachos).
Nivel 2	Uso correcto y semántico de los métodos HTTP (GET, POST, PUT/PATCH, DELETE) y códigos de estado (200, 201, 400, 401, 403, 404, 500). Es el nivel esperado en esta guía.
Nivel 3 (HATEOAS)	El servidor indica los recursos relacionados y transiciones de estado permitidas desde el estado actual.

Concepto 3 — Seguridad con JSON Web Tokens (JWT)

JWT (RFC 7519) es un estándar abierto para la transmisión segura de "claims" auto-contenidos, firmados digitalmente por un proveedor de identidad. Un endpoint protegido debe declarar explícitamente su esquema de seguridad; nunca debe asumirse que un endpoint es seguro "por defecto" si no se configuró de forma explícita.

Concepto 4 — Privacidad en los esquemas de respuesta

Un esquema de respuesta que retorna la dirección, cédula y teléfono de un cliente sin ofuscación, incluso en un endpoint autenticado, es una violación del principio de mínimo privilegio y de la Ley 1581 de 2012. La práctica correcta es definir esquemas de respuesta públicos separados (ej. DespachoPublicResponse) que excluyan o censuren los datos sensibles según quién consulta la información.

Regla de oro de esta guía: *"Ningún endpoint se expone sin declarar explícitamente su esquema de seguridad, y ningún esquema de respuesta expone datos personales sin una razón de negocio justificada."*

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, qué significa que una API alcance el Nivel 2 del Modelo de Madurez de Richardson.

Respuesta del estudiante: Nivel 2 usa recursos identificados por URI, verbos HTTP y estados correctos. No exige HATEOAS. Horizonte usa GET para consultas y POST para crear/importar, con sesión y CSRF; la elección de autenticación es independiente del nivel Richardson.

Ejercicio 2

Para un caso ficticio (gestión de pedidos de una farmacia), diseña 3 endpoints (crear, consultar, listar) indicando verbo HTTP, URI y códigos de estado esperados.

Respuesta del estudiante: Farmacia ficticia: POST /pedidos/ crea, devuelve 201 o 400; GET /pedidos/{id}/ devuelve 200 o 404; GET /pedidos/?page=1 lista con 200. En todos, falta de autenticación 401 si usa bearer y permiso insuficiente 403. POST incluye body JSON, GET no.

Nivel Intermedio (OpenAPI + seguridad + auditoría a la IA)

Ejercicio 3

Redacta el fragmento OpenAPI 3.0 (YAML) para uno de los endpoints del Ejercicio 2, incluyendo el esquema de datos completo.

Respuesta del estudiante: Especificación OpenAPI 3.0 completa del endpoint ficticio incluida en bloque YAML del taller. Incluye identificador, descripción, cantidad y estado, campos requeridos, respuesta 201 y errores. Es ejemplo académico, no endpoint real de Horizonte.

Ejercicio 4

```
security:
  - bearerAuth: []
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
```

Incorpora un bloque de seguridad como el anterior a tu especificación y explica en qué endpoints lo aplicarías y en cuáles no (ej. un endpoint de login no requiere el token que aún no existe).

Respuesta del estudiante: El ejemplo de farmacia usa bearerAuth HTTP bearer JWT en pedidos y security: [] en login. Para Horizonte el contrato debe describir cookie sessionid y CSRF en escritura. No marco JWT en endpoints que realmente usan sesión ni expongo documentos personales en respuestas.

Ejercicio 5

Pide a una IA generativa que genere la especificación OpenAPI de tus endpoints reales. Audita: ¿configuró el esquema de seguridad de forma explícita? ¿el esquema de respuesta expone datos personales sin ofuscación? ¿usó semántica correcta de verbos (nunca un GET con cuerpo, ni un POST para actualizar)? Documenta el prompt, la respuesta y tu corrección.

Respuesta del estudiante: Prompt: documenta el dashboard real con seguridad y errores sin inventar campos. Respuesta: GET /api/companies/{company_id}/dashboard/?horizon=30, sesión, horizonte enum 30/60/90; 200, 400 inválido, 403 sin sesión, 404 empresa ajena, 409 sin saldo/cortes incompatibles. Auditoría: views.py permite además composición actual del Resumen; inventario no es contrato completo. No adjuntar cédulas, claves ni tokens de ejemplo reales.

Artefacto académico completo — propuesta o ejemplo según respuesta anterior
openapi: 3.0.3
info: {title: Pedidos farmacia ejemplo, version: 1.0.0}

```
paths:
  /pedidos/:
    post:
      security: [{bearerAuth: []}]
      requestBody:
        required: true
        content:
          application/json:
            schema: {$ref: '#/components/schemas/NuevoPedido'}
      responses:
        '201':
          description: Creado
          content:
            application/json:
              schema: {$ref: '#/components/schemas/Pedido'}
        '400': {description: Datos inválidos}
        '401': {description: Sin autenticación}
        '403': {description: Sin permiso}
components:
  securitySchemes:
    bearerAuth: {type: http, scheme: bearer, bearerFormat: JWT}
  schemas:
    NuevoPedido:
      type: object
      required: [descripcion, cantidad]
      additionalProperties: false
      properties:
        descripcion: {type: string, minLength: 1, maxLength: 200}
        cantidad: {type: integer, minimum: 1}
    Pedido:
      allOf:
        - {$ref: '#/components/schemas/NuevoPedido'}
        - type: object
          required: [id, estado]
          properties:
            id: {type: integer, minimum: 1}
            estado: {type: string, enum: [recibido, preparado, entregado]}
# Ejemplo ficticio. Horizonte usa sesión y CSRF; no JWT.
```

Lista de autoevaluación antes de entregar

- Diseñé el contrato de mi API antes de escribir el código del controlador (Design-First).
- Mi API alcanza el Nivel 2 de Richardson: verbos y códigos de estado correctos.
- Configuré explícitamente el esquema de seguridad JWT en los endpoints que lo requieren.
- Mis esquemas de respuesta no exponen datos personales sensibles sin justificación.
- Diligencié completamente la FPI-22 con mi contrato OpenAPI validado.

Foro de reflexión

Título: *"¿Es HATEOAS un adorno académico purista o aporta valor real de ingeniería?"*

Redacte una reflexión de 250–300 palabras respondiendo: implementar HATEOAS (Nivel 3 de Richardson) requiere que el backend devuelva links dinámicos, aumentando la complejidad del payload y el parseo en el

frontend. ¿Vale la pena este esfuerzo en el contexto de su proyecto integrador? Argumente con base científica y con el contexto real de su MiPyme.

Reflexión del estudiante (274 palabras)
En Horizonte, considero que mantener sesión Django y CSRF para la web; documentar OpenAPI sin introducir JWT innecesario. Nivel 2 usa recursos identificados por URI, verbos HTTP y estados correctos. No exige HATEOAS. Horizonte usa GET para consultas y POST para crear/importar, con sesión y CSRF; la elección de autenticación es independiente del nivel Richardson.

El caso concreto que orienta mi análisis es el siguiente: Especificación OpenAPI 3.0 completa del endpoint ficticio incluida en bloque YAML del taller. Incluye identificador, descripción, cantidad y estado, campos requeridos, respuesta 201 y errores. Es ejemplo académico, no endpoint real de Horizonte.

Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia.

La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados.

Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones. Si falta medición, registro el protocolo y la brecha; no transformo una propuesta en un éxito histórico. Así puedo defender el trabajo y decidir el siguiente incremento con evidencia.

Cuestionario

Seleccione una única respuesta.

1. Si una API responde con `403 Forbidden` tras recibir una petición autenticada con un token válido, esto significa:

- A. El token ha expirado o su firma es inválida.
- B. El cliente se autenticó correctamente, pero no posee los permisos requeridos para el recurso (falta de autorización). ✅
- C. El recurso solicitado no existe en el servidor.
- D. La base de datos externa está caída.

2. El principal beneficio del enfoque API Design-First es:

- A. Permite escribir código más rápido sin planificar ni documentar.
- B. Al definir el contrato antes, el equipo frontend puede mockear respuestas y trabajar en paralelo con el backend. ✅
- C. Reduce el consumo de memoria RAM del servidor.
- D. Garantiza código libre de fallos de lógica de negocio automáticamente.

3. Un API en Nivel 0 del Modelo de Madurez de Richardson se caracteriza por:

- A. Usar URIs específicas para cada recurso.
- B. Usar HTTP solo como mecanismo de túnel, similar a RPC sobre un único POST. ✅
- C. Implementar HATEOAS completo.
- D. Usar exclusivamente el verbo GET.

4. Declarar `security: []` (vacío) en un endpoint de administración expone el sistema porque:

- A. Obliga a usar HTTPS de forma redundante.
- B. Deja el endpoint sin ningún esquema de seguridad, accesible sin autenticación. ✅
- C. Es la forma correcta de proteger un endpoint sensible.
- D. Solo afecta a los endpoints de tipo GET.

5. Retornar la cédula y el teléfono de un cliente sin ofuscación en la respuesta pública de un endpoint es un riesgo relacionado con:

- A. El Modelo de Madurez de Richardson exclusivamente.
- B. La privacidad de datos personales bajo la Ley 1581 de 2012. ✅
- C. Un error de sintaxis en el archivo YAML.
- D. El versionamiento semántico de la API.

6. Usar el verbo `POST` para actualizar un recurso existente en vez de `PUT`/`PATCH` es un ejemplo de:

- A. Buena práctica recomendada en el Nivel 2 de Richardson.
- B. Una desviación de la semántica correcta de verbos HTTP. ✅
- C. Un requisito obligatorio de JWT.
- D. La forma correcta de implementar HATEOAS.

7. Según la regla de oro de esta guía, un endpoint:

- A. Es seguro por defecto aunque no se declare ningún esquema.
- B. Nunca se expone sin declarar explícitamente su esquema de seguridad. ✅
- C. Solo necesita seguridad si maneja pagos.
- D. No requiere ningún esquema OpenAPI si el equipo es pequeño.

Cuestionario resuelto: 1. B; 2. B; 3. B; 4. B; 5. B; 6. B; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-22

Contrato de API y Seguridad JWT del Proyecto Integrador

Esta ficha continúa la FPI-21: diseña el contrato formal de al menos un endpoint por cada Bounded Context relevante.

A. Identificación de la ficha

Código de la ficha	FPI-22
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	Ingeniería de APIs: contratos OpenAPI, REST y seguridad JWT

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Recuperar y validar la FPI-21

Bounded Contexts (recuperados de la FPI-21)

Copia aquí los contextos delimitados para los que diseñarás endpoints en esta ficha.

Respuesta: Acceso, empresas, banca/importaciones, clasificación y proyección; priorizar dashboard y cargas idempotentes.

¿Algo cambió desde la ficha anterior?

Qué cambió	Por qué cambió	Impacto en el proyecto
Estado del producto	Las16 guías anteriores eran un corte antiguo	Se usa matriz vigente y se conserva historial
Tipo de beneficiario	SaaS multiempresa según estudiante	No se simula una empresa aliada única

Si nada cambió, escribe "Sin cambios" en la primera fila y deja la tabla así.

C. Aplicación específica — Contrato de API

1. Enlace al contrato OpenAPI 3.0 validado

Ruta del archivo openapi.yaml en el repositorio.

Respuesta: Contrato existente parcial e inventario docs/openapi-inventory.json. No afirmar openapi.yaml completo; verificar ruta vigente en repositorio antes de publicar contrato.

2. Endpoints diseñados

Verbo HTTP	URI	Códigos de estado esperados	¿Requiere JWT?
GET	/api/companies/	200,403	No; sesión
GET	/api/companies/{id}/dashboard/	200,400,403,404,409	No; sesión
POST	/api/companies/{id}/imports/	202,400,403,404,503	No; sesión y CSRF

3. Evidencia de validación en Postman/Swagger Editor

Captura o enlace a la colección de Postman con al menos 3 endpoints probados.

Respuesta: Se usan APIClient y evidencia de QA HTTP documentada; no hay colección Postman creada en esta entrega. Comandos y tests identificados en FPI-23.

Registro de Decisión Arquitectónica — ADR N.° 06

Título	ADR-06 Contrato API
Estado	Aceptado como registro académico; implementación y brechas descritas por separado
Contexto	SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar. Contrato API
Decisión	Mantener sesión Django y CSRF para la web; documentar OpenAPI sin introducir JWT innecesario.
Consecuencias (positivas y negativas)	Menor complejidad y límites explícitos. Requiere medición, aceptación y revisión externa para completar evidencia.

D. Reflexión y trazabilidad

¿Qué aprendimos?

Resume lo aprendido sobre API Design-First, Richardson y JWT.

Respuesta: Nivel 2 usa recursos identificados por URI, verbos HTTP y estados correctos. No exige HATEOAS. Horizonte usa GET para consultas y POST para crear/importar, con sesión y CSRF; la elección de autenticación es independiente del nivel Richardson.

¿Cómo se aplica al proyecto?

Explica cómo el contrato de API facilitará el trabajo del resto del semestre. Respuesta: Mantener sesión Django y CSRF para la web; documentar OpenAPI sin introducir JWT innecesario.

Decisiones tomadas

Decisión	Justificación
Mantener sesión Django y CSRF para la web; documentar OpenAPI sin introducir JWT innecesario.	Alineación con riesgo financiero y capacidad individual
Separar evidencia y propuesta	Evitar certificar despliegues, revisiones o mediciones no realizados

Evidencia generada

Estado documentado: Contrato OpenAPI validado sin advertencias. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Colección de Postman con pruebas. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: ADR N.° 06 de seguridad registrado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Dificultades o ajustes identificados

¿Qué endpoint fue más difícil de asegurar correctamente? Respuesta: Asegurar objetos por empresa y revalidar tarea asíncrona; sesión no sustituye autorización.

Próximo paso

¿Qué debe hacer el equipo antes de la siguiente guía? Respuesta: Probar contrato con BD temporal y errores esperados.

Conexión con la siguiente guía

El contrato de API validado aquí es el que someterás a pruebas de integración automatizadas en la siguiente guía, verificando que la implementación real cumpla exactamente lo que este contrato promete.

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5

Comprensión conceptual (15%)	Confunde verbos y códigos de estado	Comprende conceptos básicos	Comprende lo esencial	Aplica correctamente Richardson Nivel 2	Explica y relaciona con profundidad
Calidad del contrato OpenAPI (25%)	No valida o tiene errores graves	Contrato básico con advertencias	Contrato válido con desviaciones menores	Contrato completo y correcto	Contrato ejemplar, sin advertencias
Configuración de seguridad JWT (20%)	Sin esquema de seguridad	Seguridad parcial o mal aplicada	Seguridad aceptable en los endpoints críticos	Seguridad bien aplicada y justificada	Seguridad de nivel profesional
Rigor de la auditoría a la IA (15%)	No audita o copia sin procesar	Críticas superficiales	Detecta errores básicos	Auditoría robusta con corrección concreta	Auditoría excepcional y bien argumentada
Participación en el foro (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada
Trazabilidad respecto a la FPI-21 (15%)	No recupera lo construido antes	Copia sin verificar vigencia	Valida lo esencial	Valida y actualiza correctamente	Trazabilidad ejemplar con causa e impacto de cambios

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%).

Recursos académicos y bibliografía

- Fielding, Roy. Architectural Styles and the Design of Network-based Software Architectures.
- OpenAPI Initiative. OpenAPI Specification 3.0.
- RFC 7519. JSON Web Token (JWT).
- IEEE Computer Society. SWEBOK Guide, Version 4.0 (Software Construction).

UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 07 resuelta

Estrategia y Pirámide de Pruebas: Integración, Contrato (Pact) y Aceptación

Asignatura	Ingeniería de Software II
Corte	Segundo Corte
Duración	Semana 7
Proyecto integrador	Fase 23: Suite de pruebas de integración aisladas y validación de criterios de aceptación del MVP
Herramientas	pytest-mock/Mockito, Pact Framework, Postman/Supertest + IA generativa auditada
Formato de entrega	Documento con taller resuelto y Ficha de Aplicación al Proyecto Integrador (FPI-23) diligenciada
Fecha límite	Domingo de la Semana 7, 11:59 p.m. (ver fecha exacta en el aula virtual)
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar.
Corte documental 30/10/2026. Se contrastaron código, compose.yaml, .github/workflows/ci.yml y docs/IMPLEMENTATION_STATUS.md. El registro del 18/10 informa 235 pruebas PostgreSQL: 233 aprobadas y 2 omitidas; es evidencia histórica, no una nueva ejecución de esta entrega. No se certifica producción actual, ASVS, p95 representativo ni validación con empresarios. Presupuesto inicial 0 USD y respaldo manual por decisión del proyecto.

Fase 23 del Proyecto Integrador — Evolución del MVP

En Software I aprendiste a probar unidades de código en aislamiento. En el mundo real, los sistemas fallan principalmente en los puntos de fricción e integración entre componentes: bases de datos, llamadas de red externas o contratos con terceros desalineados. Una suite repleta de pruebas unitarias verdes puede darte una falsa sensación de seguridad si esos puntos de integración no se prueban con el mismo rigor.

Esta guía retoma el contrato OpenAPI de la FPI-22: verificarás mediante pruebas automatizadas de integración que la implementación real de tu API cumple exactamente lo que ese contrato promete, con dependencias externas correctamente aisladas. El resultado se registra en la FPI-23.

Resultados de aprendizaje

- Diseñar una estrategia de pruebas balanceada según la Pirámide de Cohn.
- Aislar dependencias externas (APIs de terceros) en pruebas de integración usando mocks de infraestructura.
- Explicar el propósito de las Pruebas de Contrato (Consumer-Driven Contracts) con Pact.
- Automatizar la validación de criterios de Pruebas de Aceptación (UAT) definidos con historias de usuario.
- Auditar críticamente pruebas de integración generadas por IA, detectando fugas hacia bases de datos reales.

Objetivos

General

Diseñar e implementar una estrategia integral de pruebas para el MVP, balanceando pruebas unitarias, de integración y de aceptación según la Pirámide de Cohn.

Específicos

- Explicar por qué la cúspide de la pirámide (E2E/UI) debe ser la más delgada por su fragilidad y costo.
- Aislar una llamada HTTP externa mediante mocks, sin depender de servicios reales durante las pruebas.
- Configurar una base de datos efímera en memoria para pruebas de integración.
- Explicar el valor de las pruebas de contrato para arquitecturas con componentes desacoplados.
- Registrar la evidencia de la suite de integración en la FPI-23.

Saberes previos

Antes de iniciar, responde (no se califica):

- ¿Recuerdas la diferencia entre Stub y Mock (Guía 14 de Software I)?

Respuesta del estudiante: Stub devuelve datos controlados; mock verifica interacciones. Evito simular la regla interna que pretendo probar.

- ¿Tu API consume alguna API externa (geolocalización, pagos, mensajería)?

Respuesta del estudiante: Existe Mock Bank; no se certifica proveedor bancario externo ni API de pagos. El SaaS no mueve dinero.

- ¿Alguna vez una prueba "pasó" localmente pero falló en otro entorno por depender de datos que ya no existían?

Respuesta del estudiante: Es un riesgo de fixtures persistentes. Uso BD temporal y datos creados por cada test; no invento un fallo histórico específico.

Concepto 1 — La Pirámide de Pruebas de Cohn

Optimiza la inversión en pruebas distribuyéndolas en tres niveles: una base amplia de pruebas unitarias (rápidas y económicas), una capa media de pruebas de integración y servicio (APIs y bases de datos), y una cúspide delgada de pruebas de extremo a extremo (E2E/UI), que deben ser pocas por su fragilidad y alto costo de mantenimiento.

Concepto 2 — Pruebas de integración con aislamiento de infraestructura

Una prueba de integración verifica que varios componentes reales de tu sistema (controlador, lógica de negocio, base de datos) colaboran correctamente. Sin embargo, las dependencias externas al sistema (APIs de terceros, pasarelas de pago) deben seguir mockeándose en la frontera de la infraestructura, para no depender de servicios que pueden estar caídos, ser lentos o tener costo. La base de datos, en cambio, puede usarse real pero efímera (en memoria o en un contenedor limpio) para verificar la integración genuina con la persistencia.

Concepto 3 — Pruebas de Contrato (Consumer-Driven Contracts) con Pact

Cuando dos componentes de software se comunican de forma asíncrona o distribuida, el consumidor de un servicio genera de forma automática un contrato simulado (Pact file). El proveedor del servicio debe validar que su implementación cumple ese contrato antes de desplegar, evitando errores destructivos en producción por desalineación entre ambos lados.

> *Analogía territorial: un taller de calzado de San Gerardo diseña el molde de cartón para la puntera de un zapato escolar y lo envía a un taller costurero en Villa del Rosario. La Prueba de Contrato es que ambos talleres usen una regla física milimétrica estándar para verificar que el acople de la puntera encaje idéntico; si un taller mide en pulgadas y el otro en centímetros, el zapato final se ensambla deforme. El "contrato de medidas" evita fabricar miles de zapatos dañados antes de darse cuenta del error.*

Concepto 4 — Pruebas de Aceptación (UAT)

Verifican sistemáticamente el software frente a las historias de usuario y criterios de aceptación Gherkin (Guía 6 de Software I), demostrando que el sistema entrega el valor de negocio acordado con los stakeholders reales — no solo que "el código funciona" en un sentido técnico aislado.

Regla de oro de esta guía: *"Ninguna prueba de integración escribe datos en la base de datos de desarrollo real; siempre usa una instancia efímera y aislada."*

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, por qué la cúspide de la Pirámide de Cohn (E2E) debe tener menos pruebas que la base.

Respuesta del estudiante: Las pruebas E2E atraviesan navegador y servicios, cuestan más tiempo y tienen más causas de inestabilidad. La base debe comprobar muchas reglas pequeñas; la integración verifica ORM/API y pocos recorridos E2E confirman tareas críticas. La proporción se decide por riesgo, no por una cuota universal.

Ejercicio 2

Para un caso ficticio con un endpoint que consulta una API externa de clima, describe cómo aislarías esa dependencia en una prueba de integración.

Respuesta del estudiante: Inyecto un cliente de clima y simulo timeout, 200 y respuesta malformada en su frontera. Mantengo API, serialización y BD de pruebas reales. Verifico error controlado y tiempo límite; no hago mock del resultado del propio método que quiero comprobar.

Nivel Intermedio (integración real + contrato + auditoría a la IA)

Ejercicio 3

Diseña una prueba de integración real para uno de tus endpoints del contrato OpenAPI (FPI-22), usando una base de datos efímera en memoria.

Respuesta del estudiante: Caso real: ApplicationTests.test_company_isolation, BD efímera creada por Django, usuario miembro de A, solicitud al dashboard de B =>404; listado incluye solo A. Código en backend/tests/test_application.py. La prueba evita datos de desarrollo y usa force_authenticate, por lo que no verifica login ni CSRF.

Ejercicio 4

Si tu MVP consume o expone una API que otro componente consume, describe qué contrato (Pact) definirías entre consumidor y proveedor.

Respuesta del estudiante: Contrato consumidor React/proveedor Django: GET dashboard con horizonte permitido, montos serializados sin pérdida y puntos por fecha; errores 400/403/404/409 acordados. Pact sería complemento, no está instalado ni ejecutado. Validar contrato no garantiza semántica financiera ni aislamiento.

Ejercicio 5

Pide a una IA generativa que genere una suite de pruebas de integración para tu endpoint. Audita: ¿la IA hizo mock de la infraestructura externa correctamente, o mockeó entidades internas del negocio? ¿la prueba escribe accidentalmente en tu base de datos de desarrollo real? Documenta el prompt, la respuesta y tu corrección.

Respuesta del estudiante: Prompt: genera pruebas de dashboard con tenant y horizonte inválido usando TestCase. Respuesta: crear A/B, membresía A, usar APIClient y afirmar 404 B y 400 para 31 días. Auditoría: no mockear Company ni project_obligations en una prueba de integración; BD temporal aislada. En evidencia se registra la ejecución focal de la suite existente, no una suite nueva integrada.

Lista de autoevaluación antes de entregar

- Mi suite de pruebas está balanceada según la Pirámide de Cohn, no es solo E2E frágil.
- Mis pruebas de integración usan una base de datos efímera, no la de desarrollo real.
- Aíslo correctamente las llamadas a APIs externas en la frontera de la infraestructura.
- Audité si la IA mockeó el lugar correcto (infraestructura, no lógica de negocio).
- Diligencié completamente la FPI-23 con evidencia real de pruebas verdes.

Foro de reflexión

Título: *"La falacia del 100% de cobertura de código: ¿es el fanatismo del coverage una métrica de vanidad?"*

Redacte una reflexión de 250–300 palabras respondiendo: ¿garantiza una suite con 100% de cobertura de código unitario la ausencia de fallos críticos operacionales en producción? Analice el ROI de automatizar pruebas E2E lentas frente a pruebas de integración ligeras en pipelines ágiles.

Reflexión del estudiante (294 palabras)
En Horizonte, considero que probar API con ORM real en BD temporal y simular solo proveedores externos o despacho de cola. Las pruebas E2E atraviesan navegador y servicios, cuestan más tiempo y tienen más causas de inestabilidad. La base debe comprobar muchas reglas pequeñas; la integración verifica ORM/API y pocos recorridos E2E confirman tareas críticas. La proporción se decide por riesgo, no por una cuota universal.

El caso concreto que orienta mi análisis es el siguiente: Caso real: ApplicationTests.test_company_isolation, BD efímera creada por Django, usuario miembro de A, solicitud al dashboard de B =>404; listado incluye solo A. Código en backend/tests/test_application.py. La prueba evita datos de desarrollo y usa force_authenticate, por lo que no verifica login ni CSRF.

Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia.

La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados.

Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones. Si falta medición, registro el protocolo y la brecha; no transformo una propuesta en un éxito histórico. Así puedo defender el trabajo y decidir el siguiente incremento con evidencia.

Cuestionario

Seleccione una única respuesta.

1. Según la Pirámide de Cohn, el nivel que debe tener menos pruebas por su fragilidad y costo es:

- A. Pruebas unitarias.
- B. Pruebas de integración y APIs.
- C. Pruebas de extremo a extremo (E2E) y de interfaz de usuario. ✅
- D. Pruebas de compilación en el IDE local.

2. Las pruebas de contrato consumer-driven (Pact) son eficientes porque:

- A. Ejecutan interfaces gráficas reales simulando clics de usuarios.
- B. Permiten validar de forma asíncrona si el proveedor cumple con las firmas de datos pactadas sin desplegar todo el sistema. ✅
- C. Eliminan la necesidad de escribir pruebas unitarias.
- D. Optimizan directamente el consumo de base de datos de producción.

3. En una prueba de integración, la base de datos ideal a utilizar es:

- A. La base de datos de producción real.
- B. Una instancia efímera y aislada (en memoria o contenedor limpio). ✅
- C. Un archivo Excel exportado manualmente.
- D. No se necesita ninguna base de datos en pruebas de integración.

4. Al probar un endpoint que consume una API externa de geolocalización, la práctica correcta es:

- A. Llamar siempre a la API real en cada ejecución de la suite.
- B. Mockear la llamada HTTP externa en la frontera de la infraestructura. ✅
- C. Eliminar del código toda referencia a la API externa.
- D. Ejecutar la prueba solo una vez al mes.

5. Las pruebas de Aceptación (UAT) verifican principalmente:

- A. La velocidad de compilación del proyecto.
- B. Que el software entrega el valor de negocio acordado, frente a las historias de usuario y criterios de aceptación. ✅
- C. Solo la sintaxis del código fuente.
- D. El número de commits realizados por el equipo.

6. Un 100% de cobertura de código unitario, por sí solo:

- A. Garantiza la ausencia total de fallos operacionales en producción.
- B. No garantiza la ausencia de fallos de integración, contrato o de negocio real. ✅
- C. Elimina la necesidad de cualquier otro tipo de prueba.
- D. Es un requisito legal bajo la Ley 1581 de 2012.

7. Según la regla de oro de esta guía, una prueba de integración:

- A. Puede escribir libremente en la base de datos de desarrollo real.
- B. Nunca escribe datos en la base de datos de desarrollo real; usa siempre una instancia efímera. ✅
- C. Debe ejecutarse manualmente antes de cada despliegue.
- D. No requiere ningún tipo de aislamiento de dependencias.

Cuestionario resuelto: 1. C; 2. B; 3. B; 4. B; 5. B; 6. B; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-23

Suite de Pruebas de Integración del Proyecto Integrador

Esta ficha continúa la FPI-22: verifica que la implementación real de tu API cumple el contrato OpenAPI diseñado en esa ficha.

A. Identificación de la ficha

Código de la ficha	FPI-23
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	Pirámide de pruebas: integración, contrato y aceptación

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Recuperar y validar la FPI-22

Endpoints del contrato OpenAPI (recuperados de la FPI-22)

Copia aquí los endpoints que se probarán en esta ficha. Respuesta: GET /api/companies/ y GET /api/companies/{id}/dashboard/?horizon=30.

¿Algo cambió desde la ficha anterior?

Qué cambió	Por qué cambió	Impacto en el proyecto
Estado del producto	Las16 guías anteriores eran un corte antiguo	Se usa matriz vigente y se conserva historial
Tipo de beneficiario	SaaS multiempresa según estudiante	No se simula una empresa aliada única

Si nada cambió, escribe "Sin cambios" en la primera fila y deja la tabla así.

C. Aplicación específica — Pruebas de integración

1. Mapa de la estrategia de pruebas (Pirámide aplicada al MVP)

Describe cuántas pruebas unitarias, de integración y E2E tiene tu proyecto actualmente. Respuesta: Último corte documentado:235 tests PostgreSQL,233 aprobados y2 omitidos. No hay clasificación exacta unitarios/integración/E2E ni cobertura medida; no inventar reparto. QA navegador está documentado por casos, no equivale a suite E2E completa.

2. Evidencia de la suite de integración verde

Ruta del archivo de pruebas y comando de ejecución. Respuesta: backend/tests/test_application.py; .venv/Scripts/python.exe backend/manage.py test tests.test_application.ApplicationTests.test_company_isolation. Resultado focal actual en evidencia_academica.json.

3. Auditoría a la IA en la generación de pruebas

Elemento auditado	Hallazgo y corrección del equipo
¿La IA mockeó la infraestructura externa correctamente, sin mockear el núcleo de negocio?	Prompt: genera pruebas de dashboard con tenant y horizonte inválido usando TestCase. Respuesta: crear A/B, membresía A, usar APIClient y afirmar 404 B y 400 para 31 días. Auditoría: no mockear Company ni project_obligations en una prueba de integración; BD temporal aislada. En evidencia se registra la ejecución focal de la suite existente, no una suite nueva integrada.
¿Alguna prueba escribió accidentalmente en la base de datos de desarrollo real?	Se contrasta contra código y resultados, no contra confianza del LLM. Probar API con ORM real en BD temporal y simular solo proveedores externos o despacho de cola.

Registro de Decisión Arquitectónica — ADR N.° 07

Título	ADR-07 Pruebas de integración
Estado	Aceptado como registro académico; implementación y brechas descritas por separado
Contexto	SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar. Pruebas de integración
Decisión	Probar API con ORM real en BD temporal y simular solo proveedores externos o despacho de cola.
Consecuencias (positivas y negativas)	Menor complejidad y límites explícitos. Requiere medición, aceptación y revisión externa para completar evidencia.
Estrategia de mocking e integración para APIs externas	Probar API con ORM real en BD temporal y simular solo proveedores externos o despacho de cola.

D. Reflexión y trazabilidad

¿Qué aprendimos?

Resume lo aprendido sobre la pirámide de pruebas y el aislamiento de dependencias.

Respuesta: Las pruebas E2E atraviesan navegador y servicios, cuestan más tiempo y tienen más causas de inestabilidad. La base debe comprobar muchas reglas pequeñas; la integración verifica ORM/API y pocos recorridos E2E confirman tareas críticas. La proporción se decide por riesgo, no por una cuota universal.

¿Cómo se aplica al proyecto?

Explica qué tan confiable es ahora la integración real de tu API.

Respuesta: La prueba focal verifica aislamiento en su escenario. La integración con proveedores reales, red y carga no está cubierta por ese caso.

Decisiones tomadas

Decisión	Justificación
Probar API con ORM real en BD temporal y simular solo proveedores externos o despacho de cola.	Alineación con riesgo financiero y capacidad individual
Separar evidencia y propuesta	Evitar certificar despliegues, revisiones o mediciones no realizados

Evidencia generada

Estado documentado: Suite de integración verde en Git. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Evidencia de aislamiento de infraestructura externa. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: ADR N.° 07 registrado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Dificultades o ajustes identificados

¿Qué dependencia externa fue más difícil de aislar?

Respuesta: La frontera de cola/worker: simular despacho no comprueba Redis ni ejecución real.

Próximo paso

¿Qué debe hacer el equipo antes de la siguiente guía?

Respuesta: Añadir pruebas del cálculo puro y plan de carga local.

Conexión con la siguiente guía

Esta suite de integración se ampliará con TDD y pruebas de rendimiento (k6) en la siguiente guía, para verificar no solo que el sistema funciona, sino que resiste carga concurrente real.

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5
Comprensión conceptual (15%)	Confunde integración con unitarias	Comprende conceptos básicos	Comprende lo esencial	Aplica correctamente la pirámide de Cohn	Explica y relaciona con profundidad
Calidad de la suite de integración (30%)	No hay pruebas reales o no ejecutan	Pruebas básicas con fugas de infraestructura	Pruebas aceptables, aisladas en su mayoría	Pruebas robustas y bien aisladas	Suite ejemplar, aislamiento impecable
Análisis de contrato y aceptación (15%)	No analiza contratos ni UAT	Análisis superficial	Análisis aceptable de al menos un caso	Análisis sólido y bien argumentado	Análisis riguroso y ejemplar
Rigor de la auditoría a la IA (15%)	No audita o copia sin procesar	Críticas superficiales	Detecta errores básicos	Auditoría robusta con corrección concreta	Auditoría excepcional y bien argumentada
Participación en el foro (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada
Trazabilidad respecto a la FPI-22 (15%)	No recupera lo construido antes	Copia sin verificar vigencia	Valida lo esencial	Valida y actualiza correctamente	Trazabilidad ejemplar con causa e impacto de cambios

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%).

Recursos académicos y bibliografía

- Cohn, Mike. Succeeding with Agile (Pirámide de Pruebas).
- Pact Foundation. Pact Documentation (Consumer-Driven Contracts).
- ISO/IEC/IEEE 29119. Software Testing.
- IEEE Computer Society. SWEBOK Guide, Version 4.0 (Software Testing).

Comprobaciones de esta entrega
30/10/2026:26 comprobaciones del cálculo contractual aprobaron (18 equivalencias y8 rechazos de horizonte), sin BD. La prueba real tests.test_application.ApplicationTests.test_company_isolation también pasó sobre BD temporal. La variante académica no se integró al backend. Evidencias: _trabajo/evidencia_academica.json y prueba_aislamiento.log.

UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 08 resuelta

Automatización Avanzada: TDD, Pruebas de Regresión y Rendimiento con k6

Asignatura	Ingeniería de Software II
Corte	Segundo Corte (Cierre de Corte)
Duración	Semana 8
Proyecto integrador	Fase 24: Suite de regresión bajo TDD y prueba de rendimiento (k6) del endpoint crítico del MVP
Herramientas	pytest-xdist, k6 de Grafana + IA generativa auditada
Formato de entrega	Documento con taller resuelto y Ficha de Aplicación al Proyecto Integrador (FPI-24) diligenciada
Fecha límite	Domingo de la Semana 8, 11:59 p.m. (ver fecha exacta en el aula virtual)
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar.
Corte documental 30/10/2026. Se contrastaron código, compose.yaml, .github/workflows/ci.yml y docs/IMPLEMENTATION_STATUS.md. El registro del 18/10 informa 235 pruebas PostgreSQL: 233 aprobadas y 2 omitidas; es evidencia histórica, no una nueva ejecución de esta entrega. No se certifica producción actual, ASVS, p95 representativo ni validación con empresarios. Presupuesto inicial 0 USD y respaldo manual por decisión del proyecto.

Fase 24 del Proyecto Integrador — Cierre del Primer Corte de Evolución

A medida que el software evoluciona, existe el riesgo de que los cambios de hoy dañen accidentalmente las características estables construidas ayer. Para blindar el sistema, la ingeniería moderna recurre a las pruebas de regresión automáticas y a TDD como motor de diseño. Pero la estabilidad no es solo correctitud lógica: también implica cumplir con los niveles de servicio del negocio bajo carga real, lo cual solo se descubre sometiendo el sistema a presión.

Esta guía cierra el ciclo de aseguramiento de calidad iniciado en la FPI-23: ampliarás tu suite con disciplina TDD y la someterás, por primera vez, a una prueba de carga real con k6 sobre el endpoint más crítico de tu API. El resultado se registra en la FPI-24.

Resultados de aprendizaje

- Aplicar el micro-ciclo Red-Green-Refactor de Test-Driven Development (TDD).
- Diseñar una suite de pruebas de regresión que blinde funcionalidades ya estables.
- Escribir un script de prueba de rendimiento con k6, incluyendo rampas de carga y umbrales (thresholds).
- Interpretar métricas de percentil (p95) de latencia bajo concurrencia.
- Auditar críticamente un script de carga generado por IA, evitando ataques de denegación de servicio accidentales.

Objetivos

General

Blindar el MVP contra regresiones mediante TDD y validar su comportamiento bajo carga concurrente real con pruebas de rendimiento en k6.

Específicos

- Aplicar el ciclo Red-Green-Refactor a una funcionalidad real del MVP.
- Diseñar un script de k6 con una rampa de subida, sostenimiento y bajada de usuarios virtuales.
- Configurar umbrales (thresholds) de p95 de latencia y tasa de error.
- Interpretar los resultados de un test de carga para identificar cuellos de botella.
- Registrar la evidencia de regresión y rendimiento en la FPI-24.

Saberes previos

Antes de iniciar, responde (no se califica):

- ¿Alguna vez corregiste un bug y, sin darte cuenta, rompiste otra función que antes funcionaba?

Respuesta del estudiante: Las regresiones son un riesgo real de evolución. Pruebas de idempotencia, Decimal y tenant protegen comportamientos antes de cambiar; no relato un incidente personal no registrado.

- ¿Qué endpoint de tu API crees que recibiría más tráfico concurrente en un día de alta demanda de la MiPyme?

Respuesta del estudiante: El dashboard es consulta central, pero sin analítica de uso no puedo afirmar que sea el más solicitado. Lo elijo como candidato para carga.

- ¿Sabes qué significa "percentil 95" en el contexto de tiempos de respuesta?

Respuesta del estudiante: Es un percentil de las duraciones medidas, no promedio ni garantía de que todas las solicitudes cumplen el límite.

Concepto 1 — Pruebas de regresión automáticas

Reejecución sistemática de una suite de pruebas seleccionada para garantizar que los cambios en la base de código no han introducido nuevos fallos. En flujos DevOps, estas suites deben automatizarse en cada commit o Pull Request, no ejecutarse manualmente "cuando alguien se acuerda".

Concepto 2 — Test-Driven Development (TDD)

Disciplina de diseño de Kent Beck basada en un micro-ciclo de tres pasos: Red (escribir una prueba que falle, porque la funcionalidad aún no existe), Green (escribir el código mínimo para que la prueba pase) y Refactor (mejorar la calidad interna sin romper la prueba).

> *Analogía territorial: en las zapaterías artesanales de Cúcuta, un maestro zapatero nunca empieza a cortar el cuero a ciegas sobre una horma. Primero fabrica un molde de papel exacto (la Prueba) basado en la talla del cliente. El primer trozo de cuero tosco puede no encajar (estado Red). El artesano rebaja y pule los bordes (diseño del código) hasta que la pieza se acopla milimétricamente al molde (estado Green). Solo entonces cose el cuero de forma definitiva y da brillo al zapato sin riesgo de arruinar el material (fase Refactor).*

Concepto 3 — Pruebas de rendimiento (Load Testing) con k6

Evalúan el comportamiento de un sistema bajo carga para medir atributos de calidad clave: latencia promedio, tasa de errores, rendimiento (throughput) y consumo de recursos. Un script de k6 típico define etapas (stages): una rampa de subida de usuarios virtuales, un sostenimiento en el pico, y una bajada.

```
export const options = {
  stages: [
    { duration: '10s', target: 50 },  // Rampa de subida
    { duration: '15s', target: 50 },  // Sostenimiento
    { duration: '5s', target: 0 },    // Bajada
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'],  // p95 < 500ms
    http_req_failed: ['rate<0.01'],    // Tasa de error < 1%
  },
};
```

Un p95 de 500ms significa que el 95% de las peticiones respondieron en 500ms o menos; el 5% restante puede revelar cuellos de botella que pasan desapercibidos en pruebas manuales de un solo usuario.

Concepto 4 — Riesgos de las pruebas de carga sin control

Un test de carga apuntado accidentalmente contra producción, o con miles de usuarios virtuales sobre infraestructura modesta, se convierte en un ataque de denegación de servicio autoinfligido. Las pruebas de rendimiento deben ejecutarse siempre en entornos de desarrollo o staging, con un número de usuarios virtuales realista para la escala real del negocio.

Regla de oro de esta guía: *"Ninguna prueba de carga se ejecuta contra un entorno de producción activo, y ningún umbral de rendimiento se define sin relacionarlo con el volumen real de la MiPyme."*

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, el ciclo Red-Green-Refactor de TDD.

Respuesta del estudiante: Red: escribir criterio y observar fallo por comportamiento faltante. Green: implementar lo mínimo para pasar. Refactor: mejorar estructura sin alterar resultados y repetir regresión. Un test verde añadido después no demuestra haber seguido TDD.

Ejercicio 2

Interpreta el siguiente resultado: un test de k6 arroja p95 = 2500ms. Explica qué significa técnicamente y si sería aceptable para un endpoint de consulta de inventario.

Respuesta del estudiante: p95=2500 ms indica que aproximadamente el 95% de las duraciones observadas no supera 2,5 s, según estimación de percentil. Si la meta es menor de 2 s falla; no se decide aceptación solo con promedio. Revisar muestra, errores, arranque frío y concurrencia.

Nivel Intermedio (TDD real + k6 + auditoría a la IA)

Ejercicio 3

Aplica el ciclo TDD completo a una función pequeña y real de tu MVP que aún no tenga prueba (ej. cálculo de un descuento). Documenta las tres fases con capturas o commits.

Respuesta del estudiante: Demostración TDD aislada: prueba contractual de horizonte inválido primero contra stub que lo acepta (Red esperado); implementación que rechaza 45 (Green); variante con guard clauses que conserva resultados (Refactor). evidencia_academica.json guarda resultados. Es ejercicio sobre política real del forecast, sin commits inventados ni cambios en aplicación.

Ejercicio 4

Escribe un script de k6 para el endpoint más crítico de tu contrato OpenAPI (FPI-22), con una rampa de máximo 50 usuarios virtuales y umbrales de p95 y tasa de error.

Respuesta del estudiante: Script k6 completo incluido al final: destino obligatorio localhost, sesión de prueba en variable de entorno, rampa 0→10→50→0, máximo 50 VUs, p95<2000 ms y error<1%. No se ejecuta aquí carga contra producción ni se asignan resultados ficticios.

Ejercicio 5

Pide a una IA generativa que genere un script de k6 para tu endpoint. Audita: ¿propuso un número de usuarios virtuales realista para tu MiPyme, o miles que tumbarían tu servidor? ¿el script apunta accidentalmente a producción? Documenta el prompt, la respuesta y tu corrección.

Respuesta del estudiante: Prompt: genera carga acotada y autenticada del dashboard en localhost. Respuesta: script del taller con host validado, horizonte30 y cookie sessionid. Auditoría: 401/403 serían fallos de ensayo, no latencia útil; no usar miles de VUs ni secretos dentro del script. Arrancar con 1 VU antes de 50; p95 y errores quedan no medidos en esta entrega.

Artefacto académico completo — propuesta o ejemplo según respuesta anterior

```
import http from 'k6/http';
import {check, sleep} from 'k6';
const base = __ENV.BASE_URL || 'http://127.0.0.1:8000';
if (!/^http:\/\/(127\.0\.0\.1|localhost):\d+$/.test(base))
  throw new Error('Solo entorno local');
if (!__ENV.SESSION_ID || !__ENV.COMPANY_ID) throw new Error('Falta sesión sintética');
export const options = {
  stages: [{duration:'20s',target:10},{duration:'40s',target:50},
          {duration:'20s',target:0}],
  thresholds: {http_req_duration:['p(95)<2000'], http_req_failed:['rate<0.01']}
};
export default function () {
  const r = http.get(`${base}/api/companies/${__ENV.COMPANY_ID}/dashboard/?horizon=30`,
    {headers:{Cookie:`sessionid=${__ENV.SESSION_ID}`}});
  check(r, {'dashboard200': r => r.status === 200});
  sleep(1);
```

}

Lista de autoevaluación antes de entregar

- Apliqué el ciclo Red-Green-Refactor a una funcionalidad real, no ficticia.
- Mi script de k6 tiene umbrales (thresholds) explícitos, no solo mide sin criterio de aceptación.
- El número de usuarios virtuales de mi prueba es realista para el volumen de mi MiPyme.
- Verifiqué que la prueba de carga nunca apunte a un entorno de producción real.
- Diligencié completamente la FPI-24 con evidencia de regresión y rendimiento.

Foro de reflexión

Título: *"¿Es TDD un dogma impracticable en la realidad de la consultoría de software local?"*

Redacte una reflexión de 250–300 palabras respondiendo: ¿es TDD una utopía académica que triplica el tiempo de desarrollo, o una disciplina de supervivencia financiera para evitar regresiones costosas en proyectos que deben evolucionar? Justifique su postura con evidencia de su propia experiencia en el taller.

Reflexión del estudiante (284 palabras)
En Horizonte, considero que usar TDD en cambios pequeños y medir carga local autenticada antes de elevar concurrencia. Red: escribir criterio y observar fallo por comportamiento faltante. Green: implementar lo mínimo para pasar. Refactor: mejorar estructura sin alterar resultados y repetir regresión. Un test verde añadido después no demuestra haber seguido TDD.

El caso concreto que orienta mi análisis es el siguiente: Demostración TDD aislada: prueba contractual de horizonte inválido primero contra stub que lo acepta (Red esperado); implementación que rechaza 45 (Green); variante con guard clauses que conserva resultados (Refactor). evidencia_academica.json guarda resultados. Es ejercicio sobre política real del forecast, sin commits inventados ni cambios en aplicación.

Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia.

La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados.

Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones. Si falta medición, registro el protocolo y la brecha; no transformo una propuesta en un éxito histórico. Así puedo defender el trabajo y decidir el siguiente incremento con evidencia.

Cuestionario

Seleccione una única respuesta.

1. En el ciclo TDD, el objetivo técnico fundamental de la fase de Refactorización es:

- A. Compilar todo el código de nuevo sin cambios.

- B. Mejorar la calidad interna del código (eliminar duplicidad, mejorar nombres, desacoplar) garantizando que la prueba permanezca en verde. ✅
- C. Cambiar la lógica de negocio para agregar nuevas características.
- D. Escribir nuevas aserciones no planificadas.

2. Si un test de carga con k6 arroja un p95 de 2500ms, esto se interpreta como:

- A. El 100% de los usuarios experimentaron exactamente 2.5 segundos.
- B. El 95% de las peticiones respondieron en 2500ms o menos, lo cual ayuda a identificar cuellos de botella latentes. ✅
- C. Solo el 5% de los usuarios pudieron acceder al sistema.
- D. La base de datos se cayó y bloqueó todas las conexiones.

3. Las pruebas de regresión deben ejecutarse:

- A. Solo una vez, al final del proyecto.
- B. De forma automatizada en cada commit o Pull Request, dentro del flujo DevOps. ✅
- C. Manualmente, cuando el equipo lo recuerde.
- D. Únicamente si el cliente lo solicita explícitamente.

4. Ejecutar un test de carga de k6 con 10.000 usuarios virtuales contra el servidor de producción real de una MiPyme pequeña es un riesgo porque:

- A. Es la forma correcta y recomendada de validar rendimiento.
- B. Puede convertirse en un ataque de denegación de servicio autoinfligido sobre infraestructura modesta. ✅
- C. k6 no permite configurar más de 100 usuarios virtuales.
- D. No representa ningún riesgo si el pipeline está en verde.

5. Los "thresholds" (umbrales) en un script de k6 sirven para:

- A. Limitar la cantidad de código que puede escribir un desarrollador.
- B. Definir criterios de aceptación cuantitativos (ej. p95 < 500ms) que determinan si la prueba pasa o falla. ✅
- C. Configurar la conexión a la base de datos.
- D. Sustituir la necesidad de pruebas unitarias.

6. La fase "Red" del ciclo TDD corresponde a:

- A. Escribir una prueba unitaria pequeña que falla porque la funcionalidad aún no existe. ✅
- B. Desplegar el sistema en producción.
- C. Refactorizar código ya probado.
- D. Ejecutar un test de carga con k6.

7. Según la regla de oro de esta guía, un umbral de rendimiento debe:

- A. Ser siempre el más estricto posible, sin importar el contexto.
- B. Definirse en relación con el volumen real de tráfico de la MiPyme, no de forma arbitraria. ✅
- C. Ignorarse si el pipeline de CI está en verde.
- D. Aplicarse solo a servicios que consumen bases de datos NoSQL.

Cuestionario resuelto: 1. B; 2. B; 3. B; 4. B; 5. B; 6. A; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-24

Regresión TDD y Rendimiento del Proyecto Integrador

Esta ficha continúa la FPI-23: amplía la suite de integración con disciplina TDD y valida el endpoint más crítico bajo carga real.

A. Identificación de la ficha

Código de la ficha	FPI-24
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	TDD, pruebas de regresión y rendimiento con k6

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Recuperar y validar la FPI-23

Suite de integración (recuperada de la FPI-23)

Resume el estado de la suite de pruebas de integración antes de esta ficha.

Respuesta: Suite histórica PostgreSQL con controles tenant, roles y cálculos. Los casos focales de esta entrega no reemplazan la regresión completa.

¿Algo cambió desde la ficha anterior?

Qué cambió	Por qué cambió	Impacto en el proyecto
Estado del producto	Las16 guías anteriores eran un corte antiguo	Se usa matriz vigente y se conserva historial
Tipo de beneficiario	SaaS multiempresa según estudiante	No se simula una empresa aliada única

Si nada cambió, escribe "Sin cambios" en la primera fila y deja la tabla así.

C. Aplicación específica — TDD y rendimiento

1. Evidencia del ciclo TDD aplicado

Enlace al commit o código que demuestra Red-Green-Refactor sobre una función real.

Respuesta: Demostración TDD aislada: prueba contractual de horizonte inválido primero contra stub que lo acepta (Red esperado); implementación que rechaza 45 (Green); variante con guard clauses que conserva

resultados (Refactor). evidencia_academica.json guarda resultados. Es ejercicio sobre política real del forecast, sin commits inventados ni cambios en aplicación.

2. Resultado de la prueba de rendimiento (k6)

Endpoint probado	Configuración (VUs, duración)	Resultado (p95, tasa de error)
GET dashboard local	0→10→50→0 en80s, sesión sintética	No ejecutado k6; p95/error N/D

Registro de Decisión Arquitectónica — ADR N.° 08

Título	ADR-08 TDD y rendimiento
Estado	Aceptado como registro académico; implementación y brechas descritas por separado
Contexto	SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar. TDD y rendimiento
Decisión	Usar TDD en cambios pequeños y medir carga local autenticada antes de elevar concurrencia.
Consecuencias (positivas y negativas)	Menor complejidad y límites explícitos. Requiere medición, aceptación y revisión externa para completar evidencia.
Impacto técnico (ej. necesidad de caché o índices)	Usar TDD en cambios pequeños y medir carga local autenticada antes de elevar concurrencia.

D. Reflexión y trazabilidad

¿Qué aprendimos?

Resume lo aprendido sobre TDD, regresión y pruebas de rendimiento.

Respuesta: Red: escribir criterio y observar fallo por comportamiento faltante. Green: implementar lo mínimo para pasar. Refactor: mejorar estructura sin alterar resultados y repetir regresión. Un test verde añadido después no demuestra haber seguido TDD.

¿Cómo se aplica al proyecto?

Explica qué tan resiliente es ahora tu MVP ante carga concurrente.

Respuesta: Resiliencia bajo50VUs aún no medida. No inferirla por tests funcionales ni por el script k6.

Decisiones tomadas

Decisión	Justificación
Usar TDD en cambios pequeños y medir carga local autenticada antes de elevar concurrencia.	Alineación con riesgo financiero y capacidad individual
Separar evidencia y propuesta	Evitar certificar despliegues, revisiones o mediciones no realizados

Evidencia generada

Estado documentado: Evidencia del ciclo TDD. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Script de k6 y resultado de la prueba de carga. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: ADR N.° 08 registrado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Dificultades o ajustes identificados

¿Qué umbral de rendimiento fue más difícil de cumplir?

Respuesta: p95<2s sigue sin ensayo representativo, por lo que no se dice que se cumplió o incumplió.

Próximo paso

¿Qué debe hacer el equipo antes de la siguiente guía?

Respuesta: Integrar gate de regresión y acordar ensayo local reproducible.

Conexión con la siguiente guía

Con la calidad y el rendimiento verificados, el Segundo Corte se dedicará a la industrialización del pipeline: en la siguiente guía automatizarás toda esta suite dentro de un pipeline de Integración Continua profesional con Quality Gates.

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5
Comprensión conceptual (15%)	Confunde TDD con pruebas tradicionales	Comprende conceptos básicos	Comprende lo esencial	Aplica correctamente el ciclo TDD	Explica y relaciona con profundidad

Evidencia real de TDD (20%)	No hay evidencia de TDD	Evidencia parcial o simulada	Evidencia aceptable de al menos un ciclo	Evidencia clara y bien documentada	Evidencia ejemplar y sistemática
Calidad del script y resultado de k6 (25%)	No hay script o no ejecuta	Script básico sin thresholds	Script funcional con thresholds básicos	Script robusto con umbrales realistas	Script y análisis de nivel profesional
Rigor de la auditoría a la IA (15%)	No audita o copia sin procesar	Críticas superficiales	Detecta riesgos básicos	Auditoría robusta con corrección concreta	Auditoría excepcional y bien argumentada
Participación en el foro (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada
Trazabilidad respecto a la FPI-23 (15%)	No recupera lo construido antes	Copia sin verificar vigencia	Valida lo esencial	Valida y actualiza correctamente	Trazabilidad ejemplar con causa e impacto de cambios

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%).

Recursos académicos y bibliografía

- Beck, Kent. Test-Driven Development: By Example.
- Grafana Labs. k6 Documentation.
- IEEE Computer Society. SWEBOK Guide, Version 4.0 (Software Testing).

Comprobaciones de esta entrega
30/10/2026:26 comprobaciones del cálculo contractual aprobaron (18 equivalencias y8 rechazos de horizonte), sin BD. La prueba real tests.test_application.ApplicationTests.test_company_isolation también pasó sobre BD temporal. La variante académica no se integró al backend. Evidencias: _trabajo/evidencia_academica.json y prueba_aislamiento.log.

UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 09 resuelta

DevOps y CI/CD: Integración Continua Profesional, Quality Gates y Gestión de Artefactos

Asignatura	Ingeniería de Software II
Corte	Segundo Corte (Inicio)
Duración	Semana 9
Proyecto integrador	Fase 25: Pipeline de Integración Continua profesional con Quality Gate del MVP
Herramientas	GitHub Actions, SonarCloud/SonarQube + IA generativa auditada
Formato de entrega	Documento con taller resuelto y Ficha de Aplicación al Proyecto Integrador (FPI-25) diligenciada
Fecha límite	Domingo de la Semana 9, 11:59 p.m. (ver fecha exacta en el aula virtual)
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar.
Corte documental 30/10/2026. Se contrastaron código, compose.yaml, .github/workflows/ci.yml y docs/IMPLEMENTATION_STATUS.md. El registro del 18/10 informa 235 pruebas PostgreSQL: 233 aprobadas y 2 omitidas; es evidencia histórica, no una nueva ejecución de esta entrega. No se certifica producción actual, ASVS, p95 representativo ni validación con empresarios. Presupuesto inicial 0 USD y respaldo manual por decisión del proyecto.

Fase 25 del Proyecto Integrador — Evolución del MVP

En Software I automatizaste un pipeline básico (Guía 15). Ahora lo llevas a nivel profesional: no basta con que las pruebas corran automáticamente, el pipeline debe bloquear activamente la integración de código que no cumpla estándares mínimos de calidad, medidos con análisis estático de código, no solo con pruebas verdes.

Esta guía retoma toda la suite construida en la FPI-23 y FPI-24: la integrarás en un pipeline con tres etapas (linter, test, analysis) que actúa como una Puerta de Calidad real, no decorativa. El resultado se registra en la FPI-25.

Resultados de aprendizaje

- Diseñar un pipeline de CI con múltiples jobs (linter, test, analysis) como código.

- Configurar Quality Gates que bloqueen la fusión de Pull Requests que incumplan estándares mínimos.
- Gestionar de forma inmutable los artefactos construidos mediante versionamiento.
- Configurar el uso seguro de secrets dinámicos del repositorio.
- Auditar críticamente un archivo YAML de pipeline generado por IA, detectando alucinaciones de sintaxis y riesgos de secretos.

Objetivos

General

Automatizar un pipeline de Integración Continua profesional que bloquee activamente la integración de código que incumpla los Quality Gates definidos.

Específicos

- Explicar por qué la CI exige integrar frecuentemente y verificar automáticamente cada integración.
- Configurar al menos tres jobs lógicos en un workflow de GitHub Actions.
- Definir un Quality Gate (ej. cobertura mínima, cero vulnerabilidades críticas) que bloquee el build si no se cumple.
- Usar variables secretas del repositorio en vez de credenciales en texto plano.
- Registrar la evidencia del pipeline funcionando en la FPI-25.

Saberes previos

Antes de iniciar, responde (no se califica):

- ¿Recuerdas cómo configurar un pipeline básico de CI en GitHub Actions (Guía 15 de Software I)?

Respuesta del estudiante: Sí: .github/workflows/ci.yml contiene jobs backend, frontend, PostgreSQL y desktop; contenido local verificado.

- ¿Qué significa que un pipeline "se ponga en rojo"?

Respuesta del estudiante: Rojo indica fallo de un paso o condición; debe impedir promoción y merge si el check es requerido.

- ¿Tu equipo tiene algún secreto (token, contraseña) escrito en texto plano en algún archivo del repositorio?

Respuesta del estudiante: Compose y CI contienen claves sintéticas explícitas de desarrollo. No son secreto bancario productivo; no deben reutilizarse fuera del ensayo. No se imprimen archivos privados.

Concepto 1 — Integración Continua profesional

La CI obliga al equipo a integrar su código en la rama principal frecuentemente. Cada integración se verifica automáticamente mediante construcción del build, análisis estático (linters/seguridad) y ejecución de tests de regresión. Lanzar builds manuales desde la máquina de un desarrollador introduce inconsistencias graves ("en mi máquina sí funciona").

Concepto 2 — Quality Gates (Puertas de Calidad)

Criterios automáticos y mandatorios definidos en el pipeline (ej. cobertura de pruebas > 80%, cero code smells críticos, cero vulnerabilidades conocidas) que bloquean la integración de un Pull Request si el código no cumple el estándar mínimo. Un Quality Gate meramente informativo (que no bloquea el build) no es una puerta de calidad: es una sugerencia ignorable.

> *Analogía territorial: en el puesto de control aduanero del Puente Internacional Simón Bolívar, los camiones no cruzan de forma descontrolada. Existe un Quality Gate aduanero: báscula 1 (revisión de documentos, como el linter), báscula 2 (pesaje del camión, como los tests unitarios) y báscula 3 (escáner de rayos X, como el análisis de seguridad). Si el camión falla una sola báscula, la barrera se bloquea automáticamente y el camión regresa a la zona de carga para corregir el defecto: el producto dañado nunca cruza la frontera.*

Concepto 3 — Gestión inmutable de artefactos

Los paquetes funcionales resultantes del pipeline (compilados, imágenes de contenedores) deben versionarse de forma única e inmutable, con firmas de commit o versionamiento semántico, para garantizar que lo que se prueba en staging sea exactamente lo que se despliega en producción — sin recompilaciones intermedias que puedan introducir diferencias.

Concepto 4 — Gestión segura de secretos en el pipeline

Ningún token, contraseña o clave de API debe escribirse en texto plano dentro del archivo YAML del pipeline. GitHub Actions provee variables secretas cifradas del repositorio, accesibles con la sintaxis `${{ secrets.NOMBRE }}`, que nunca se imprimen en los logs de ejecución.

Regla de oro de esta guía: *"Si el análisis estático detecta un fallo de calidad crítico, el build debe romperse de inmediato en rojo; jamás se debe deshabilitar el Quality Gate para 'avanzar más rápido'."*

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, la diferencia entre un pipeline de CI básico (solo ejecuta pruebas) y uno con Quality Gates reales.

Respuesta del estudiante: CI ejecuta verificaciones; un Quality Gate agrega criterios que hacen terminar con error e impiden promover artefactos si fallan. Una cobertura publicada pero ignorada no bloquea nada. Además, la protección de rama debe requerir checks para impedir merges.

Ejercicio 2

Diseña, en pseudocódigo YAML, los tres jobs lógicos (linter, test, analysis) que tendría el pipeline de un backend en Python.

Respuesta del estudiante: YAML conceptual: jobs: lint ejecuta Ruff; test depende de lint y ejecuta Django sobre BD temporal; analysis depende de test y ejecuta coverage report --fail-under=80. Todo comando que falle propaga exit code. El umbral 80% es propuesta para código nuevo, no cobertura observada.

Nivel Intermedio (pipeline real + auditoría a la IA)

Ejercicio 3

Configura tu pipeline real en GitHub Actions con al menos tres jobs, integrando la suite de pruebas de las FPI-23 y FPI-24.

Respuesta del estudiante: Pipeline real: .github/workflows/ci.yml tiene backend (Ruff, formato, migraciones y tests), frontend (npm ci/test/build), backend-postgres con servicio16 y desktop con Node tests. Se verificó su contenido; no se modifica ni se dispara una publicación al resolver documentos.

Ejercicio 4

Configura un Quality Gate real (ej. con SonarCloud) que bloquee el build si la cobertura cae por debajo de un umbral definido por tu equipo.

Respuesta del estudiante: Propuesta concreta: coverage run --source=backend/apps backend/manage.py test backend/tests y coverage report --fail-under=80. Falta instalar/bloquear coverage y configurar required check antes de afirmar gate operativo. No se encontró SonarCloud ni evidencia de porcentaje actual.

Ejercicio 5

Pide a una IA generativa que genere el YAML completo de tu pipeline. Audita: ¿usó versiones actualizadas de las GitHub Actions (ej. @v4, no @v1)? ¿quemó algún secreto en texto plano? ¿el Quality Gate realmente bloquea el build o es solo informativo? Documenta el prompt, la respuesta y tu corrección.

Respuesta del estudiante: Prompt: audita CI sin revelar secretos. Respuesta: hay checkout@v4, setup-python@v5 y setup-node@v4 en archivo leído; usar referencias soportadas y fijadas al integrar. Auditoría: no se declaran últimas versiones sin verificar proveedor; la clave PostgreSQL es sintética de CI, no secreto productivo. Tests sí fallan el job; cobertura y protección de rama no están demostradas.

Artefacto académico completo — propuesta o ejemplo según respuesta anterior

```
# Propuesta académica adicional al CI real; no workflow integrado
name: coverage-gate
on: [push, pull_request]
jobs:
  gate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.13'}
      - run: pip install -r backend/requirements.txt coverage
      - run: coverage run --source=backend/apps backend/manage.py test backend/tests
      - run: coverage report --fail-under=80
# Fijar coverage en dependencias y requerir este check antes de integrar.
```

Lista de autoevaluación antes de entregar

- Mi pipeline tiene al menos tres jobs lógicos automatizados.
- Configuré un Quality Gate que realmente bloquea la fusión si no se cumple.
- No hay ningún secreto en texto plano en mi archivo YAML.
- Audité si la IA usó acciones desactualizadas o dejó el Quality Gate como meramente informativo.
- Diligencié completamente la FPI-25 con evidencia de mi pipeline en verde.

Foro de reflexión

Título: *"La tiranía del pipeline verde: ¿genera la automatización extrema una falsa sensación de seguridad técnica?"*

Redacte una reflexión de 250–300 palabras respondiendo: si un equipo tiene un pipeline impecable con 100% de tests automáticos aprobados y cero code smells, ¿puede garantizar que el cliente final no experimentará fallos de lógica de negocio o caídas operacionales debidas a problemas ambientales de la nube? Analice críticamente los límites de la verificación frente a la validación real.

Reflexión del estudiante (277 palabras)
En Horizonte, considero que conservar CI existente y proponer gate bloqueante de cobertura sin fingir aprobación remota. CI ejecuta verificaciones; un Quality Gate agrega criterios que hacen terminar con error e impiden promover artefactos si fallan. Una cobertura publicada pero ignorada no bloquea nada. Además, la protección de rama debe requerir checks para impedir merges.

El caso concreto que orienta mi análisis es el siguiente: Pipeline real: .github/workflows/ci.yml tiene backend (Ruff, formato, migraciones y tests), frontend (npm ci/test/build), backend-postgres con servicio16 y desktop con Node tests. Se verificó su contenido; no se modifica ni se dispara una publicación al resolver documentos.

Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia.

La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados.

Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones. Si falta medición, registro el protocolo y la brecha; no transformo una propuesta en un éxito histórico. Así puedo defender el trabajo y decidir el siguiente incremento con evidencia.

Cuestionario

Seleccione una única respuesta.

1. Un archivo de definición de Pipeline como Código (YAML) tiene como función fundamental:

- A. Guardar datos confidenciales de usuarios finales en la nube.
- B. Describir de forma declarativa, versionable y reproducible los pasos para construir, verificar y empaquetar el sistema automáticamente. ✅
- C. Acelerar el compilador local aumentando procesadores virtuales.
- D. Eliminar bugs lógicos de negocio de forma autónoma.

2. Si en el pipeline se detecta un fallo de aserción en una prueba de regresión, la consecuencia innegociable es:

- A. Saltar el test fallido e integrar el código de todas formas.
- B. El build debe romperse de inmediato en rojo, bloqueando la fusión y notificando al equipo. ✅

- C. Desplegar en producción y corregir con un hotfix manual en caliente.
- D. Borrar el caso de prueba que falló para que el pipeline vuelva a verde.

3. Un Quality Gate meramente informativo (que no bloquea el build) se considera:

- A. Una puerta de calidad completa y efectiva.
- B. Una sugerencia ignorable que debilita el propósito real de un Quality Gate. ✅
- C. El estándar recomendado por ISO/IEC/IEEE 12207.
- D. Suficiente para garantizar la calidad del sistema.

4. La gestión inmutable de artefactos busca garantizar que:

- A. Cada entorno recompile el código de forma distinta.
- B. Lo que se prueba en staging sea exactamente lo que se despliega en producción. ✅
- C. Los artefactos nunca se versionen.
- D. Solo el ambiente de producción tenga artefactos válidos.

5. La sintaxis `${{ secrets.SONAR_TOKEN }}` en GitHub Actions sirve para:

- A. Imprimir el token en los logs para depuración.
- B. Acceder a una variable secreta cifrada del repositorio sin exponerla en texto plano. ✅
- C. Generar automáticamente un nuevo token cada ejecución.
- D. Deshabilitar el análisis de seguridad del pipeline.

6. Usar una acción de GitHub desactualizada como `actions/checkout@v1` en un pipeline nuevo es un ejemplo de:

- A. Una práctica recomendada de estabilidad.
- B. Una posible alucinación de sintaxis de la IA que debe corregirse a una versión actual y soportada. ✅
- C. Un requisito obligatorio de SonarCloud.
- D. La única forma de configurar un Quality Gate.

7. Según la regla de oro de esta guía, ante un fallo crítico de calidad detectado por el análisis estático:

- A. Se debe deshabilitar el Quality Gate para no atrasar la entrega.
- B. El build debe romperse de inmediato en rojo; nunca se debe deshabilitar el Quality Gate para avanzar más rápido. ✅
- C. Se ignora si el cliente tiene prisa.
- D. Se corrige solo si el fallo es de severidad baja.

Cuestionario resuelto: 1. B; 2. B; 3. B; 4. B; 5. B; 6. B; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-25

Pipeline de CI Profesional con Quality Gate del Proyecto Integrador

Esta ficha continúa la FPI-24: automatiza toda la suite de pruebas construida hasta ahora dentro de un pipeline con Quality Gate real.

A. Identificación de la ficha

Código de la ficha	FPI-25
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	CI/CD profesional: pipelines, Quality Gates y artefactos

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Recuperar y validar la FPI-24

Suite de pruebas (recuperada de la FPI-23/24)

Resume qué pruebas se integrarán en el pipeline de esta ficha.

Respuesta: Django temporal y PostgreSQL, Ruff, migraciones, npm test/build y Node desktop; pruebas focales aisladas son apoyo documental.

¿Algo cambió desde la ficha anterior?

Qué cambió	Por qué cambió	Impacto en el proyecto
Estado del producto	Las16 guías anteriores eran un corte antiguo	Se usa matriz vigente y se conserva historial
Tipo de beneficiario	SaaS multiempresa según estudiante	No se simula una empresa aliada única

Si nada cambió, escribe "Sin cambios" en la primera fila y deja la tabla así.

C. Aplicación específica — Pipeline profesional

1. Enlace al workflow YAML

Ruta del archivo .github/workflows/ci.yml

Respuesta: .github/workflows/ci.yml, inspeccionado en esta entrega.

2. Configuración del Quality Gate

Criterio del Quality Gate	Umbral definido por el equipo
Tests/regresión	Sin fallos; job no ignora exit code
Lint/formato/migraciones	Ruff y makemigrations --check
Cobertura propuesta	80% código nuevo, baseline por medir; no operativo aún

3. Evidencia de ejecución exitosa (los tres jobs en verde)

Captura o enlace al historial de GitHub Actions.

Respuesta: Historial remoto no inspeccionado; archivo local no certifica ejecución GitHub Actions ni protección de rama.

Registro de Decisión Arquitectónica — ADR N.° 09

Título	ADR-09 CI y gates
Estado	Aceptado como registro académico; implementación y brechas descritas por separado
Contexto	SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar. CI y gates
Decisión	Conservar CI existente y proponer gate bloqueante de cobertura sin fingir aprobación remota.
Consecuencias (positivas y negativas)	Menor complejidad y límites explícitos. Requiere medición, aceptación y revisión externa para completar evidencia.

D. Reflexión y trazabilidad

¿Qué aprendimos?

Resume lo aprendido sobre CI profesional y Quality Gates.

Respuesta: CI ejecuta verificaciones; un Quality Gate agrega criterios que hacen terminar con error e impiden promover artefactos si fallan. Una cobertura publicada pero ignorada no bloquea nada. Además, la protección de rama debe requerir checks para impedir merges.

¿Cómo se aplica al proyecto?

Explica cómo el pipeline protege ahora la calidad del MVP.

Respuesta: Conservar CI existente y proponer gate bloqueante de cobertura sin fingir aprobación remota.

Decisiones tomadas

Decisión	Justificación
Conservar CI existente y proponer gate bloqueante de cobertura sin fingir aprobación remota.	Alineación con riesgo financiero y capacidad individual
Separar evidencia y propuesta	Evitar certificar despliegues, revisiones o mediciones no realizados

Evidencia generada

Estado documentado: Pipeline con tres jobs en verde. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Quality Gate configurado y bloqueando builds fallidos. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: ADR N.° 09 registrado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Dificultades o ajustes identificados

¿Qué job fue más difícil de configurar correctamente?

Respuesta: Preparar PostgreSQL de CI y distinguir clave sintética de secreto de producción.

Próximo paso

¿Qué debe hacer el equipo antes de la siguiente guía?

Respuesta: Preparar imagen reproducible y validar usuario runtime.

Conexión con la siguiente guía

Con el pipeline de CI consolidado, la siguiente guía empaquetará el sistema en contenedores Docker optimizados, que serán el artefacto inmutable que este mismo pipeline construirá y verificará en cada integración.

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5
Comprensión conceptual (15%)	Confunde CI con ejecución manual	Comprende conceptos básicos	Comprende lo esencial	Aplica correctamente CI y Quality Gates	Explica y relaciona con profundidad

Configuración del pipeline (25%)	No hay pipeline o no ejecuta	Pipeline básico con un solo job	Pipeline con tres jobs funcionales	Pipeline robusto y bien estructurado	Pipeline de nivel profesional
Rigor del Quality Gate (25%)	No bloquea builds fallidos	Quality Gate meramente informativo	Quality Gate bloquea en casos básicos	Quality Gate bien calibrado y efectivo	Quality Gate ejemplar, alineado a estándares reales
Rigor de la auditoría a la IA (15%)	No audita o copia sin procesar	Críticas superficiales	Detecta errores básicos	Auditoría robusta con corrección concreta	Auditoría excepcional y bien argumentada
Participación en el foro (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada
Trazabilidad respecto a la FPI-24 (10%)	No recupera lo construido antes	Copia sin verificar vigencia	Valida lo esencial	Valida y actualiza correctamente	Trazabilidad ejemplar con causa e impacto de cambios

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%).

Recursos académicos y bibliografía

- Humble, Jez; Farley, David. Continuous Delivery.
- GitHub. GitHub Actions Documentation.
- SonarSource. SonarQube/SonarCloud Documentation.
- IEEE Computer Society. SWEBOK Guide, Version 4.0 (Software Engineering Operations).

UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 10 resuelta

Contenerización Avanzada con Docker: Multi-Stage Builds, Persistencia y Docker Compose

Asignatura	Ingeniería de Software II
Corte	Segundo Corte
Duración	Semana 10
Proyecto integrador	Fase 26: Contenerización optimizada y orquestación local del MVP con Docker Compose
Herramientas	Docker Desktop, Docker CLI, Docker Compose + IA generativa auditada
Formato de entrega	Documento con taller resuelto y Ficha de Aplicación al Proyecto Integrador (FPI-26) diligenciada
Fecha límite	Domingo de la Semana 10, 11:59 p.m. (ver fecha exacta en el aula virtual)
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar.
Corte documental 30/10/2026. Se contrastaron código, compose.yaml, .github/workflows/ci.yml y docs/IMPLEMENTATION_STATUS.md. El registro del 18/10 informa 235 pruebas PostgreSQL: 233 aprobadas y 2 omitidas; es evidencia histórica, no una nueva ejecución de esta entrega. No se certifica producción actual, ASVS, p95 representativo ni validación con empresarios. Presupuesto inicial 0 USD y respaldo manual por decisión del proyecto.

Fase 26 del Proyecto Integrador — Evolución del MVP

Uno de los problemas más históricos del desarrollo de software es la discrepancia de comportamiento entre entornos: "en mi máquina sí funcionaba". El pipeline de CI de la FPI-25 verifica tu código, pero necesita un artefacto estandarizado y portable para poder desplegarlo con confianza en cualquier entorno. Esta guía te enseña a empaquetar tu MVP en contenedores Docker optimizados y a orquestar localmente todos sus servicios de forma coordinada.

Esta guía retoma el pipeline de la FPI-25: el Dockerfile que construyas aquí será, a partir de ahora, el artefacto inmutable que ese pipeline construye y verifica en cada integración. El resultado se registra en la FPI-26.

Resultados de aprendizaje

- Explicar la diferencia entre contenedor y máquina virtual.
- Diseñar un Dockerfile optimizado con construcción multietapa (Multi-Stage Builds).
- Configurar volúmenes persistentes para no perder datos al destruir un contenedor.
- Orquestar múltiples servicios coordinados con Docker Compose.
- Auditar críticamente un Dockerfile generado por IA, detectando riesgos de privilegios y secretos expuestos.

Objetivos

General

Empaquetar el MVP en una imagen Docker optimizada y segura, orquestando sus servicios dependientes mediante Docker Compose con persistencia de datos.

Específicos

- Explicar por qué la construcción multietapa reduce el peso de una imagen de producción.
- Configurar un usuario no root en el contenedor final por razones de seguridad.
- Configurar un volumen persistente para la base de datos del sistema.
- Definir una red virtual aislada en Docker Compose.
- Registrar la evidencia de contenerización y orquestación en la FPI-26.

Saberes previos

Antes de iniciar, responde (no se califica):

- ¿Has instalado Docker Desktop en tu máquina? ¿Ejecutaste alguna vez un contenedor?

Respuesta del estudiante: La documentación registra Docker Desktop y Compose completos operativos localmente. No afirmo repetir ese arranque hoy.

- ¿Sabes qué pasa con los datos de una base de datos si el contenedor que la aloja se destruye sin un volumen configurado?

Respuesta del estudiante: La capa escribible desaparece con el contenedor; datos persistentes requieren volumen y respaldo probado.

- ¿Tu MVP tiene contraseñas de base de datos escritas directamente en el código o en algún archivo versionado en Git?

Respuesta del estudiante: Compose tiene una contraseña explícita local y CI otra sintética. Son contexto de ensayo y deben reemplazarse por gestión privada al publicar.

Concepto 1 — Contenedor vs. máquina virtual

La virtualización de hardware simula servidores completos, cada uno con su propio sistema operativo huésped completo y redundante, de alto consumo. La contenerización comparte de forma aislada el kernel del sistema operativo host, aislando únicamente los procesos lógicos del software en espacio de usuario, lo cual garantiza eficiencia milimétrica.

> *Analogía territorial: el calzado fabricado en San Gerardo o los ladrillos de Villa del Rosario se enviaban tradicionalmente en bultos amarrados de formas distintas, causando retrasos y caídas de mercancía al cambiar de camión en la frontera de La Parada. La adopción de contenedores estandarizados de carga internacional resolvió esto de raíz: sin importar qué contengan, el camión, la grúa y el barco usan los mismos enganches. Docker hace exactamente lo mismo con tu código: lo empaqueta con todas sus dependencias en una "caja estandarizada" que corre idéntico en tu laptop, el pipeline o la nube.*

Concepto 2 — Construcción multietapa (Multi-Stage Builds)

Define múltiples bloques FROM en un único Dockerfile: se usa una imagen pesada con herramientas de desarrollo (compiladores, SDKs) en la primera fase, y se copian únicamente los binarios limpios de producción hacia una imagen final minimalista (ej. Alpine Linux), reduciendo el peso de gigabytes a escasos megabytes.

```
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
USER node
CMD ["node", "dist/index.js"]
```

Concepto 3 — Persistencia con volúmenes y orquestación con Docker Compose

El sistema de archivos interno de un contenedor es efímero: si se destruye sin un volumen configurado, los datos se pierden de forma irreversible. Docker Compose define y ejecuta aplicaciones de múltiples contenedores de forma declarativa mediante un archivo YAML, definiendo redes virtuales lógicas para aislar la comunicación e intercomunicar los contenedores de forma coordinada.

Concepto 4 — Seguridad del contenedor: usuario no root

Un contenedor que corre por defecto con privilegios de superusuario (root) expone el sistema host si el backend tiene un exploit. La práctica correcta es crear y activar un usuario no root con privilegios limitados dentro del Dockerfile.

Regla de oro de esta guía: *"Ninguna contraseña o credencial se queda quemada en duro dentro del Dockerfile o docker-compose.yml; siempre se externaliza mediante variables de entorno ignoradas en el .gitignore."*

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, qué hace la instrucción COPY --from=builder en un Dockerfile multietapa.

Respuesta del estudiante: COPY --from=builder copia artefactos de una etapa previa a la imagen final. Permite dejar compiladores y cachés fuera, pero no garantiza que librerías nativas de ejecución estén presentes. Deben verificarse compatibilidad y permisos.

Ejercicio 2

Explica qué le sucede a los datos de una base de datos SQLite si su contenedor se destruye sin volumen persistente configurado.

Respuesta del estudiante: Si SQLite está en la capa escribible del contenedor, eliminarlo destruye ese archivo. Un volumen sobre la ruta de la BD conserva datos entre recreaciones; también requiere respaldo y prueba de restauración. Un volumen no protege de borrado lógico ni de docker compose down -v.

Nivel Intermedio (Dockerfile real + auditoría a la IA)

Ejercicio 3

Escribe el Dockerfile multietapa real de tu MVP, verificando que la imagen final use un usuario no root.

Respuesta del estudiante: El Dockerfile real backend/Dockerfile usa python:3.13-slim, instalación previa de requisitos y runserver; no es multietapa y no declara USER. Se incluye propuesta multietapa no root en el taller; no la describo como desplegada. Debe validar wheels nativos, permisos y servidor de producción antes de integrar.

Ejercicio 4

Escribe el docker-compose.yml que orqueste tu backend junto con su base de datos, con volumen persistente y red aislada.

Respuesta del estudiante: Compose real compose.yaml: PostgreSQL16 con postgres_data, Redis7, migración, backend, Celery y frontend; puertos locales ligados a127.0.0.1 y dependencias con healthchecks. La contraseña literal es local de desarrollo, no trasladable a producción. La red por defecto de Compose no equivale a red interna sin salida.

Ejercicio 5

Pide a una IA generativa que genere tu Dockerfile y docker-compose.yml. Audita: ¿usó una imagen base actualizada (LTS)? ¿el contenedor corre como root por defecto? ¿reordenó las capas de forma que la caché de compilación se aproveche bien? ¿alguna contraseña quedó en texto plano? Documenta el prompt, la respuesta y tu corrección.

Respuesta del estudiante: Prompt: endurece el contenedor sin ocultar diferencias de entorno. Respuesta: builder genera wheels, runtime instala y usa appuser; secretos por entorno/gestor. Auditoría: propuesta no verificada como build de esta entrega; tamaño y reducción son N/D. Compose existente se usa como referencia local, no se inventa captura de contenedores actuales.

Artefacto académico completo — propuesta o ejemplo según respuesta anterior

```
# Propuesta académica; validar build y dependencias nativas antes de integrar
FROM python:3.13-slim AS builder
WORKDIR /build
COPY backend/requirements.txt .
RUN pip wheel --wheel-dir /wheels -r requirements.txt
FROM python:3.13-slim AS runtime
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
```

```
RUN useradd --create-home appuser
COPY --from=builder /wheels /wheels
RUN pip install --no-cache-dir /wheels/* && rm -rf /wheels
WORKDIR /app/backend
COPY --chown=appuser:appuser backend/ ./
USER appuser
EXPOSE 8000
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000", "--noreload"]
# runserver solo ensayo local; elegir servidor WSGI/ASGI para producción.
```

Lista de autoevaluación antes de entregar

- Mi Dockerfile usa construcción multietapa y una imagen base actualizada.
- Mi contenedor corre con un usuario no root.
- Configuré un volumen persistente para mi base de datos.
- No hay ninguna contraseña en texto plano en mi Dockerfile ni en mi docker-compose.yml.
- Diligencié completamente la FPI-26 con evidencia de mi contenerización funcionando.

Foro de reflexión

Título: *"¿Es Docker Alpine siempre la mejor opción o introduce costos ocultos de compatibilidad?"*

Redacte una reflexión de 250–300 palabras respondiendo: usar imágenes Alpine reduce el tamaño a escasos megabytes, pero puede causar fallos de compilación crípticos con dependencias nativas. ¿Justifica el ahorro de espacio el esfuerzo técnico extra? Argumente con base en su propia experiencia del taller.

Reflexión del estudiante (279 palabras)
En Horizonte, considero que distinguir Compose local operativo de una imagen productiva endurecida, aún mejorable. COPY --from=builder copia artefactos de una etapa previa a la imagen final. Permite dejar compiladores y cachés fuera, pero no garantiza que librerías nativas de ejecución estén presentes. Deben verificarse compatibilidad y permisos.

El caso concreto que orienta mi análisis es el siguiente: El Dockerfile real backend/Dockerfile usa python:3.13-slim, instalación previa de requisitos y runserver; no es multietapa y no declara USER. Se incluye propuesta multietapa no root en el taller; no la describo como desplegada. Debe validar wheels nativos, permisos y servidor de producción antes de integrar.

Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia.

La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados.

Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones. Si falta medición, registro el protocolo y la brecha; no transformo una propuesta en un éxito histórico. Así puedo defender el trabajo y decidir el siguiente incremento con evidencia.

Cuestionario

Seleccione una única respuesta.

1. La función técnica de COPY --from=builder en un Dockerfile multietapa es:

- A. Importar dependencias de bases de datos externas directamente al contenedor.
- B. Copiar únicamente los archivos compilados de producción desde una etapa temporal hacia la imagen final minimalista. ✅
- C. Copiar todo el historial de Git al contenedor.
- D. Duplicar la memoria RAM asignada al contenedor.

2. Si un contenedor se destruye sin un volumen persistente configurado, los datos internos:

- A. Se guardan automáticamente en una base SQLite de respaldo.
- B. Se pierden de forma irreversible debido a la naturaleza efímera del contenedor. ✅
- C. Se suben automáticamente a un repositorio público.
- D. Se recuperan solos al reiniciar el sistema.

3. La diferencia esencial entre contenedor y máquina virtual es que:

- A. Ambos requieren su propio sistema operativo huésped completo.
- B. El contenedor comparte el kernel del host, aislando solo procesos en espacio de usuario, lo que lo hace más eficiente. ✅
- C. La máquina virtual siempre pesa menos que un contenedor.
- D. No existe ninguna diferencia técnica relevante.

4. Ejecutar un contenedor con privilegios de root por defecto representa un riesgo porque:

- A. Mejora automáticamente el rendimiento del backend.
- B. Expone el sistema host si el backend tiene una vulnerabilidad explotable. ✅
- C. Es un requisito obligatorio de Docker Compose.
- D. Reduce el tamaño de la imagen final.

5. Reordenar el Dockerfile para copiar primero el archivo de dependencias (package.json/requirements.txt) antes que el resto del código tiene como propósito:

- A. Aumentar el tamaño final de la imagen.
- B. Aprovechar la caché de compilación de capas de Docker, acelerando builds posteriores. ✅
- C. Evitar el uso de Docker Compose.
- D. Forzar la reinstalación completa de dependencias en cada build.

6. Docker Compose permite principalmente:

- A. Sustituir por completo la necesidad de un Dockerfile.

- B. Definir y ejecutar aplicaciones de múltiples contenedores de forma declarativa y coordinada. ✅
- C. Eliminar la necesidad de redes virtuales.
- D. Compilar código fuente sin necesidad de contenedores.

7. Según la regla de oro de esta guía, las credenciales de base de datos:

- A. Pueden escribirse directamente en el docker-compose.yml si el repositorio es privado.
- B. Nunca deben quedar en duro; se externalizan mediante variables de entorno ignoradas en Git. ✅
- C. Deben imprimirse en los logs para fines de auditoría.
- D. No son necesarias si se usa una imagen oficial de PostgreSQL.

Cuestionario resuelto: 1. B; 2. B; 3. B; 4. B; 5. B; 6. B; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-26

Contenerización y Orquestación del Proyecto Integrador

Esta ficha continúa la FPI-25: empaqueta el MVP como el artefacto inmutable que tu pipeline de CI construirá y verificará.

A. Identificación de la ficha

Código de la ficha	FPI-26
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	Contenerización avanzada: Multi-Stage Builds y Docker Compose

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Recuperar y validar la FPI-25

Pipeline de CI (recuperado de la FPI-25)

Resume el estado del pipeline que consumirá el artefacto Docker de esta ficha.

Respuesta: CI valida backend/frontend; no se encontró publicación de imagen vinculada a un release en esta revisión.

¿Algo cambió desde la ficha anterior?

Qué cambió	Por qué cambió	Impacto en el proyecto
Estado del producto	Las16 guías anteriores eran un corte antiguo	Se usa matriz vigente y se conserva historial
Tipo de beneficiario	SaaS multiempresa según estudiante	No se simula una empresa aliada única

Si nada cambió, escribe "Sin cambios" en la primera fila y deja la tabla así.

C. Aplicación específica — Contenerización

1. Enlaces a los archivos Dockerfile y docker-compose.yml

Rutas en el repositorio.

Respuesta: backend/Dockerfile; frontend/Dockerfile; compose.yaml. Propuesta multietapa incluida en taller, no reemplaza archivos.

2. Informe de optimización de peso de imagen

Imagen base inicial sugerida por IA	Imagen final optimizada	Reducción de peso (MB/%)
python:3.13-slim actual	Propuesta runtime no root con wheels	N/D: no se midieron imágenes antes/después

3. Evidencia de ejecución de docker compose up -d

Captura de terminal mostrando los contenedores corriendo con volumen persistente.

Respuesta: docs/IMPLEMENTATION_STATUS.md registra Compose operativo el18/10. No se adjunta una captura actual fabricada; para repetir: docker compose ps y logs, sin borrar volumen.

Registro de Decisión Arquitectónica — ADR N.° 10

Título	ADR-10 Docker y persistencia
Estado	Aceptado como registro académico; implementación y brechas descritas por separado
Contexto	SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar. Docker y persistencia
Decisión	Distinguir Compose local operativo de una imagen productiva endurecida, aún mejorable.
Consecuencias (positivas y negativas)	Menor complejidad y límites explícitos. Requiere medición, aceptación y revisión externa para completar evidencia.

D. Reflexión y trazabilidad

¿Qué aprendimos?

Resume lo aprendido sobre contenerización y orquestación.

Respuesta: COPY --from=builder copia artefactos de una etapa previa a la imagen final. Permite dejar compiladores y cachés fuera, pero no garantiza que librerías nativas de ejecución estén presentes. Deben verificarse compatibilidad y permisos.

¿Cómo se aplica al proyecto?

Explica cómo la contenerización facilitará el despliegue del MVP.

Respuesta: Distinguir Compose local operativo de una imagen productiva endurecida, aún mejorable.

Decisiones tomadas

Decisión	Justificación
Distinguir Compose local operativo de una imagen productiva endurecida, aún mejorable.	Alineación con riesgo financiero y capacidad individual
Separar evidencia y propuesta	Evitar certificar despliegues, revisiones o mediciones no realizados

Evidencia generada

Estado documentado: Dockerfile multietapa funcional. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: docker-compose.yml con volumen y red aislada. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: ADR N.° 10 registrado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Dificultades o ajustes identificados

¿Qué parte de la contenerización generó más errores?

Respuesta: Compatibilidad de wheels nativos y permisos de usuario runtime; todavía requiere build de la propuesta.

Próximo paso

¿Qué debe hacer el equipo antes de la siguiente guía?

Respuesta: Validar imagen y plan de publicación con healthcheck y rollback.

Conexión con la siguiente guía

La imagen Docker construida aquí es el artefacto que se desplegará automáticamente en la nube en la siguiente guía, mediante un pipeline de Despliegue Continuo (CD).

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5
Comprensión conceptual (15%)	Confunde contenedor con	Comprende conceptos básicos	Comprende lo esencial	Aplica correctamente Multi-Stage Builds	Explica y relaciona con profundidad

	máquina virtual				
Calidad del Dockerfile (25%)	No ejecuta o pesa más de 1GB	Dockerfile básico sin optimizar	Dockerfile funcional con optimización parcial	Dockerfile multietapa bien optimizado	Dockerfile ejemplar, ligero y no root
Calidad del Compose y persistencia (20%)	No orquesta o pierde datos	Orquestación básica sin volumen real	Orquestación funcional con volumen	Orquestación robusta con red aislada	Orquestación de nivel profesional
Rigor de la auditoría a la IA (15%)	No audita o copia sin procesar	Críticas superficiales	Detecta errores básicos	Auditoría robusta con corrección concreta	Auditoría excepcional y bien argumentada
Participación en el foro (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada
Trazabilidad respecto a la FPI-25 (15%)	No recupera lo construido antes	Copia sin verificar vigencia	Valida lo esencial	Valida y actualiza correctamente	Trazabilidad ejemplar con causa e impacto de cambios

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%).

Recursos académicos y bibliografía

- Docker Inc. Docker Documentation (Multi-Stage Builds).
- Docker Inc. Docker Compose Documentation.
- IEEE Computer Society. SWEBOK Guide, Version 4.0 (Software Engineering Operations).

UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 11 resuelta

DevOps: Entrega y Despliegue Continuo (CD) en Entornos de Nube

Asignatura	Ingeniería de Software II
Corte	Segundo Corte (Cierre de Corte)
Duración	Semana 11
Proyecto integrador	Fase 27: Despliegue continuo automatizado del MVP en la nube con política de rollback
Herramientas	GitHub Actions CD, Render/Railway, Docker Hub/GHCR + IA generativa auditada
Formato de entrega	Documento con taller resuelto y Ficha de Aplicación al Proyecto Integrador (FPI-27) diligenciada
Fecha límite	Domingo de la Semana 11, 11:59 p.m. (ver fecha exacta en el aula virtual)
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar.
Corte documental 30/10/2026. Se contrastaron código, compose.yaml, .github/workflows/ci.yml y docs/IMPLEMENTATION_STATUS.md. El registro del 18/10 informa 235 pruebas PostgreSQL: 233 aprobadas y 2 omitidas; es evidencia histórica, no una nueva ejecución de esta entrega. No se certifica producción actual, ASVS, p95 representativo ni validación con empresarios. Presupuesto inicial 0 USD y respaldo manual por decisión del proyecto.

Fase 27 del Proyecto Integrador — Cierre del Segundo Corte

Con tu MVP contenerizado en la FPI-26, llegas a la cumbre de los flujos de operaciones de software: el Despliegue Continuo. Automatizar la entrega de valor de forma ininterrumpida hacia el usuario final, mitigando el riesgo operacional de caídas de servicio, es una de las competencias más valoradas en la industria.

Esta guía retoma la imagen Docker de la FPI-26: configurarás el pipeline que la empaqueta y despliega automáticamente en la nube, con una política explícita de rollback ante fallos. El resultado se registra en la FPI-27.

Resultados de aprendizaje

- Diferenciar Continuous Delivery de Continuous Deployment.
- Explicar las estrategias de release Blue-Green y Canary Deployment.
- Configurar un pipeline de CD que despliegue automáticamente hacia una plataforma en la nube.
- Diseñar una política de rollback automático ante fallos en producción.
- Auditar críticamente un script de despliegue generado por IA, detectando exposición de secretos.

Objetivos

General

Automatizar el despliegue continuo del MVP contenerizado hacia una plataforma en la nube, con una política de rollback documentada.

Específicos

- Explicar por qué Continuous Deployment exige mayor madurez de pruebas que Continuous Delivery.
- Justificar cuándo Blue-Green o Canary Deployment aportarían valor a un proyecto como el suyo.
- Configurar un webhook de despliegue seguro con secretos dinámicos.
- Implementar un healthcheck posterior al despliegue.
- Registrar la evidencia del despliegue y el ADR de rollback en la FPI-27.

Saberes previos

Antes de iniciar, responde (no se califica):

- ¿Sabes qué es un webhook?

Respuesta del estudiante: Webhook es notificación HTTP entre sistemas. Un deploy hook es secreto de operación que debe almacenarse fuera del YAML.

- ¿Confiarías en desplegar automáticamente a producción cada vez que el pipeline esté en verde, sin aprobación humana?

Respuesta del estudiante: Para este SaaS prefiero entrega controlada: CI verde no acredita aceptación, restauración ni seguridad completa.

- ¿Tu MVP tiene una capa gratuita disponible en alguna plataforma cloud (Render, Railway)?

Respuesta del estudiante: Hay render.yaml y alojamiento histórico documentado. No se certifica que la rama actual esté publicada o el servicio disponible hoy.

Concepto 1 — Continuous Delivery vs. Continuous Deployment

Continuous Delivery automatiza la construcción y verificación, dejando la puesta en producción a una aprobación manual humana del negocio. Continuous Deployment automatiza de extremo a extremo el flujo, enviando cada commit verde directamente a los usuarios sin intervención manual. Elegir entre uno u otro es, otra vez, una decisión de trade-off (Guía 3): velocidad de innovación contra control humano del riesgo.

Concepto 2 — Estrategias de release: Blue-Green y Canary

Blue-Green mantiene dos entornos idénticos: "Blue" (versión estable en vivo) y "Green" (versión nueva). El tráfico se conmuta instantáneamente hacia Green; si se detecta un fallo, el router vuelve a Blue en milisegundos con cero pérdida de disponibilidad. Canary Deployment libera el cambio de forma gradual, enviando solo un porcentaje del tráfico (ej. 5%) a la versión nueva; si las métricas de error se mantienen estables, se expande al 100% progresivamente.

> *Analogía territorial: una fábrica de calzado en El Callejón despacha un lote hacia un almacén mayorista en el Puente Internacional Simón Bolívar. Blue-Green equivale a tener dos camiones idénticos en la frontera: el camión Azul (mercancía vieja aprobada) y el Verde (mercancía con diseño nuevo). El importador inspecciona el Verde; si las costuras vienen falladas, simplemente bloquea ese camión y continúa vendiendo el stock del Azul estable, garantizando que el almacén nunca se quede sin stock.*

Concepto 3 — Webhooks de despliegue y gestión segura de secretos

Un webhook de despliegue es una petición HTTP (generalmente POST) enviada por el pipeline para notificar automáticamente a la plataforma de hosting que compile o descargue la última imagen de producción. Las credenciales de ese webhook (tokens, contraseñas de Docker Hub) deben usar siempre secrets dinámicos cifrados del repositorio, nunca texto plano en el YAML.

Concepto 4 — Política de rollback y healthcheck posterior al despliegue

Ningún pipeline de CD es infalible: debe contemplar qué hacer si el despliegue falla. Un script de validación posterior (Post-Deploy Healthcheck) verifica el estado del endpoint /health tras el despliegue; si no responde correctamente, el pipeline debe alertar o revertir automáticamente al tag de versión estable anterior.

Regla de oro de esta guía: *"Ningún pipeline de CD se considera completo sin un healthcheck posterior al despliegue y una política explícita de rollback documentada."*

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, la diferencia entre Continuous Delivery y Continuous Deployment.

Respuesta del estudiante: Delivery deja una versión verificable lista para publicar con decisión explícita. Deployment publica automáticamente cada cambio que supera controles. Horizonte conserva una aprobación de entrega y pruebas posteriores para no confundir CI verde con validación.

Ejercicio 2

Explica, con un ejemplo propio, cómo Blue-Green Deployment permite un rollback en milisegundos ante un fallo detectado.

Respuesta del estudiante: Blue/Green mantiene dos versiones y conmuta tráfico a la saludable. Un fallo permite volver a la anterior mientras datos y migraciones sean compatibles. El cambio no garantiza milisegundos reales; hay conexiones activas, DNS o balanceadores y costos. No está implementado aquí.

Nivel Intermedio (pipeline de CD real + auditoría a la IA)

Ejercicio 3

Configura tu pipeline de CD real que despliegue tu imagen Docker (FPI-26) hacia una capa gratuita de Render o Railway.

Respuesta del estudiante: Existe render.yaml y evidencia histórica de alojamiento del 05/10. La rama local vigente no está certificada como desplegada. No se encontró .github/workflows/deploy.yml; se documenta propuesta manualmente disparada con deploy hook secreto y verificación posterior. No se publica ni crea servicio de pago desde esta tarea.

Ejercicio 4

Implementa un script de healthcheck posterior al despliegue que verifique el endpoint /health y falle el pipeline si el sitio no responde.

Respuesta del estudiante: Healthcheck propuesto: curl --fail --silent --show-error --max-time 10 "$BASE_URL/api/health/"; reintentar con backoff hasta un límite de arranque, y salir con error si no responde. El endpoint real es /api/health/, no /health. Comprobar JSON esperado y dependencias según lo que garantice su implementación.

Ejercicio 5

Pide a una IA generativa que genere el workflow de CD. Audita: ¿propuso escribir la contraseña de Docker Hub en claro dentro del YAML? ¿el script asume que el despliegue es infalible y no contempla un healthcheck? Documenta el prompt, la respuesta y tu corrección.

Respuesta del estudiante: Prompt: diseña entrega con secreto y rollback sin asumir dos réplicas gratis. Respuesta: workflow_dispatch, deploy hook desde secrets, espera acotada y healthcheck. Auditoría: respuesta 200 sola no prueba readiness financiera; versión anterior debe conservarse y migraciones ser compatibles. No incluir token o contraseña en YAML ni asegurar que rollback de BD es automático.

Artefacto académico completo — propuesta o ejemplo según respuesta anterior

```
# Propuesta Delivery, no ejecutada ni publicada
name: Delivery
on: workflow_dispatch
jobs:
  release:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - name: Solicitar despliegue y verificar
        env:
          HOOK: ${{ secrets.RENDER_DEPLOY_HOOK }}
          BASE_URL: ${{ vars.BASE_URL }}
        run: |
          test -n "$HOOK" && test -n "$BASE_URL"
          curl --fail --silent --show-error --max-time 30 -X POST "$HOOK" >/dev/null
          for i in $(seq 1 30); do
            if curl --fail --silent --max-time 10 "$BASE_URL/api/health/"; then exit 0; fi
            sleep 10
          done
          exit 1
# Añadir comprobación de revisión desplegada: health 200 antiguo no acredita nueva versión.
```

Lista de autoevaluación antes de entregar

- Diferencio Continuous Delivery de Continuous Deployment y justifiqué cuál aplica a mi proyecto.
- Mi pipeline de CD usa secretos dinámicos, nunca contraseñas en texto plano.
- Implementé un healthcheck posterior al despliegue.
- Documenté una política de rollback explícita.
- Diligencié completamente la FPI-27 con la URL pública de mi MVP en producción.

Foro de reflexión

Título: *"¿Continuous Deployment o Continuous Delivery para la MiPyme tradicional?"*

Redacte una reflexión de 250–300 palabras respondiendo: automatizar de extremo a extremo sin intervención humana acelera la velocidad de innovación, pero si un bug se salta los tests y daña la contabilidad de la MiPyme en producción, las pérdidas pueden ser catastróficas. ¿Es preferible la aprobación manual de un directivo local, o la automatización total? Justifique basándose en Jez Humble.

Reflexión del estudiante (283 palabras)
En Horizonte, considero que elegir Continuous Delivery con aprobación de publicación y healthcheck, compatible con presupuesto cero. Delivery deja una versión verificable lista para publicar con decisión explícita. Deployment publica automáticamente cada cambio que supera controles. Horizonte conserva una aprobación de entrega y pruebas posteriores para no confundir CI verde con validación.

El caso concreto que orienta mi análisis es el siguiente: Existe render.yaml y evidencia histórica de alojamiento del 05/10. La rama local vigente no está certificada como desplegada. No se encontró .github/workflows/deploy.yml; se documenta propuesta manualmente disparada con deploy hook secreto y verificación posterior. No se publica ni crea servicio de pago desde esta tarea.

Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia.

La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados.

Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones. Si falta medición, registro el protocolo y la brecha; no transformo una propuesta en un éxito histórico. Así puedo defender el trabajo y decidir el siguiente incremento con evidencia.

Cuestionario

Seleccione una única respuesta.

1. La diferencia fundamental en la conmutación de tráfico de Blue-Green Deployment es que:

- A. El tráfico se distribuye aleatoriamente entre todos los usuarios.
- B. Mantiene dos entornos idénticos y desvía el 100% del tráfico al entorno nuevo mediante enrutamiento instantáneo, permitiendo rollbacks en milisegundos. ✅

- C. Apaga el servidor Blue varias horas para copiar archivos al servidor Green.
- D. Solo permite acceso a usuarios administradores mediante VPN.

2. Un Webhook de Despliegue es:

- A. Un virus diseñado para inyectar scripts en la base de datos de producción.
- B. Una petición HTTP enviada por el pipeline para notificar a la plataforma de hosting que despliegue la última imagen. ✅
- C. Un protocolo de encriptación asimétrica de disco duro.
- D. Un programa de diseño gráfico en la nube.

3. Continuous Deployment, a diferencia de Continuous Delivery:

- A. Requiere siempre aprobación manual antes de producción.
- B. Automatiza de extremo a extremo el flujo, enviando cada commit verde directamente a producción sin intervención manual. ✅
- C. No permite el uso de pipelines automatizados.
- D. Solo aplica a proyectos sin pruebas automatizadas.

4. En Canary Deployment, el cambio se libera:

- A. Al 100% de los usuarios de forma inmediata.
- B. De forma gradual, enviando primero un porcentaje del tráfico y expandiendo progresivamente si las métricas son estables. ✅
- C. Solo los fines de semana.
- D. Sin ningún tipo de monitoreo posterior.

5. Escribir la contraseña de Docker Hub en claro dentro del YAML de un workflow de CD es un riesgo porque:

- A. Ralentiza el pipeline de forma significativa.
- B. Expone la credencial a cualquiera con acceso al repositorio, comprometiendo la seguridad del despliegue. ✅
- C. Es la práctica recomendada por GitHub Actions.
- D. Solo afecta el rendimiento, no la seguridad.

6. Un healthcheck posterior al despliegue tiene como propósito:

- A. Medir el consumo de CPU del desarrollador.
- B. Verificar que el sistema desplegado responde correctamente, permitiendo detectar fallos justo después del despliegue. ✅
- C. Sustituir la necesidad de pruebas unitarias.
- D. Configurar automáticamente el DNS del dominio.

7. Según la regla de oro de esta guía, un pipeline de CD:

- A. Está completo con solo desplegar el código, sin verificación adicional.
- B. No se considera completo sin un healthcheck posterior y una política explícita de rollback. ✅
- C. No necesita documentar cómo revertir un despliegue fallido.
- D. Debe evitar cualquier tipo de automatización de despliegue.

Cuestionario resuelto: 1. B; 2. B; 3. B; 4. B; 5. B; 6. B; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-27

Despliegue Continuo del Proyecto Integrador

Esta ficha continúa la FPI-26: despliega automáticamente el artefacto Docker construido en esa ficha hacia una plataforma en la nube.

A. Identificación de la ficha

Código de la ficha	FPI-27
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	Entrega y Despliegue Continuo (CD) en la nube

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Recuperar y validar la FPI-26

Imagen Docker (recuperada de la FPI-26)

Resume el estado de la imagen que se desplegará en esta ficha.

Respuesta: Imagen actual local funcional para desarrollo, con deuda USER/runserver; propuesta productiva aún no verificada.

¿Algo cambió desde la ficha anterior?

Qué cambió	Por qué cambió	Impacto en el proyecto
Estado del producto	Las16 guías anteriores eran un corte antiguo	Se usa matriz vigente y se conserva historial
Tipo de beneficiario	SaaS multiempresa según estudiante	No se simula una empresa aliada única

Si nada cambió, escribe "Sin cambios" en la primera fila y deja la tabla así.

C. Aplicación específica — Despliegue continuo

1. Enlace al workflow de CD

Ruta del archivo .github/workflows/deploy.yml

Respuesta: No existe deploy.yml en la revisión. Propuesta de workflow se incluye en taller con estado no integrado.

2. URL pública de la aplicación en producción

Enlace vivo, ej. https://api-mipyme.onrender.com/health Respuesta: Ruta de salud verificada en compose: /api/health/. Evidencia remota histórica del05/10; no se declara una URL viva actual sin comprobarla.

Registro de Decisión Arquitectónica — ADR N.° 11

Título	ADR-11 Entrega controlada
Estado	Aceptado como registro académico; implementación y brechas descritas por separado
Contexto	SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar. Entrega controlada
Decisión	Elegir Continuous Delivery con aprobación de publicación y healthcheck, compatible con presupuesto cero.
Consecuencias (positivas y negativas)	Menor complejidad y límites explícitos. Requiere medición, aceptación y revisión externa para completar evidencia.
Políticas de recuperación ante fallos (rollback)	Elegir Continuous Delivery con aprobación de publicación y healthcheck, compatible con presupuesto cero.

D. Reflexión y trazabilidad

¿Qué aprendimos?

Resume lo aprendido sobre CD, Blue-Green/Canary y rollback. Respuesta: Delivery deja una versión verificable lista para publicar con decisión explícita. Deployment publica automáticamente cada cambio que supera controles. Horizonte conserva una aprobación de entrega y pruebas posteriores para no confundir CI verde con validación.

¿Cómo se aplica al proyecto?

Explica cómo el MVP ahora se actualiza automáticamente en producción. Respuesta: No se acredita actualización automática de producción de esta rama. La política es Delivery con aprobación y healthcheck.

Decisiones tomadas

Decisión	Justificación
Elegir Continuous Delivery con aprobación de publicación y healthcheck, compatible con presupuesto cero.	Alineación con riesgo financiero y capacidad individual
Separar evidencia y propuesta	Evitar certificar despliegues, revisiones o mediciones no realizados

Evidencia generada

Estado documentado: Pipeline de CD funcional y verde. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: URL pública accesible con healthcheck. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: ADR N.° 11 de rollback registrado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Dificultades o ajustes identificados

¿Qué parte del despliegue generó más errores?

Respuesta: Arranque frío, migraciones y compatibilidad de datos limitan rollback.

Próximo paso

¿Qué debe hacer el equipo antes de la siguiente guía?

Respuesta: Incorporar controles SAST/SCA antes de promover candidato.

Conexión con la siguiente guía

Con el MVP corriendo en producción de forma automatizada, el Tercer Corte se enfoca en blindar ese entorno: la siguiente guía integra seguridad (DevSecOps) directamente en tu pipeline.

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5
Comprensión conceptual (15%)	Confunde Delivery con Deployment	Comprende conceptos básicos	Comprende lo esencial	Diferencia y aplica correctamente	Explica y relaciona con profundidad
Funcionamiento del pipeline de CD (30%)	No despliega o falla críticamente	Despliegue manual parcial	Despliegue automatizado funcional	Despliegue robusto con healthcheck	Despliegue de nivel profesional,

					100% automatizado
Gestión segura de secretos (15%)	Credenciales expuestas en claro	Gestión parcial de secretos	Gestión aceptable con secrets dinámicos	Gestión robusta y bien documentada	Gestión ejemplar sin ninguna exposición
Rigor de la auditoría a la IA (15%)	No audita o copia sin procesar	Críticas superficiales	Detecta riesgos básicos	Auditoría robusta con corrección concreta	Auditoría excepcional y bien argumentada
Participación en el foro (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada
Trazabilidad respecto a la FPI-26 (15%)	No recupera lo construido antes	Copia sin verificar vigencia	Valida lo esencial	Valida y actualiza correctamente	Trazabilidad ejemplar con causa e impacto de cambios

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%).

Recursos académicos y bibliografía

- Humble, Jez; Farley, David. Continuous Delivery.
- Fowler, Martin. BlueGreenDeployment (martinfowler.com).
- IEEE Computer Society. SWEBOK Guide, Version 4.0 (Software Engineering Operations).

UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 12 resuelta

DevSecOps: Seguridad en el Pipeline, Escaneo de Vulnerabilidades (SAST/SCA) y Gestión de Secretos

Asignatura	Ingeniería de Software II
Corte	Tercer Corte (Inicio)
Duración	Semana 12
Proyecto integrador	Fase 28: Blindaje del pipeline con escaneo automático de vulnerabilidades y secretos
Herramientas	Trivy/Semgrep, GitHub Dependabot, GitGuardian + IA generativa auditada
Formato de entrega	Documento con taller resuelto y Ficha de Aplicación al Proyecto Integrador (FPI-28) diligenciada
Fecha límite	Domingo de la Semana 12, 11:59 p.m. (ver fecha exacta en el aula virtual)
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar.
Corte documental 30/10/2026. Se contrastaron código, compose.yaml, .github/workflows/ci.yml y docs/IMPLEMENTATION_STATUS.md. El registro del 18/10 informa 235 pruebas PostgreSQL: 233 aprobadas y 2 omitidas; es evidencia histórica, no una nueva ejecución de esta entrega. No se certifica producción actual, ASVS, p95 representativo ni validación con empresarios. Presupuesto inicial 0 USD y respaldo manual por decisión del proyecto.

Fase 28 del Proyecto Integrador — Evolución del MVP

Con tu MVP desplegándose automáticamente en producción (FPI-27), comienza el Tercer Corte: tratar la seguridad como una auditoría de inspección final es extremadamente riesgoso y costoso. Corregir una inyección SQL o una dependencia vulnerable cuando el sistema ya opera en producción exige parches en caliente apresurados que amenazan la estabilidad. Esta guía integra la seguridad directamente en tu pipeline, bajo el principio de Shift-Left Security.

Esta guía retoma el pipeline de CI de la FPI-25: le añadirás etapas de escaneo automático de vulnerabilidades y secretos que bloqueen la fusión de código inseguro antes de llegar a producción. El resultado se registra en la FPI-28.

Resultados de aprendizaje

- Explicar el principio de Shift-Left Security.
- Diferenciar el análisis estático de seguridad (SAST) del análisis de composición de software (SCA).
- Integrar un escáner de vulnerabilidades (Trivy/Semgrep) que falle el build ante hallazgos críticos.
- Gestionar secretos de forma externalizada, sin credenciales en texto plano.
- Auditar críticamente falsos positivos de seguridad sugeridos por IA.

Objetivos

General

Integrar disciplina DevSecOps en el pipeline del MVP, detectando y bloqueando vulnerabilidades de código y dependencias antes del despliegue.

Específicos

- Explicar por qué introducir seguridad al final del ciclo de vida es más costoso que hacerlo desde el principio.
- Configurar un escaneo SAST que detecte patrones de código inseguro.
- Configurar un escaneo SCA que detecte dependencias con CVE conocidos.
- Filtrar falsos positivos sin ignorar vulnerabilidades reales.
- Registrar el informe de vulnerabilidades y su mitigación en la FPI-28.

Saberes previos

Antes de iniciar, responde (no se califica):

- ¿Sabes qué es un CVE (Common Vulnerabilities and Exposures)?

Respuesta del estudiante: CVE identifica una vulnerabilidad divulgada; su severidad y aplicabilidad se contrastan con versión y exposición real.

- ¿Alguna vez tu proyecto usó una librería desactualizada sin saberlo?

Respuesta del estudiante: La documentación registra actualización de source-map-js; no deduzco CVE ni exposición de cualquier dependencia antigua sin escaneo actual.

- ¿Tu pipeline actual (FPI-25) incluye algún paso de escaneo de seguridad?

Respuesta del estudiante: El CI inspeccionado tiene Ruff y tests, pero no un gate dedicado SAST/SCA completo demostrado.

Concepto 1 — Shift-Left Security

Enfoque que introduce las verificaciones de seguridad en las fases más tempranas del ciclo de vida, de manera que las vulnerabilidades se corrijan antes de que el código sea fusionado, en vez de descubrirlas en una auditoría de penetración semanas antes del lanzamiento.

> *Analogía territorial: el puesto de control aduanero en el Puente Internacional Simón Bolívar decide inspeccionar si los camiones traen mercancía peligrosa. Si la DIAN inspeccionara cada camión manualmente al final de la vía en Cúcuta, la fila colapsaría la autopista. Para evitarlo, la aduana implementa un escáner preventivo en la entrada de la zona de pesaje local: si detecta mercancía sospechosa, detiene el camión inmediatamente en la fase de carga, evitando el colapso de la autopista.*

Concepto 2 — SAST y SCA

Herramienta	Qué examina
SAST (Static Application Security Testing)	El código fuente propio, sin ejecutarlo, buscando patrones inseguros: inyección SQL directa, hashes obsoletos (MD5), rutas de escape no seguras.
SCA (Software Composition Analysis)	La cadena de suministro de dependencias de terceros, cruzando sus firmas contra bases de datos públicas de vulnerabilidades conocidas (CVE).

Concepto 3 — Filtrado de falsos positivos sin comprometer la seguridad

No toda alerta de un escáner es crítica: una dependencia vulnerable que solo se usa en desarrollo (devDependencies) y nunca llega a producción representa un riesgo mucho menor que una en el runtime real. La respuesta correcta no es ignorar el escáner por completo, sino configurar filtros justificados (ej. archivo .trivyignore) documentando por qué cada exclusión es segura.

Concepto 4 — Gestión de secretos y reportes privados

Los reportes de vulnerabilidades nunca deben publicarse en un servidor externo público: exponen el mapa de fallos del sistema a un potencial atacante. Deben almacenarse como artefactos privados de GitHub Actions o en el dashboard de Security del repositorio.

Regla de oro de esta guía: *"El pipeline debe fallar (código de salida distinto de 0) ante cualquier vulnerabilidad crítica o alta sin mitigar; deshabilitar esa falla para 'no atrasar la entrega' anula por completo el propósito de DevSecOps."*

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, la diferencia entre SAST y SCA.

Respuesta del estudiante: SAST analiza código y patrones inseguros sin ejecutar la aplicación; SCA analiza componentes y vulnerabilidades/licencias de dependencias. Se complementan con prueba dinámica y revisión humana. Cero hallazgos de una herramienta no demuestra ausencia de vulnerabilidades.

Ejercicio 2

Explica por qué corregir una vulnerabilidad de inyección SQL en producción es más costoso que detectarla antes de fusionar el Pull Request.

Respuesta del estudiante: Antes del merge se corrige un fragmento y se repite test; después puede requerir contención, recuperación, revisión de accesos y comunicación a afectados. El costo depende de impacto y datos. En Horizonte se preserva ORM parametrizado y aislamiento por empresa.

Nivel Intermedio (escaneo real + auditoría a la IA)

Ejercicio 3

Integra Trivy o Semgrep en tu pipeline real, configurando que el build falle ante una vulnerabilidad crítica o alta.

Respuesta del estudiante: No existe ci-security.yml operativo en la fuente inspeccionada. Se incluye especificación de job propuesto: escanear fuente e imagen con herramienta aprobada; fallo en HIGH/CRITICAL mediante exit-code 1; artefacto privado y retención limitada. Integrar y verificar herramienta/versiones es tarea posterior, no escaneo ejecutado.

Ejercicio 4

Si tu escáner detecta un falso positivo real, documenta por qué lo consideras seguro de ignorar y cómo lo configuraste en el archivo de exclusiones.

Respuesta del estudiante: No se observó un falso positivo real documentado para esta entrega. No crear exclusiones preventivas. Toda excepción futura requiere ID, versión, evidencia de no explotabilidad, responsable y vencimiento; severidad alta no se silencia solo para poner CI verde.

Ejercicio 5

Pide a una IA generativa que configure el workflow de seguridad. Audita: ¿usó parámetros deprecados de la herramienta? ¿propuso subir el reporte de vulnerabilidades a un servidor público externo? Documenta el prompt, la respuesta y tu corrección.

Respuesta del estudiante: Prompt: configura revisión de seguridad sin exportar datos sensibles. Respuesta: resultados privados, sin body de solicitudes y gate bloqueante por severidad. Auditoría: no se ejecutó SCA hoy y no se inventan CVE; las opciones deben contrastarse con versión fijada al instalar. Un secreto encontrado exige revocación/rotación, no solo borrarlo del último commit.

Artefacto académico completo — propuesta o ejemplo según respuesta anterior

```
# Especificación del gate para integrar tras fijar versión de Trivy
# trivy fs --severity HIGH,CRITICAL --exit-code 1 .
# trivy image --severity HIGH,CRITICAL --exit-code 1 horizonte:candidato
# No ejecutar con continue-on-error. No usar ignore-unfixed para ocultar riesgo.
# Registrar hash de imagen, versión del escáner, fecha y base de vulnerabilidades.
```

Lista de autoevaluación antes de entregar

- Mi pipeline falla (código de salida distinto de 0) ante vulnerabilidades críticas o altas.
- Filtré falsos positivos con justificación documentada, sin ignorar riesgos reales.
- Mis reportes de seguridad se almacenan de forma privada, no en un servidor público.
- Audité si la IA propuso comandos deprecados o exposición de reportes.
- Diligencié completamente la FPI-28 con mi informe de vulnerabilidades corregidas.

Foro de reflexión

Título: *"¿Es la seguridad un freno costoso a la agilidad del desarrollo en las MiPymes regionales?"*

Redacte una reflexión de 250–300 palabras respondiendo: implementar DevSecOps obliga a detener pipelines por dependencias desactualizadas, ralentizando la entrega. Si el cliente solo desea ver su sitio operando rápido, ¿cómo justifica un ingeniero el costo adicional en horas que implica blindar el pipeline? Debata con bases legales (Ley 1581 de 2012).

Reflexión del estudiante (282 palabras)
En Horizonte, considero que añadir análisis bloqueante y revisión humana de hallazgos, sin inventar CVE ni enviar reportes sensibles. SAST analiza código y patrones inseguros sin ejecutar la aplicación; SCA analiza componentes y vulnerabilidades/licencias de dependencias. Se complementan con prueba dinámica y revisión humana. Cero hallazgos de una herramienta no demuestra ausencia de vulnerabilidades.

El caso concreto que orienta mi análisis es el siguiente: No existe ci-security.yml operativo en la fuente inspeccionada. Se incluye especificación de job propuesto: escanear fuente e imagen con herramienta aprobada; fallo en HIGH/CRITICAL mediante exit-code 1; artefacto privado y retención limitada. Integrar y verificar herramienta/versiones es tarea posterior, no escaneo ejecutado.

Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia.

La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados.

Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones. Si falta medición, registro el protocolo y la brecha; no transformo una propuesta en un éxito histórico. Así puedo defender el trabajo y decidir el siguiente incremento con evidencia.

Cuestionario

Seleccione una única respuesta.

1. La diferencia fundamental entre SAST y SCA es que:

- A. SAST escanea la interfaz gráfica y SCA el rendimiento de la base de datos.
- B. SAST examina el código propio buscando debilidades semánticas; SCA analiza dependencias de terceros buscando CVEs conocidos. ✅
- C. SAST requiere desplegar la aplicación en vivo; SCA se ejecuta solo localmente.
- D. SAST analiza binarios compilados y SCA lee especificaciones OpenAPI.

2. El principio de Shift-Left Security consiste en:

- A. Ejecutar una auditoría de penetración manual una semana antes del lanzamiento final.
- B. Configurar en el pipeline un escaneo automático de secretos y vulnerabilidades en cada Pull Request antes de la fusión. ✅

- C. Instalar un antivirus en el servidor de producción del cliente.
- D. Encriptar el código fuente en el disco duro de cada desarrollador.

3. Una dependencia vulnerable que solo se usa en devDependencies y nunca llega a producción:

- A. Siempre debe bloquear el pipeline con la misma severidad que una en producción.
- B. Puede filtrarse de forma justificada, ya que su riesgo real es menor. ✅
- C. Nunca debe reportarse por el escáner de seguridad.
- D. Debe eliminarse automáticamente el proyecto completo.

4. Publicar el reporte de vulnerabilidades en un servidor web público externo es riesgoso porque:

- A. Aumenta el tiempo de ejecución del pipeline.
- B. Expone el mapa de fallos de seguridad de la MiPyme a un potencial atacante. ✅
- C. Es obligatorio hacerlo según ISO/IEC/IEEE 12207.
- D. Reduce el rendimiento del servidor de producción.

5. Configurar el pipeline para que siempre retorne código de salida 0, sin importar los hallazgos de seguridad, representa:

- A. Una buena práctica de DevSecOps.
- B. Anular el propósito del Quality Gate de seguridad, dejándolo meramente informativo. ✅
- C. El comportamiento esperado y recomendado de Trivy.
- D. Una mejora de rendimiento del pipeline.

6. Un archivo `.trivyignore` debe usarse para:

- A. Ocultar todas las vulnerabilidades sin excepción.
- B. Excluir de forma justificada y documentada hallazgos que representan un riesgo real controlado o irrelevante. ✅
- C. Eliminar la necesidad de todo escaneo de seguridad.
- D. Sustituir la necesidad de pruebas unitarias.

7. Según la regla de oro de esta guía, el pipeline debe:

- A. Fallar siempre que exista cualquier código nuevo, sin importar el resultado del escaneo.
- B. Fallar ante cualquier vulnerabilidad crítica o alta sin mitigar. ✅
- C. Ignorar los hallazgos de seguridad si el cliente tiene prisa.
- D. Ejecutarse solo una vez al mes.

Cuestionario resuelto: 1. B; 2. B; 3. B; 4. B; 5. B; 6. B; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-28

Pipeline Asegurado del Proyecto Integrador

Esta ficha continúa la FPI-25/27: blinda el pipeline existente con escaneo automático de vulnerabilidades y gestión segura de secretos.

A. Identificación de la ficha

Código de la ficha	FPI-28
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	DevSecOps: SAST/SCA y gestión de secretos

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Recuperar y validar la FPI-27

Pipeline de CI/CD (recuperado de fichas anteriores)

Resume el estado del pipeline al que se añadirá el escaneo de seguridad.

Respuesta: CI real con Ruff/tests/build; no existe gate de seguridad dedicado demostrado.

¿Algo cambió desde la ficha anterior?

Qué cambió	Por qué cambió	Impacto en el proyecto
Estado del producto	Las16 guías anteriores eran un corte antiguo	Se usa matriz vigente y se conserva historial
Tipo de beneficiario	SaaS multiempresa según estudiante	No se simula una empresa aliada única

Si nada cambió, escribe "Sin cambios" en la primera fila y deja la tabla así.

C. Aplicación específica — Seguridad del pipeline

1. Enlace al pipeline asegurado

Ruta del archivo .github/workflows/ci-security.yml

Respuesta: ci-security.yml no integrado; especificación bloqueante incluida en el taller.

2. Informe de vulnerabilidades detectadas y corregidas

Componente/Dependencia	Herramienta	Vulnerabilidad (CVE)	Severidad	Mitigación aplicada
Backend y dependencias	SAST/SCA propuestos	N/D: sin escaneo actual	No determinada	No excluir sin evidencia; integrar gate privado
Docker runtime	Revisión Dockerfile	No corresponde CVE	Riesgo por evaluar	Propuesta USER explícito y servidor productivo

Registro de Decisión Arquitectónica — ADR N.° 12

Título	ADR-12 DevSecOps
Estado	Aceptado como registro académico; implementación y brechas descritas por separado
Contexto	SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar. DevSecOps
Decisión	Añadir análisis bloqueante y revisión humana de hallazgos, sin inventar CVE ni enviar reportes sensibles.
Consecuencias (positivas y negativas)	Menor complejidad y límites explícitos. Requiere medición, aceptación y revisión externa para completar evidencia.
Riesgo territorial mitigado	Añadir análisis bloqueante y revisión humana de hallazgos, sin inventar CVE ni enviar reportes sensibles.

D. Reflexión y trazabilidad

¿Qué aprendimos?

Resume lo aprendido sobre Shift-Left Security, SAST y SCA.

Respuesta: SAST analiza código y patrones inseguros sin ejecutar la aplicación; SCA analiza componentes y vulnerabilidades/licencias de dependencias. Se complementan con prueba dinámica y revisión humana. Cero hallazgos de una herramienta no demuestra ausencia de vulnerabilidades.

¿Cómo se aplica al proyecto?

Explica qué tan seguro es ahora tu pipeline.

Respuesta: Hay controles de sesión/CSRF/tenant y threat model; el pipeline no está certificado seguro ni con cero vulnerabilidades.

Decisiones tomadas

Decisión	Justificación
Añadir análisis bloqueante y revisión humana de hallazgos, sin inventar CVE ni enviar reportes sensibles.	Alineación con riesgo financiero y capacidad individual
Separar evidencia y propuesta	Evitar certificar despliegues, revisiones o mediciones no realizados

Evidencia generada

Estado documentado: Pipeline con escaneo SAST/SCA activo y bloqueante. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Informe de vulnerabilidades corregidas. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: ADR N.° 12 registrado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Dificultades o ajustes identificados

¿Qué vulnerabilidad fue más difícil de corregir?

Respuesta: No hay CVE actual verificado que deba atribuirse al proyecto. No inventar una vulnerabilidad para llenar la ficha.

Próximo paso

¿Qué debe hacer el equipo antes de la siguiente guía?

Respuesta: Elegir señales mínimas de observabilidad y revisar PII.

Conexión con la siguiente guía

Con la seguridad blindada, la siguiente guía te enseñará a que tu sistema "cuente su propia historia" en producción mediante observabilidad y telemetría.

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5
Comprensión conceptual (15%)	Confunde SAST con SCA	Comprende conceptos básicos	Comprende lo esencial	Aplica correctamente Shift-Left Security	Explica y relaciona con profundidad

Configuración del escaneo bloqueante (30%)	No escanea o no bloquea builds	Escaneo activo pero no bloqueante	Escaneo bloqueante básico	Escaneo robusto y bien calibrado	Escaneo de nivel profesional
Calidad del informe de vulnerabilidades (15%)	No reporta o reporte vacío	Informe superficial	Informe aceptable con mitigaciones básicas	Informe completo y bien documentado	Informe ejemplar y riguroso
Rigor de la auditoría a la IA (15%)	No audita o copia sin procesar	Críticas superficiales	Detecta riesgos básicos	Auditoría robusta con corrección concreta	Auditoría excepcional y bien argumentada
Participación en el foro (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada
Trazabilidad respecto a fichas anteriores (15%)	No recupera lo construido antes	Copia sin verificar vigencia	Valida lo esencial	Valida y actualiza correctamente	Trazabilidad ejemplar con causa e impacto de cambios

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%).

Recursos académicos y bibliografía

- NIST. Secure Software Development Framework (SSDF SP 800-218).
- Aqua Security. Trivy Documentation.
- OWASP. Top 10 Web Application Security Risks.
- IEEE Computer Society. SWEBOK Guide, Version 4.0 (Software Security).

UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 13 resuelta

Operación y Observabilidad: Telemetría (Logs, Métricas, Trazas) con OpenTelemetry y Grafana

Asignatura	Ingeniería de Software II
Corte	Tercer Corte
Duración	Semana 13
Proyecto integrador	Fase 29: Instrumentación de telemetría y dashboard de observabilidad del MVP
Herramientas	OpenTelemetry SDK, Prometheus, Grafana, Jaeger + IA generativa auditada
Formato de entrega	Documento con taller resuelto y Ficha de Aplicación al Proyecto Integrador (FPI-29) diligenciada
Fecha límite	Domingo de la Semana 13, 11:59 p.m. (ver fecha exacta en el aula virtual)
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar.
Corte documental 30/10/2026. Se contrastaron código, compose.yaml, .github/workflows/ci.yml y docs/IMPLEMENTATION_STATUS.md. El registro del 18/10 informa 235 pruebas PostgreSQL: 233 aprobadas y 2 omitidas; es evidencia histórica, no una nueva ejecución de esta entrega. No se certifica producción actual, ASVS, p95 representativo ni validación con empresarios. Presupuesto inicial 0 USD y respaldo manual por decisión del proyecto.

Fase 29 del Proyecto Integrador — Evolución del MVP

Con tu MVP asegurado y desplegado (FPI-28), un ingeniero de sistemas maduro no espera pasivamente a que el cliente reporte una falla. Un sistema profesional debe contar su propia historia de ejecución en tiempo real a través de datos medibles. Esta guía te enseña a instrumentar tu código para emitir señales de telemetría y visualizarlas en tableros interactivos.

Esta guía retoma el endpoint crítico que sometiste a prueba de carga en la FPI-24: lo instrumentarás con OpenTelemetry para observar su comportamiento real en producción, no solo en pruebas simuladas. El resultado se registra en la FPI-29.

Resultados de aprendizaje

- Diferenciar Monitoreo (métrica binaria reactiva) de Observabilidad (deducción del estado interno mediante señales).
- Explicar los tres pilares de la observabilidad: Logs estructurados, Métricas y Trazas distribuidas (M.E.L.T).
- Instrumentar un método real con OpenTelemetry (Span, métrica, log estructurado).
- Configurar un dashboard interactivo en Grafana/Prometheus.
- Auditar críticamente código de instrumentación generado por IA, previniendo fugas de PII en la telemetría.

Objetivos

General

Instrumentar el MVP con OpenTelemetry para observar su comportamiento en producción y visualizar sus señales en un dashboard funcional.

Específicos

- Explicar la diferencia entre monitoreo y observabilidad con un ejemplo propio.
- Instrumentar un método crítico con Span, métrica y log estructurado en formato JSON.
- Configurar una tasa de muestreo (sampling) que no degrade el rendimiento del sistema.
- Anonimizar datos personales (PII) antes de registrarlos en la telemetría.
- Registrar el dashboard y el ADR de retención de logs en la FPI-29.

Saberes previos

Antes de iniciar, responde (no se califica):

- ¿Alguna vez tuviste que adivinar por qué falló un sistema porque no había ningún registro (log) de lo que pasó?

Respuesta del estudiante: La falta de correlación dificulta diagnóstico; no atribuyo a esta entrega un incidente ocurrido. Propongo señales mínimas reproducibles.

- ¿Sabes qué es un identificador único (ID) y por qué sería útil para rastrear una petición a través de varios componentes?

Respuesta del estudiante: Un request_id aleatorio correlaciona pasos de una solicitud. No debe ser cédula, correo ni etiqueta de alta cardinalidad en métricas.

- ¿Tu código actual imprime datos personales de clientes (cédula, teléfono) en los logs de la consola?

Respuesta del estudiante: No se realizó auditoría exhaustiva de logs. No hay evidencia para certificar ausencia de PII; el diseño propuesto la excluye y exige revisión.

Concepto 1 — Observabilidad vs. Monitoreo

El monitoreo consiste en saber si un sistema específico está caído o levantado (métrica binaria reactiva externa). La observabilidad es la capacidad de deducir el estado interno de ejecución de un sistema complejo basándose únicamente en el análisis y correlación de las señales de telemetría emitidas, sin necesidad de adivinar o desarmar el sistema en producción.

> *Analogía territorial: en las alfarerías tradicionales de Villa del Rosario, el monitoreo tradicional de un horno de ladrillos es simplemente ver si la chimenea echa humo (prendido, sí o no). La observabilidad es contar con sensores térmicos y de presión digitales en cada cámara que envían datos en tiempo real. Si un lote sale agrietado, el ingeniero analiza las trazas de calor y presión de las últimas 12 horas para saber exactamente por qué falló el horno sin apagarlo ni inspeccionarlo físicamente.*

Concepto 2 — Los tres pilares de la observabilidad (M.E.L.T)

Pilar	Descripción
Logs estructurados	Registro inmutable de eventos discretos en formato de máquina estructurado (JSON), con metadatos del contexto de ejecución.
Métricas	Datos numéricos agregados en el tiempo (consumo de memoria, tasa de peticiones por segundo, número de excepciones).
Trazas distribuidas	Grafo del recorrido completo de una solicitud a través de múltiples componentes, identificado con TraceId/SpanId, permitiendo detectar cuellos de botella.

OpenTelemetry es el estándar de la CNCF para recopilar, procesar y exportar telemetría de forma unificada, independiente del proveedor de almacenamiento (vendor-agnostic).

Concepto 3 — Muestreo (Sampling) y overhead de rendimiento

Generar trazas y logs detallados consume espacio en disco, CPU y ancho de banda. Crear trazas en exceso, sin configurar una tasa de muestreo, puede degradar el rendimiento del propio sistema que se intenta observar. Una tasa de muestreo justificada (ej. 0.1 a 0.2) reduce ese overhead sin perder visibilidad estadística.

Concepto 4 — Privacidad en la telemetría (Ley 1581)

Registrar la cédula o el teléfono de un cliente directamente en los atributos de un Span o en el JSON de logs es una violación de la Ley 1581 de 2012, incluso si el propósito es solo de depuración técnica. Toda variable sensible debe ofuscarse o excluirse por completo del payload de telemetría.

Regla de oro de esta guía: *"Ningún dato personal identificable (PII) se registra en claro en un log, métrica o traza, sin importar qué tan útil parezca para depurar un problema."*

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, la diferencia entre monitoreo y observabilidad.

Respuesta del estudiante: Monitoreo detecta condiciones conocidas con indicadores; observabilidad permite investigar el estado interno a partir de señales. Horizonte necesita relacionar HTTP, cola y ejecución del cálculo sin registrar datos financieros de clientes.

Ejercicio 2

Explica por qué el formato JSON estructurado es preferible a un log de texto plano tradicional para el análisis automatizado.

Respuesta del estudiante: JSON aporta campos estables para filtrar y correlacionar, por ejemplo event, route, duration_ms y request_id. No es seguro por sí mismo: exige sanitizar excepciones, evitar PII y definir retención. Texto libre dificulta análisis pero puede seguir siendo útil en desarrollo.

Nivel Intermedio (instrumentación real + auditoría a la IA)

Ejercicio 3

Instrumenta con OpenTelemetry el endpoint crítico de tu MVP: crea un Span, emite una métrica agregada y escribe un log estructurado en JSON.

Respuesta del estudiante: Diseño propuesto: span forecast.dashboard, histograma de duración y contador por route/status; log JSON con request_id aleatorio y resultado. No incluir cuerpo, cédula, saldo ni etiqueta company_id. No se encontró stack OpenTelemetry/Grafana operativo verificado; propuesta adjunta sin afirmar instrumentación integrada.

Ejercicio 4

Configura un dashboard básico en Grafana/Prometheus que muestre tasa de peticiones, latencia y porcentaje de errores de tu endpoint.

Respuesta del estudiante: Dashboard propuesto RED: rate peticiones/s; histogram_quantile(0.95, rate(bucket[5m])); errores5xx/total en igual ventana. Usar route normalizada, no URL con IDs. Mostrar huecos y n=0 como sin datos. No existe captura de Grafana ni p95 real de esta entrega.

Ejercicio 5

Pide a una IA generativa que instrumente un método que reciba datos de un cliente (nombre, cédula, monto). Audita: ¿registró la cédula del cliente en claro en el Span o en el log? ¿configuró una tasa de muestreo razonable? Documenta el prompt, la respuesta y tu corrección.

Respuesta del estudiante: Prompt: instrumenta operación de cliente sin filtrar PII. Respuesta: no guardar nombre/cédula/monto; usar request_id, duración y código; muestreo inicial de trazas10% como supuesto a evaluar. Auditoría: sampling no anonimiza; métricas son agregadas sin IDs de cliente. El presupuesto cero descarta contratar servicios sin decisión expresa.

Artefacto académico completo — propuesta o ejemplo según respuesta anterior

```
# Propuesta de instrumentación; requiere SDK/proveedor/exportador configurados
import json, logging, time
from opentelemetry import trace, metrics
tracer = trace.get_tracer("horizonte.forecast")
meter = metrics.get_meter("horizonte.forecast")
duration = meter.create_histogram("dashboard.duration", unit="s")
counter = meter.create_counter("dashboard.calls")
def observe_calculation(calculate, request_id):
    started = time.perf_counter()
    outcome = "error"
    try:
        with tracer.start_as_current_span("forecast.dashboard"):
            result = calculate()
            outcome = "ok"
            return result
    finally:
```

```
        elapsed = time.perf_counter() - started
        labels = {"operation":"dashboard", "outcome":outcome}
        duration.record(elapsed, labels)
        counter.add(1, labels)
        logging.getLogger("horizonte").info(json.dumps({
            "event":"dashboard", "request_id":request_id,
            "duration_ms":round(elapsed*1000), "outcome":outcome}))
# No añadir datos financieros, empresa, cédula ni body a logs/labels.
```

Lista de autoevaluación antes de entregar

- Diferencio con claridad monitoreo de observabilidad.
- Instrumenté un endpoint real con Span, métrica y log estructurado.
- Configuré una tasa de muestreo justificada para no degradar el rendimiento.
- Verifiqué que ningún dato personal quede expuesto en mi telemetría.
- Diligencié completamente la FPI-29 con evidencia de mi dashboard funcionando.

Foro de reflexión

Título: *"El costo de saberlo todo: ¿es la observabilidad un overhead insostenible para el hardware humilde de una MiPyme?"*

Redacte una reflexión de 250–300 palabras respondiendo: si una MiPyme opera en producción con un servidor de bajo costo mensual, ¿vale la pena consumir parte de sus recursos de hardware en telemetría detallada para incidentes que quizás ocurran una vez al mes? Debata con bases de ingeniería.

Reflexión del estudiante (272 palabras)
En Horizonte, considero que instrumentar señales mínimas sin PII y sin cardinalidad por empresa o movimiento. Monitoreo detecta condiciones conocidas con indicadores; observabilidad permite investigar el estado interno a partir de señales. Horizonte necesita relacionar HTTP, cola y ejecución del cálculo sin registrar datos financieros de clientes.

El caso concreto que orienta mi análisis es el siguiente: Diseño propuesto: span forecast.dashboard, histograma de duración y contador por route/status; log JSON con request_id aleatorio y resultado. No incluir cuerpo, cédula, saldo ni etiqueta company_id. No se encontró stack OpenTelemetry/Grafana operativo verificado; propuesta adjunta sin afirmar instrumentación integrada.

Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia.

La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados.

Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones. Si falta medición, registro el protocolo y la brecha; no transformo una propuesta en un éxito histórico. Así puedo defender el trabajo y decidir el siguiente incremento con evidencia.

Cuestionario

Seleccione una única respuesta.

1. La función técnica fundamental de un TraceId y un SpanId en OpenTelemetry es:

- A. Autenticar JSON Web Tokens antes de consultas REST.
- B. Correlacionar y agrupar múltiples eventos de logs y operaciones distribuidas bajo una misma solicitud original. ✅
- C. Encriptar la base de datos de producción mediante hashes MD5.
- D. Reducir el peso de las imágenes Docker de producción.

2. El formato JSON estructurado es el estándar recomendado para logs en producción porque:

- A. Hace que el archivo ocupe 90% menos espacio automáticamente.
- B. Permite el parseo automatizado de campos por herramientas de telemetría, facilitando consultas indexadas y alertas. ✅
- C. Oculta automáticamente todas las contraseñas sin programar nada.
- D. Es compatible con compiladores de C++.

3. La observabilidad, a diferencia del monitoreo tradicional, permite:

- A. Solo saber si el sistema está caído o no, sin más detalle.
- B. Deducir el estado interno de un sistema complejo mediante la correlación de logs, métricas y trazas. ✅
- C. Sustituir por completo la necesidad de pruebas automatizadas.
- D. Eliminar la necesidad de un pipeline de CI/CD.

4. Configurar una tasa de muestreo (Sampling Rate) en trazas tiene como propósito:

- A. Aumentar la cantidad de trazas generadas sin límite.
- B. Reducir el overhead de rendimiento sin perder visibilidad estadística del sistema. ✅
- C. Eliminar por completo la necesidad de métricas.
- D. Encriptar automáticamente los logs generados.

5. Registrar la cédula de un cliente directamente en un atributo de un Span de OpenTelemetry es:

- A. Una práctica recomendada para depuración rápida.
- B. Una violación de la Ley 1581 de 2012 sobre protección de datos personales. ✅
- C. Obligatorio para el correcto funcionamiento de Jaeger.
- D. Irrelevante si el sistema es de uso interno.

6. Los tres pilares de la observabilidad (M.E.L.T) son:

- A. Migración, Escalamiento, Latencia y Trazabilidad.

- B. Logs estructurados, Métricas y Trazas distribuidas. ✅
- C. Monitoreo, Encriptación, Logs y Testing.
- D. Modelos, Entidades, Lógica y Transacciones.

7. Según la regla de oro de esta guía, un dato personal identificable (PII):

- A. Puede registrarse en logs si el propósito es depurar un error.
- B. Nunca se registra en claro en un log, métrica o traza, sin importar la utilidad percibida. ✅
- C. Solo debe ofuscarse si el cliente lo solicita explícitamente.
- D. Es aceptable exponerlo si el sistema es de uso interno de la empresa.

Cuestionario resuelto: 1. B; 2. B; 3. B; 4. B; 5. B; 6. B; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-29

Telemetría y Observabilidad del Proyecto Integrador

Esta ficha continúa la FPI-28: instrumenta el sistema asegurado para que "cuente su propia historia" en producción.

A. Identificación de la ficha

Código de la ficha	FPI-29
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	Operación y observabilidad: OpenTelemetry y Grafana

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Recuperar y validar la FPI-28

Endpoint crítico y estado de seguridad (recuperados de fichas anteriores)

Indica qué endpoint se instrumentará y confirma que el pipeline de seguridad sigue vigente.

Respuesta: GET dashboard; CI funcional vigente en archivo, gate seguridad propuesto no integrado.

¿Algo cambió desde la ficha anterior?

Qué cambió	Por qué cambió	Impacto en el proyecto
Estado del producto	Las16 guías anteriores eran un corte antiguo	Se usa matriz vigente y se conserva historial
Tipo de beneficiario	SaaS multiempresa según estudiante	No se simula una empresa aliada única

Si nada cambió, escribe "Sin cambios" en la primera fila y deja la tabla así.

C. Aplicación específica — Observabilidad

1. Enlace al código instrumentado

Ruta del controlador instrumentado en GitHub.

Respuesta: backend/apps/forecast/views.py; no se integró OpenTelemetry desde esta tarea documental.

2. Captura del dashboard en Grafana/Prometheus

Adjunta o describe el dashboard con tasa de peticiones, latencia p95 y porcentaje de errores.

Respuesta: Dashboard propuesto RED: rate peticiones/s; histogram_quantile(0.95, rate(bucket[5m])); errores5xx/total en igual ventana. Usar route normalizada, no URL con IDs. Mostrar huecos y n=0 como sin datos. No existe captura de Grafana ni p95 real de esta entrega.

Registro de Decisión Arquitectónica — ADR N.° 13

Título	ADR-13 Observabilidad
Estado	Aceptado como registro académico; implementación y brechas descritas por separado
Contexto	SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar. Observabilidad
Decisión	Instrumentar señales mínimas sin PII y sin cardinalidad por empresa o movimiento.
Consecuencias (positivas y negativas)	Menor complejidad y límites explícitos. Requiere medición, aceptación y revisión externa para completar evidencia.
Política de rotación y retención de logs	Instrumentar señales mínimas sin PII y sin cardinalidad por empresa o movimiento.
Impacto de costo (almacenamiento)	Instrumentar señales mínimas sin PII y sin cardinalidad por empresa o movimiento.

D. Reflexión y trazabilidad

¿Qué aprendimos?

Resume lo aprendido sobre observabilidad y los pilares M.E.L.T.

Respuesta: Monitoreo detecta condiciones conocidas con indicadores; observabilidad permite investigar el estado interno a partir de señales. Horizonte necesita relacionar HTTP, cola y ejecución del cálculo sin registrar datos financieros de clientes.

¿Cómo se aplica al proyecto?

Explica qué visibilidad nueva tiene ahora el equipo sobre el comportamiento del sistema.

Respuesta: El diseño aporta correlación y medidas propuestas. El SaaS ya registra jobs/auditorías; no se inventa stack Grafana operativo.

Decisiones tomadas

Decisión	Justificación
Instrumentar señales mínimas sin PII y sin cardinalidad por empresa o movimiento.	Alineación con riesgo financiero y capacidad individual
Separar evidencia y propuesta	Evitar certificar despliegues, revisiones o mediciones no realizados

Evidencia generada

Estado documentado: Código de instrumentación en el repositorio. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Dashboard funcional en Grafana. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: ADR N.° 13 de retención de logs registrado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Dificultades o ajustes identificados

¿Qué señal de telemetría fue más difícil de interpretar?

Respuesta: Un p95 sin tamaño de muestra, ventana ni errores puede inducir conclusiones equivocadas.

Próximo paso

¿Qué debe hacer el equipo antes de la siguiente guía?

Respuesta: Acordar SLI y preparar runbook verificable con un único operador.

Conexión con la siguiente guía

Las señales de telemetría capturadas aquí son la base cuantitativa de la siguiente guía, donde definirás formalmente indicadores y objetivos de confiabilidad (SLI/SLO) bajo el paradigma SRE.

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5
Comprensión conceptual (15%)	Confunde monitoreo con observabilidad	Comprende conceptos básicos	Comprende lo esencial	Aplica correctamente los pilares M.E.L.T	Explica y relaciona con profundidad
Calidad de la instrumentación (25%)	No instrumenta o no ejecuta	Instrumentación básica incompleta	Instrumentación funcional de los tres pilares	Instrumentación robusta y bien documentada	Instrumentación de nivel profesional

Calidad del dashboard (20%)	No hay dashboard funcional	Dashboard básico sin alertas	Dashboard funcional con métricas clave	Dashboard completo y bien diseñado	Dashboard ejemplar con umbrales de alerta
Rigor de la auditoría a la IA (15%)	No audita o copia sin procesar	Críticas superficiales	Detecta fugas de PII básicas	Auditoría robusta con corrección concreta	Auditoría excepcional y bien argumentada
Participación en el foro (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada
Trazabilidad respecto a fichas anteriores (15%)	No recupera lo construido antes	Copia sin verificar vigencia	Valida lo esencial	Valida y actualiza correctamente	Trazabilidad ejemplar con causa e impacto de cambios

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%).

Recursos académicos y bibliografía

- CNCF. OpenTelemetry Documentation.
- Grafana Labs. Grafana y Prometheus Documentation.
- Majors, Charity; Fong-Jones, Liz; Miranda, George. Observability Engineering.
- IEEE Computer Society. SWEBOK Guide, Version 4.0 (Software Engineering Operations).

UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 14 resuelta

SRE: Indicadores (SLI), Objetivos (SLO), Presupuestos de Error y Gestión de Incidentes

Asignatura	Ingeniería de Software II
Corte	Tercer Corte
Duración	Semana 14
Proyecto integrador	Fase 30: Definición de SLI/SLO, Runbook de contingencia y simulación de Postmortem del MVP
Herramientas	Grafana Alerts, plantillas de Postmortem y Runbook + IA generativa auditada
Formato de entrega	Documento con taller resuelto y Ficha de Aplicación al Proyecto Integrador (FPI-30) diligenciada
Fecha límite	Domingo de la Semana 14, 11:59 p.m. (ver fecha exacta en el aula virtual)
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar.
Corte documental 30/10/2026. Se contrastaron código, compose.yaml, .github/workflows/ci.yml y docs/IMPLEMENTATION_STATUS.md. El registro del 18/10 informa 235 pruebas PostgreSQL: 233 aprobadas y 2 omitidas; es evidencia histórica, no una nueva ejecución de esta entrega. No se certifica producción actual, ASVS, p95 representativo ni validación con empresarios. Presupuesto inicial 0 USD y respaldo manual por decisión del proyecto.

Fase 30 del Proyecto Integrador — Evolución del MVP

Con la telemetría de la FPI-29 ya generando datos reales, puedes por fin responder con números, y no con intuición, la pregunta: ¿qué tan confiable es tu sistema? El paradigma Site Reliability Engineering (SRE), creado por Google, demuestra que el 100% de confiabilidad es el objetivo incorrecto para casi cualquier sistema: buscar la perfección es prohibitivamente costoso y frena la velocidad de entrega.

Esta guía retoma las métricas capturadas en la FPI-29: las convertirás en un Indicador de Nivel de Servicio (SLI) formal, negociarás un Objetivo (SLO) realista para tu MiPyme, y prepararás al equipo para reaccionar ante fallas reales con un Runbook y un Postmortem sin culpables. El resultado se registra en la FPI-30.

Resultados de aprendizaje

- Definir un Indicador de Nivel de Servicio (SLI) cuantitativo a partir de telemetría real.
- Negociar un Objetivo de Nivel de Servicio (SLO) realista según el contexto de negocio.
- Calcular el Presupuesto de Error (Error Budget) correspondiente a un SLO.
- Redactar un Runbook de contingencia técnico y libre de ambigüedad.
- Aplicar la disciplina de Blameless Postmortem ante un incidente simulado.

Objetivos

General

Definir indicadores y objetivos de confiabilidad cuantitativos para el MVP, y preparar al equipo para responder a incidentes mediante Runbooks y Postmortems sin culpables.

Específicos

- Explicar por qué buscar el 100% de disponibilidad es económicamente ineficiente.
- Calcular el Error Budget correspondiente a un SLO dado, en horas o minutos de tolerancia al mes.
- Redactar un Runbook con pasos de recuperación probados y no alucinados.
- Explicar el propósito del análisis "sin culpa" en un Postmortem.
- Registrar la matriz SLI/SLO, el Runbook y el Postmortem simulado en la FPI-30.

Saberes previos

Antes de iniciar, responde (no se califica):

- ¿Alguna vez un servicio (luz, internet, agua) te falló y te pareció razonable, y otra vez te pareció inaceptable? ¿Qué diferenció ambos casos?

Respuesta del estudiante: Un ejemplo hipotético: importa la duración y si impide cobrar o consultar caja. La expectativa depende del horario y perjuicio, no solo del porcentaje.

- ¿Crees que tu MVP necesita una disponibilidad del 99.9% o del 99.99%? ¿Por qué?

Respuesta del estudiante: No justifico99,99% con presupuesto0USD. Inicio con meta de ensayo99%, sin convertirla en SLA productivo; acordar expectativa con usuarios.

- ¿Alguna vez presenciaste que, tras un error, alguien fue "culpado" en lugar de analizar la causa raíz del proceso?

Respuesta del estudiante: No invento una experiencia personal. Un postmortem de Horizonte debe examinar proceso, guardas y recuperación, no culpar al único estudiante.

Concepto 1 — SLI, SLO y Error Budget

Concepto	Definición
SLI (Service Level Indicator)	Indicador cuantitativo de la calidad del servicio (ej. % de peticiones HTTP exitosas respondidas en menos de 200ms en el mes).

SLO (Service Level Objective)	El objetivo numérico de disponibilidad definido para el SLI, acordado con el negocio (ej. cumplirse el 99.5% del tiempo en 30 días).
Error Budget	El espacio de fallo permitido por el SLO (ej. un SLO de 99.5% permite un presupuesto de error del 0.5%). Si se agota, el equipo debe detener nuevas características y dedicarse a estabilizar.

> *Analogía territorial: imagina el suministro de energía de la CENS en el Área Metropolitana de Cúcuta. El SLI es la medida de voltaje constante de 110V en el tomacorriente. El SLO acordado con Villa del Rosario es que el voltaje se mantenga estable el 99.0% del mes. El Error Budget es el 1.0% de tiempo permitido para cortes (aproximadamente 7.2 horas al mes). Si la CENS excede ese presupuesto por fallas imprevistas, la alcaldía le bloquea la licencia para expandir la red a nuevos barrios, obligándola a reparar primero los transformadores averiados.*

Concepto 2 — El Error Budget como mecanismo de control automático

El Error Budget no es un simple indicador decorativo: funciona como balanceador automático de calidad. Si se agota debido a incidentes operacionales, el equipo debe detener de forma innegociable los despliegues de nuevas características y dedicar el 100% de sus esfuerzos a la estabilización de la infraestructura.

Concepto 3 — Runbooks de contingencia

Un Runbook es un manual de contingencia técnico, detallado y libre de ambigüedad para operadores del sistema ante una alerta específica (ej. "base de datos no responde"). Debe incluir pasos verificados y probados, no comandos alucinados o mágicos, y debe adaptarse al organigrama real del equipo (que en una MiPyme puede ser un único desarrollador, no un equipo de diez especialistas).

Concepto 4 — Blameless Postmortem

Práctica de SRE para el análisis forense de incidentes. En lugar de culpar al programador por un error de digitación, se asume que las personas intentaron hacer lo correcto con la información disponible, y se identifican las fallas de los procesos, automatizaciones y mecanismos de defensa sistémicos que permitieron la propagación del error.

Regla de oro de esta guía: *"El SLO se dicta por la experiencia del usuario y el ROI económico de la MiPyme, no por el capricho del programador de querer sistemas indestructibles."*

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, la función del Error Budget en un equipo SRE.

Respuesta del estudiante: Error Budget cuantifica fallos permitidos por el SLO en una ventana. Si se agota, se prioriza confiabilidad sobre entregas riesgosas; no es permiso para ignorar clientes. Se define por solicitudes o tiempo, con exclusiones explícitas y observabilidad suficiente.

Ejercicio 2

Calcula el Error Budget (en horas al mes) que resulta de un SLO del 99.5%, y compáralo con el de un SLO del 99.95%.

Respuesta del estudiante: Mes convencional de30d =720h. SLO99,5%:720×0,005=3,6h (216min). SLO99,95%:720×0,0005=0,36h (21,6min). No mezclar meses distintos. Para99,9% serían43,2min;99,99%4,32min.

Nivel Intermedio (Runbook + Postmortem real + auditoría a la IA)

Ejercicio 3

Redacta un Runbook real para tu MVP ante una alerta crítica plausible (ej. "base de datos no responde"), adaptado a que el único responsable disponible eres tú o tu equipo.

Respuesta del estudiante: Runbook individual: confirmar alcance en /api/health/ y logs; ejecutar docker compose ps y docker compose logs --tail100 backend postgres; verificar pg_isready en postgres; si servicio detenido, docker compose up -d postgres backend worker. No borrar volumen ni usar down -v. Comprobar login/consulta sintética e idempotencia; escalar al proveedor si es alojamiento externo. Registrar inicio, mitigación y evidencia sin PII.

Ejercicio 4

Simula un incidente real o hipotético de tu proyecto (ej. una caída de 30 minutos) y redacta un Postmortem sin culpables, identificando la causa raíz de proceso, no de persona.

Respuesta del estudiante: Postmortem hipotético: caída30min por reinicio sin verificar readiness BD. Línea de tiempo:0min alerta,5 diagnóstico,15 arranque BD,25 app lista,30 validación. Causa de proceso: despliegue sin prueba de recuperación y dependencia lista. Acciones: healthcheck, ensayo aislado y runbook. No se afirma incidente ocurrido ni se culpa a Adrian.

Ejercicio 5

Pide a una IA generativa que redacte tu Runbook. Audita: ¿los comandos propuestos existen realmente y funcionan en tu entorno? ¿asumió un equipo de soporte irreal para una MiPyme? ¿propuso herramientas propietarias de pago innecesarias? Documenta el prompt, la respuesta y tu corrección.

Respuesta del estudiante: Prompt: audita runbook para único operador Windows con Docker. Respuesta: usar servicios del compose real, sin equipo24/7 ni herramientas pagas. Auditoría: comandos son procedimientos, no se ejecutaron reinicios hoy; confirmar motor activo y no usar nombres supuestos. No prometer recuperación sin backup probado.

Lista de autoevaluación antes de entregar

- Definí un SLI cuantitativo real, basado en telemetría de la FPI-29.
- Calculé correctamente el Error Budget correspondiente a mi SLO.
- Mi Runbook tiene comandos verificados, no alucinados por la IA.
- Mi Postmortem identifica fallas de proceso, no culpa a una persona.
- Diligencié completamente la FPI-30 con mi matriz SLI/SLO, Runbook y Postmortem.

Foro de reflexión

Título: *"La tiranía del 99.99%: el costo financiero del sobrediseño de disponibilidad"*

Redacte una reflexión de 250–300 palabras respondiendo: un SLO del 99.9% permite 43 minutos de caída al mes; uno del 99.99% permite solo 4 minutos, requiriendo infraestructura redundante costosa. ¿Cómo demuestra un ingeniero que el SLO debe dictarse por la experiencia del usuario y el ROI, y no por el capricho de tener sistemas indestructibles?

Reflexión del estudiante (272 palabras)
En Horizonte, considero que definir SLO de ensayo, runbook individual y postmortem sin culpables; no prometer SLA gratuito. Error Budget cuantifica fallos permitidos por el SLO en una ventana. Si se agota, se prioriza confiabilidad sobre entregas riesgosas; no es permiso para ignorar clientes. Se define por solicitudes o tiempo, con exclusiones explícitas y observabilidad suficiente. El caso concreto que orienta mi análisis es el siguiente: Runbook individual: confirmar alcance en /api/health/ y logs; ejecutar docker compose ps y docker compose logs --tail100 backend postgres; verificar pg_isready en postgres; si servicio detenido, docker compose up -d postgres backend worker. No borrar volumen ni usar down -v. Comprobar login/consulta sintética e idempotencia; escalar al proveedor si es alojamiento externo. Registrar inicio, mitigación y evidencia sin PII. Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia. La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados. Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones.

Cuestionario

Seleccione una única respuesta.

1. La función del Error Budget en equipos SRE modernos es:

- A. Asignar el presupuesto financiero para comprar hardware en la nube.
- B. Servir como balanceador automático de calidad: si se agota por incidentes, bloquea nuevas características hasta restaurar la estabilidad. ✅
- C. Despedir a los desarrolladores que cometan fallos de código.
- D. Pagar multas legales al cliente por incumplimiento del software.

2. La característica central de un Blameless Postmortem es:

- A. Encontrar rápidamente al culpable para deshabilitar sus accesos.
- B. Identificar deficiencias en las salvaguardas, procesos y arquitectura que permitieron que el error se propagara. ✅
- C. Omitir las fallas técnicas complejas de los informes públicos.
- D. Convencer al cliente de que el sistema nunca se cayó.

3. Un SLO del 99.5% mensual corresponde a un Error Budget de aproximadamente:

- A. Cero minutos de tolerancia.
- B. Alrededor de 3.6 horas de indisponibilidad tolerada al mes. ✅

- C. Todo el mes de indisponibilidad permitida.
- D. Un Error Budget no se puede calcular a partir de un SLO.

4. Un Runbook, según esta guía, debe:

- A. Asumir un equipo de soporte de diez especialistas sin importar el tamaño real de la organización.
- B. Contener pasos técnicos verificados y adaptados al organigrama real y limitado del equipo. ✅
- C. Incluir únicamente comandos genéricos sin probar.
- D. Evitar cualquier mención a Docker o bases de datos.

5. Un SLI (Service Level Indicator) se define como:

- A. Un objetivo negociado con el cliente sobre el comportamiento futuro.
- B. Un indicador cuantitativo de la calidad de servicio brindada realmente. ✅
- C. El presupuesto financiero total del proyecto.
- D. Un tipo de prueba de rendimiento con k6.

6. Exigir un SLO del 99.999% para un sistema de facturación de una MiPyme pequeña, sin justificación de negocio real, es un ejemplo de:

- A. Buena práctica obligatoria en cualquier sistema.
- B. Sobrediseño de disponibilidad que ignora el ROI económico real de la organización. ✅
- C. Un requisito legal bajo la Ley 1581 de 2012.
- D. La única forma correcta de aplicar SRE.

7. Según la regla de oro de esta guía, el SLO debe dictarse por:

- A. El capricho del programador de tener sistemas indestructibles.
- B. La experiencia del usuario y el ROI económico real de la MiPyme. ✅
- C. La moda tecnológica del momento.
- D. El número de servidores disponibles sin ningún otro criterio.

Cuestionario resuelto: 1. B; 2. B; 3. B; 4. B; 5. B; 6. B; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-30

Confiabilidad SRE del Proyecto Integrador

Esta ficha continúa la FPI-29: convierte la telemetría capturada en indicadores formales de confiabilidad y prepara al equipo ante incidentes.

A. Identificación de la ficha

Código de la ficha	FPI-30
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	SRE: SLI, SLO, Error Budgets, Runbooks y Postmortems

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Recuperar y validar la FPI-29

Telemetría disponible (recuperada de la FPI-29)

Resume qué métricas reales usarás para definir el SLI de esta ficha. Respuesta: SLI propuesto: consultas válidas de dashboard con respuesta correcta / total de consultas válidas en ensayo local. Errores tenant esperados se separan de5xx; no hay serie actual para computarlo.

¿Algo cambió desde la ficha anterior?

Qué cambió	Por qué cambió	Impacto en el proyecto
Estado del producto	Las16 guías anteriores eran un corte antiguo	Se usa matriz vigente y se conserva historial
Tipo de beneficiario	SaaS multiempresa según estudiante	No se simula una empresa aliada única

Si nada cambió, escribe "Sin cambios" en la primera fila y deja la tabla así.

C. Aplicación específica — SRE

1. Matriz de Niveles de Servicio

SLI definido	SLO acordado (%)	Error Budget permitido
Éxito de consultas válidas en ensayo local	99% meta académica, no SLA acordado	7,2h por30d si mide tiempo; solicitudes fallidas≤1% si mide peticiones

Latencia útil dashboard	95% por debajo2s meta	Hasta5% puede superar2s; registrar fallos aparte

2. Enlace al Runbook de contingencia

Ruta del archivo Runbook_[componente].md

Respuesta: Runbook completo incluido en Ejercicio3 de esta guía; no existe archivo Runbook_[componente].md creado en el repositorio por esta entrega.

3. Reporte Postmortem forense simulado

Causa raíz identificada y acciones preventivas inmediatas.

Respuesta: Postmortem hipotético: caída30min por reinicio sin verificar readiness BD. Línea de tiempo:0min alerta,5 diagnóstico,15 arranque BD,25 app lista,30 validación. Causa de proceso: despliegue sin prueba de recuperación y dependencia lista. Acciones: healthcheck, ensayo aislado y runbook. No se afirma incidente ocurrido ni se culpa a Adrian.

Registro de Decisión Arquitectónica — ADR N.° 14

Título	ADR-14 SRE e incidentes
Estado	Aceptado como registro académico; implementación y brechas descritas por separado
Contexto	SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar. SRE e incidentes
Decisión	Definir SLO de ensayo, runbook individual y postmortem sin culpables; no prometer SLA gratuito.
Consecuencias (positivas y negativas)	Menor complejidad y límites explícitos. Requiere medición, aceptación y revisión externa para completar evidencia.

D. Reflexión y trazabilidad

¿Qué aprendimos?

Resume lo aprendido sobre SLI, SLO, Error Budget y Postmortems.

Respuesta: Error Budget cuantifica fallos permitidos por el SLO en una ventana. Si se agota, se prioriza confiabilidad sobre entregas riesgosas; no es permiso para ignorar clientes. Se define por solicitudes o tiempo, con exclusiones explícitas y observabilidad suficiente.

¿Cómo se aplica al proyecto?

Explica cómo el equipo reaccionará ahora ante un incidente real.

Respuesta: Runbook individual: confirmar alcance en /api/health/ y logs; ejecutar docker compose ps y docker compose logs --tail100 backend postgres; verificar pg_isready en postgres; si servicio detenido, docker compose up -d postgres backend worker. No borrar volumen ni usar down -v. Comprobar login/consulta sintética e idempotencia; escalar al proveedor si es alojamiento externo. Registrar inicio, mitigación y evidencia sin PII.

Decisiones tomadas

Decisión	Justificación
Definir SLO de ensayo, runbook individual y postmortem sin culpables; no prometer SLA gratuito.	Alineación con riesgo financiero y capacidad individual
Separar evidencia y propuesta	Evitar certificar despliegues, revisiones o mediciones no realizados

Evidencia generada

Estado documentado: Matriz SLI/SLO/Error Budget diligenciada. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Runbook de contingencia documentado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Postmortem simulado sin culpables. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: ADR N.° 14 registrado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Dificultades o ajustes identificados

¿Qué parte del Runbook fue más difícil de verificar?

Respuesta: Verificar restauración sin tocar volumen real y evitar reinicios destructivos.

Próximo paso

¿Qué debe hacer el equipo antes de la siguiente guía?

Respuesta: Actualizar inventario residual y priorizar por impacto esperado.

Conexión con la siguiente guía

La confiabilidad medida aquí revela, casi siempre, deuda técnica residual. La siguiente guía retoma el inventario de deuda técnica de la FPI-17 para actualizarlo y diseñar el roadmap de evolución del MVP hacia el cierre del semestre.

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5
Comprensión conceptual (15%)	Confunde SLI con SLO	Comprende conceptos básicos	Comprende lo esencial	Calcula correctamente el Error Budget	Explica y relaciona con profundidad
Rigor de la matriz SLI/SLO (20%)	No define indicadores reales	Definición superficial sin datos reales	Definición aceptable con datos básicos	Definición sólida trazable a telemetría real	Definición ejemplar y bien justificada
Calidad del Runbook (25%)	No hay Runbook o es genérico	Runbook básico sin verificar	Runbook funcional con comandos probados	Runbook robusto y adaptado al equipo real	Runbook de nivel profesional
Calidad y honestidad del Postmortem (15%)	Postmortem culpa a una persona	Postmortem superficial	Postmortem aceptable sin culpables	Postmortem riguroso, identifica causa raíz de proceso	Postmortem ejemplar y de nivel profesional
Participación en el foro (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada
Trazabilidad respecto a la FPI-29 (15%)	No recupera lo construido antes	Copia sin verificar vigencia	Valida lo esencial	Valida y actualiza correctamente	Trazabilidad ejemplar con causa e impacto de cambios

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%).

Recursos académicos y bibliografía

- Beyer, Betsy; Jones, Chris; Petoff, Jennifer; Murphy, Niall Richard. Site Reliability Engineering (Google).
- Google SRE Workbook.
- IEEE Computer Society. SWEBOK Guide, Version 4.0 (Software Engineering Operations).

UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 15 resuelta

Mantenimiento y Evolución de Software: Gestión Proactiva de la Deuda Técnica Residual y Roadmap

Asignatura	Ingeniería de Software II
Corte	Tercer Corte
Duración	Semana 15
Proyecto integrador	Fase 31: Actualización del inventario de deuda técnica y diseño del roadmap de evolución del MVP
Herramientas	SonarQube para métricas de deuda, GitHub Projects Backlog + IA generativa auditada
Formato de entrega	Documento con taller resuelto y Ficha de Aplicación al Proyecto Integrador (FPI-31) diligenciada
Fecha límite	Domingo de la Semana 15, 11:59 p.m. (ver fecha exacta en el aula virtual)
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar.
Corte documental 30/10/2026. Se contrastaron código, compose.yaml, .github/workflows/ci.yml y docs/IMPLEMENTATION_STATUS.md. El registro del 18/10 informa 235 pruebas PostgreSQL: 233 aprobadas y 2 omitidas; es evidencia histórica, no una nueva ejecución de esta entrega. No se certifica producción actual, ASVS, p95 representativo ni validación con empresarios. Presupuesto inicial 0 USD y respaldo manual por decisión del proyecto.

Fase 31 del Proyecto Integrador — Evolución del MVP

El software real casi nunca se construye desde cero: en el 90% de su carrera, un ingeniero se enfrentará a código preexistente con deudas técnicas acumuladas. Tras 14 guías evolucionando tu MVP, es momento de cerrar el círculo abierto en la FPI-17: revisar qué deuda técnica original se pagó, cuál persiste, y diseñar honestamente el roadmap de evolución que continuará después de este semestre.

Esta guía retoma directamente el inventario de deuda técnica de la FPI-17: lo actualizarás con la clasificación de los cuatro tipos de mantenimiento (correctivo, adaptativo, perfectivo, preventivo) y priorizarás lo que queda pendiente en un Roadmap Técnico. El resultado se registra en la FPI-31.

Resultados de aprendizaje

- Clasificar el mantenimiento de software en correctivo, adaptativo, perfectivo y preventivo.
- Actualizar el inventario de deuda técnica original con la evolución real del semestre.
- Priorizar deuda técnica pendiente aplicando el cuadrante de Fowler y criterios de negocio.
- Diseñar un Roadmap Técnico de evolución a 6 meses para la MiPyme.
- Auditar críticamente un plan de refactorización de código legacy sugerido por IA.

Objetivos

General

Actualizar el diagnóstico de deuda técnica del MVP tras un semestre de evolución y diseñar un Roadmap Técnico viable para la MiPyme aliada.

Específicos

- Diferenciar los cuatro tipos de mantenimiento con ejemplos reales del proyecto.
- Comparar el inventario de deuda técnica de la FPI-17 contra el estado actual del sistema.
- Refactorizar al menos un módulo con deuda técnica pendiente, verificando equivalencia funcional.
- Priorizar y planificar tres hitos de evolución técnica a 6 meses.
- Registrar el inventario actualizado y el Roadmap en la FPI-31.

Saberes previos

Antes de iniciar, responde (no se califica):

- Revisa tu FPI-17: ¿cuánta de esa deuda técnica original sigue exactamente igual hoy?

Respuesta del estudiante: En este conjunto se mantiene el mismo corte: DT01, DT02 y DT03 no se declaran pagadas por escribir las respuestas.

- ¿Crees que un sistema completamente libre de deuda técnica es realista para una MiPyme?

Respuesta del estudiante: No es realista eliminar toda deuda. Registro costo y riesgo y pago primero integridad, aislamiento y recuperación.

- ¿Qué evolución te gustaría que tuviera este MVP dentro de 6 meses, si el proyecto continuara?

Respuesta del estudiante: En seis meses propondría validación con usuarios, corpus independiente y calibración predictiva, sin anteponer móvil a la web actual.

Concepto 1 — Taxonomía del mantenimiento de software

Tipo	Descripción
Correctivo	Reactivo ante fallos reportados en producción (ej. corregir un bug de redondeo en la facturación).
Adaptativo	Modificación para que el sistema funcione en un nuevo entorno operativo o tecnológico (ej. actualizar la API para una nueva versión de base de datos).

Perfectivo	Incorporar nuevas funcionalidades solicitadas por el negocio para añadir valor (ej. un módulo de descuentos para clientes recurrentes).
Preventivo	Reingeniería y refactorización orientada a evitar fallos futuros y mejorar la mantenibilidad (ej. limpiar deudas lógicas, desacoplar dependencias).

Concepto 2 — La deuda técnica genera interés compuesto

Como recordarás de la FPI-17, la deuda técnica genera interés compuesto: cada modificación sobre código sucio requiere más esfuerzo y horas de desarrollo en el futuro. Revisar honestamente si tu equipo pagó esa deuda o la dejó crecer durante el semestre es un ejercicio de madurez profesional, no una autocrítica destructiva.

> *Analogía territorial: en una ladrillera de Villa del Rosario, un mantenimiento adaptativo es ajustar la boca del quemador cuando cambia el tipo de carbón disponible. Un mantenimiento preventivo es detener el horno un fin de semana para reforzar la estructura de ladrillos refractarios antes de que aparezcan grietas térmicas destructivas. Postergar la limpieza de las cenizas para producir más rápido hoy es incurrir en una Deuda de Infraestructura cuyo interés se paga caro si el horno colapsa por fatiga de materiales.*

Concepto 3 — Priorización pragmática: ¿pagar toda la deuda o convivir con ella?

Roger Pressman afirma que un sistema completamente libre de deuda es económicamente ineficiente para la mayoría de empresas pequeñas. Si un bug de rendimiento ocurre solo una vez cada mil transacciones y corregirlo cuesta 20 horas de ingeniería, no siempre se justifica pagar esa deuda de inmediato: se prioriza según el cuadrante de Fowler (Guía 1) y el impacto real en el negocio, no por perfeccionismo técnico.

Concepto 4 — El Roadmap Técnico como compromiso honesto

Un Roadmap Técnico no es una lista de deseos: debe detallar hitos concretos, con tiempos realistas, que reflejen tanto necesidades de mantenimiento adaptativo (migrar de infraestructura) como perfectivo (nuevas funcionalidades) y preventivo (refactorizaciones pendientes).

Regla de oro de esta guía: *"Un Roadmap técnico sin fechas ni justificación de negocio es solo una lista de buenas intenciones; cada hito debe tener un disparador claro que justifique cuándo abordarlo."*

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, la diferencia entre mantenimiento correctivo y preventivo.

Respuesta del estudiante: Correctivo elimina un defecto observado; preventivo reduce probabilidad o costo de fallos futuros. Corregir duplicación de ingreso es correctivo; separar cálculo y añadir pruebas antes de nuevas reglas es preventivo. Ambos deben conservar integridad de datos.

Ejercicio 2

Para un caso ficticio, da un ejemplo de cada uno de los cuatro tipos de mantenimiento aplicado a un sistema de facturación.

Respuesta del estudiante: Facturación ficticia: correctivo, arreglar total redondeado; adaptativo, nuevo formato tributario; perfectivo, mejorar consulta por cliente; preventivo, extraer reglas y automatizar regresión. El cumplimiento tributario debe verificarse con norma aplicable antes de cambiar cálculo.

Nivel Intermedio (refactorización real + roadmap + auditoría a la IA)

Ejercicio 3

Compara tu inventario de deuda técnica de la FPI-17 contra el estado actual: ¿qué ítems se pagaron, cuáles siguen pendientes y cuáles empeoraron?

Respuesta del estudiante: Inventario del mismo corte: DT01 endurecimiento Docker sigue pendiente; DT02 p95/aceptación sigue pendiente; DT03 observabilidad/SLO sigue pendiente. No hubo integración al backend al responder guías; por tanto no declaro deuda pagada por haberla documentado. Falta criterio de aceptación de cada pago.

Ejercicio 4

Refactoriza al menos un ítem de deuda técnica pendiente de tu proyecto real, verificando con pruebas que el comportamiento no cambió.

Respuesta del estudiante: Se preparó variante aislada de project_obligations con guard clause; conserva Decimal, corte exclusivo y horizonte permitido. Se ejecutaron pruebas de equivalencia en artefacto académico. No se sustituyó el servicio del SaaS ni hay PR de integración; el siguiente cambio requerirá suite completa y revisión. Esto distingue refactor demostrado y deuda productiva pendiente.

Ejercicio 5

Pide a una IA generativa que te ayude con la ingeniería inversa de un fragmento legacy de tu sistema. Audita: ¿la propuesta de refactorización cambió la lógica de negocio real? ¿mantuvo la integridad de transacciones (commit/rollback)? ¿sugirió librerías con licencias restrictivas para medir la deuda? Documenta el prompt, la respuesta y tu corrección.

Respuesta del estudiante: Prompt: revisa equivalencia e integridad de project_obligations. Respuesta: la función pura no hace commit/rollback; las vistas y servicios de persistencia conservan transacciones fuera. Auditoría: no introducir escrituras en cálculo, no incluir vencidos automáticamente ni cambiar signo; solo biblioteca estándar. No se requiere herramienta propietaria para medir esta deuda.

Lista de autoevaluación antes de entregar

- Comparé honestamente mi deuda técnica de la FPI-17 contra el estado actual del sistema.
- Prioricé la deuda pendiente con el cuadrante de Fowler, no por perfeccionismo.
- Refactoricé al menos un ítem real, verificando equivalencia funcional con pruebas.
- Mi Roadmap tiene hitos con fechas y justificación de negocio, no es una lista de deseos.
- Diligencié completamente la FPI-31 con el inventario actualizado y el Roadmap.

Foro de reflexión

Título: *"El cuadrante pragmático de Fowler: ¿debemos pagar toda la deuda técnica o es sano convivir con ella?"*

Redacte una reflexión de 250–300 palabras respondiendo: si un bug de rendimiento ocurre solo una vez cada mil transacciones y corregirlo cuesta 20 horas de ingeniería cara, ¿se justifica pagar esa deuda ahora? Argumente su postura basándose en la velocidad de entrega del negocio y en su propia experiencia del semestre.

Reflexión del estudiante (281 palabras)
En Horizonte, considero que priorizar deuda por pérdida esperada y conservar equivalencia mediante pruebas antes de integrar cambios. Correctivo elimina un defecto observado; preventivo reduce probabilidad o costo de fallos futuros. Corregir duplicación de ingreso es correctivo; separar cálculo y añadir pruebas antes de nuevas reglas es preventivo. Ambos deben conservar integridad de datos.

El caso concreto que orienta mi análisis es el siguiente: Inventario del mismo corte: DT01 endurecimiento Docker sigue pendiente; DT02 p95/aceptación sigue pendiente; DT03 observabilidad/SLO sigue pendiente. No hubo integración al backend al responder guías; por tanto no declaro deuda pagada por haberla documentado. Falta criterio de aceptación de cada pago.

Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia.

La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados.

Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones. Si falta medición, registro el protocolo y la brecha; no transformo una propuesta en un éxito histórico. Así puedo defender el trabajo y decidir el siguiente incremento con evidencia.

Cuestionario

Seleccione una única respuesta.

1. La diferencia fundamental entre mantenimiento correctivo y preventivo es:

- A. El correctivo adapta el sistema a nuevos dispositivos; el preventivo repara fallos ya reportados.
- B. El correctivo reacciona ante errores operativos en producción; el preventivo aplica refactorización proactiva para evitar fallos futuros. ✅
- C. El correctivo reescribe la base de datos de producción; el preventivo elimina servidores de desarrollo.
- D. Ambos se ejecutan solo al final de la vida útil del sistema.

2. La metáfora de la deuda técnica de Ward Cunningham describe:

- A. El dinero que se debe a los proveedores de nube.
- B. El costo diferido en horas de desarrollo futuro provocado por decisiones apresuradas a corto plazo. ✅
- C. El salario que se debe a los ingenieros al final del proyecto.
- D. El número de líneas de código permitidas por archivo.

3. Según Pressman, un sistema completamente libre de deuda técnica:

- A. Es siempre la meta correcta y alcanzable para cualquier empresa.
- B. Puede ser económicamente ineficiente para la mayoría de empresas pequeñas. ✅
- C. Es un requisito obligatorio de ISO/IEC/IEEE 12207.
- D. Solo aplica a sistemas de gran escala empresarial.

4. Un mantenimiento adaptativo se ejemplifica con:

- A. Agregar un nuevo módulo de descuentos solicitado por el negocio.
- B. Actualizar el sistema para que funcione con una nueva versión de base de datos o sistema operativo. ✅
- C. Corregir un bug reportado por un usuario.
- D. Refactorizar código sin cambiar su comportamiento externo.

5. Un Roadmap Técnico sin fechas ni justificación de negocio se considera:

- A. Un documento profesional y completo.
- B. Solo una lista de buenas intenciones, sin valor de planificación real. ✅
- C. El formato obligatorio exigido por SWEBOK v4.
- D. Suficiente para guiar la evolución de un sistema.

6. Priorizar el pago de deuda técnica según el cuadrante de Fowler implica:

- A. Pagar siempre toda la deuda sin importar el impacto real.
- B. Evaluar el impacto en el negocio y el tipo de deuda (deliberada/inadvertida, prudente/imprudente) antes de decidir qué pagar primero. ✅
- C. Ignorar por completo cualquier deuda técnica identificada.
- D. Delegar la decisión únicamente a la IA generativa.

7. Según la regla de oro de esta guía, cada hito de un Roadmap técnico debe tener:

- A. Solo un nombre atractivo, sin más detalle.
- B. Un disparador claro que justifique cuándo abordarlo, con fechas y justificación de negocio. ✅
- C. La aprobación exclusiva de la IA generativa.
- D. Ningún vínculo con la deuda técnica identificada previamente.

Cuestionario resuelto: 1. B; 2. B; 3. B; 4. B; 5. B; 6. B; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-31

Deuda Técnica Residual y Roadmap del Proyecto Integrador

Esta ficha cierra el ciclo abierto en la FPI-17: actualiza el inventario de deuda técnica original y proyecta la evolución futura del MVP.

A. Identificación de la ficha

Código de la ficha	FPI-31
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	Mantenimiento y evolución: deuda técnica residual y roadmap

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Recuperar y validar la FPI-17

Inventario original de deuda técnica (recuperado de la FPI-17)

Copia aquí los ítems de deuda técnica identificados al inicio del semestre. Respuesta: DT01 Docker no root/multietapa; DT02 carga y aceptación; DT03 observabilidad/SLO, definidos en FPI-17 de este mismo corte.

¿Algo cambió desde la ficha anterior?

Qué cambió	Por qué cambió	Impacto en el proyecto
Estado del producto	Las16 guías anteriores eran un corte antiguo	Se usa matriz vigente y se conserva historial
Tipo de beneficiario	SaaS multiempresa según estudiante	No se simula una empresa aliada única

Si nada cambió, escribe "Sin cambios" en la primera fila y deja la tabla así.

C. Aplicación específica — Deuda residual y Roadmap

1. Inventario actualizado de deuda técnica

Ítem de deuda	Estado (pagada/pendiente/empeoró)	Impacto en calidad	Cuadrante de Fowler	Acción de pago programada

DT01 Contenedor backend	Pendiente en producto; propuesta documentada	Root implícito y servidor desarrollo	Deliberada prudente si se limita y revisa	Sprint propuesto1: build no root y regresión
DT02 Aceptación y carga	Pendiente en producto; propuesta documentada	Sin evidencia integral p95/UX	Deliberada prudente si se limita y revisa	Sprint propuesto2: protocolo reproducible
DT03 Observabilidad/SLO	Pendiente en producto; propuesta documentada	Diagnóstico y disponibilidad no acreditados	Deliberada prudente si se limita y revisa	Sprint propuesto3: señales mínimas y ensayo

2. Evidencia del Pull Request de refactorización preventiva

Enlace al PR con el refactoring aplicado en el Ejercicio 4.

Respuesta: Sin PR integrado: refactor puro académico y pruebas en evidencia_academica.json. No se cierra deuda del backend por documentar una propuesta.

3. Roadmap de evolución técnica (próximos 6 meses)

Hito	Descripción	Justificación de negocio
Sprint1 propuesto	Docker no root y permisos	Reducir riesgo operacional
Sprint2 propuesto	Carga y aceptación web	Validar utilidad antes de publicar
Sprint3 propuesto	SLO y runbook de ensayo	Operación viable individual

Registro de Decisión Arquitectónica — ADR N.° 15

Título	ADR-15 Mantenimiento
Estado	Aceptado como registro académico; implementación y brechas descritas por separado
Contexto	SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar. Mantenimiento
Decisión	Priorizar deuda por pérdida esperada y conservar equivalencia mediante pruebas antes de integrar cambios.
Consecuencias (positivas y negativas)	Menor complejidad y límites explícitos. Requiere medición, aceptación y revisión externa para completar evidencia.

D. Reflexión y trazabilidad

¿Qué aprendimos?

Resume lo aprendido sobre mantenimiento y gestión de deuda técnica.

Respuesta: Correctivo elimina un defecto observado; preventivo reduce probabilidad o costo de fallos futuros. Corregir duplicación de ingreso es correctivo; separar cálculo y añadir pruebas antes de nuevas reglas es preventivo. Ambos deben conservar integridad de datos.

¿Cómo se aplica al proyecto?

Explica qué tan diferente es la deuda técnica de tu MVP hoy frente a la FPI-17.

Respuesta: Inventario del mismo corte: DT01 endurecimiento Docker sigue pendiente; DT02 p95/aceptación sigue pendiente; DT03 observabilidad/SLO sigue pendiente. No hubo integración al backend al responder guías; por tanto no declaro deuda pagada por haberla documentado. Falta criterio de aceptación de cada pago.

Decisiones tomadas

Decisión	Justificación
Priorizar deuda por pérdida esperada y conservar equivalencia mediante pruebas antes de integrar cambios.	Alineación con riesgo financiero y capacidad individual
Separar evidencia y propuesta	Evitar certificar despliegues, revisiones o mediciones no realizados

Evidencia generada

Estado documentado: Inventario de deuda técnica actualizado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: PR de refactorización preventiva. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Roadmap de evolución a 6 meses. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: ADR N.° 15 registrado. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Dificultades o ajustes identificados

¿Qué deuda fue más difícil de priorizar?

Respuesta: Anteponer precisión y aislamiento a mejoras cosméticas o herramientas costosas.

Próximo paso

¿Qué debe hacer el equipo antes de la guía de cierre?

Respuesta: Preparar dossier y demo distinguiendo verificación y validación.

Conexión con la siguiente guía

El Roadmap y el inventario de deuda actualizados aquí son parte central del Dossier Técnico Final que presentarás en la demostración de cierre del semestre en la Guía 16.

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5
Comprensión conceptual (15%)	Confunde los tipos de mantenimiento	Comprende conceptos básicos	Comprende lo esencial	Clasifica correctamente los cuatro tipos	Explica y relaciona con profundidad
Honestidad del inventario actualizado (25%)	No actualiza o es inconsistente con la FPI-17	Actualización superficial	Actualización aceptable y honesta	Actualización rigurosa y bien priorizada	Inventario ejemplar, con trazabilidad completa
Calidad de la refactorización preventiva (20%)	No refactoriza o rompe el sistema	Refactorización parcial sin verificación	Refactorización aceptable y verificada	Refactorización sólida y bien documentada	Refactorización de nivel profesional
Viabilidad del Roadmap (15%)	Lista de deseos sin fechas ni justificación	Roadmap genérico	Roadmap aceptable con hitos básicos	Roadmap viable y bien justificado	Roadmap ejemplar, realista y priorizado
Participación en el foro (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada
Trazabilidad respecto a la FPI-17 (15%)	No recupera lo construido antes	Copia sin verificar vigencia	Valida lo esencial	Valida y actualiza correctamente	Trazabilidad ejemplar con causa e impacto de cambios

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%).

Recursos académicos y bibliografía

- Pressman, Roger. Ingeniería del Software: Un Enfoque Práctico (Mantenimiento y Evolución).
- Fowler, Martin. TechnicalDebtQuadrant (martinfowler.com).

- IEEE Computer Society. SWEBOK Guide, Version 4.0 (Software Maintenance).

Comprobaciones de esta entrega
30/10/2026:26 comprobaciones del cálculo contractual aprobaron (18 equivalencias y8 rechazos de horizonte), sin BD. La prueba real tests.test_application.ApplicationTests.test_company_isolation también pasó sobre BD temporal. La variante académica no se integró al backend. Evidencias: _trabajo/evidencia_academica.json y prueba_aislamiento.log.

UNIVERSIDAD DE PAMPLONA — SEDE VILLA DEL ROSARIO

Facultad de Ingeniería y Arquitecturas

Programa de Ingeniería de Sistemas

Guía de aprendizaje 16 resuelta

Demostración Final: Validación Sistémica, Defensa de Decisiones y Auditoría Ética de IA

Asignatura	Ingeniería de Software II
Corte	Tercer Corte (Cierre de Semestre y del Programa)
Duración	Semana 16
Proyecto integrador	Fase 32 (final): Demo Day, defensa de ADRs, bitácora ética de IA consolidada y cierre del proyecto integrador de dos semestres
Herramientas	Repositorio GitHub (tags/releases), Dashboards de Grafana, Loom + consolidación de las 31 fichas anteriores
Formato de entrega	Dossier Técnico Final, Video Demo, Defensa de ADRs y Ficha de Aplicación al Proyecto Integrador (FPI-32) — síntesis final del programa
Fecha límite	Domingo de la Semana 16, 11:59 p.m. (ver fecha exacta en el aula virtual)
Estudiante	Adrian Perez Pacheco
Profesor	Miguel Eduardo Posada Haddad
Proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Metodología	Scrum individual; un estudiante
Fecha de resolución	30 de octubre de2026

SaaS multiempresa para Villa del Rosario, Cúcuta y Norte de Santander, sin empresa beneficiaria única. Adrian es estudiante de la Universidad de Pamplona, Sede Villa del Rosario, Facultad de Ingeniería y Arquitecturas, Ingeniería de Sistemas; profesor Miguel Eduardo Posada Haddad. Scrum individual: Adrian asume producto, facilitación y desarrollo; sprints propuestos de dos semanas y revisión externa por acordar.
Corte documental 30/10/2026. Se contrastaron código, compose.yaml, .github/workflows/ci.yml y docs/IMPLEMENTATION_STATUS.md. El registro del 18/10 informa 235 pruebas PostgreSQL: 233 aprobadas y 2 omitidas; es evidencia histórica, no una nueva ejecución de esta entrega. No se certifica producción actual, ASVS, p95 representativo ni validación con empresarios. Presupuesto inicial 0 USD y respaldo manual por decisión del proyecto.

Fase 32 del Proyecto Integrador — Cierre del Programa Completo

Esta guía representa el hito supremo de tu formación en estos dos semestres. Desde la FPI-01, elicitaste un dolor real del tejido productivo regional; hoy defenderás un sistema operable, seguro, contenerizado, observable y desplegado profesionalmente en la nube. No se trata de una demostración superficial de código: defenderás tus decisiones arquitectónicas basándote en tus ADRs (N.° 01 a N.° 15), justificarás el ROI de tus estrategias de pruebas y observabilidad, y certificarás el uso ético, seguro y auditado de la IA a lo largo de todo el programa.

Esta guía cierra el ciclo completo de 32 fichas (FPI-01 a FPI-31): la FPI-32 no abre una fase nueva, consolida todo el programa en un Dossier Técnico Final que certifica tu formación como ingeniero de software.

Resultados de aprendizaje

- Diferenciar la Validación Sistémica (¿resolvimos el problema real?) de la Verificación técnica acumulada durante el programa.
- Defender decisiones arquitectónicas con evidencia de observabilidad real y trade-offs documentados en los ADRs.
- Consolidar una bitácora forense de auditoría ética de IA de todo el programa (dos semestres).
- Presentar una demostración en vivo, navegable y defendible ante un comité evaluador.
- Certificar de forma transparente el cumplimiento de la Ley 1581 de 2012 en todo el ciclo de vida del proyecto.

Objetivos

General

Cerrar formalmente el proyecto integrador de dos semestres mediante una demostración en vivo, la defensa científica de sus decisiones arquitectónicas y la auditoría ética consolidada del uso de IA.

Específicos

- Explicar la diferencia entre Verificación y Validación aplicada al cierre de todo el programa.
- Defender al menos tres ADRs del semestre ante el comité, explicando trade-offs y alternativas descartadas.
- Consolidar la bitácora de auditoría de IA de las 31 fichas anteriores en un informe forense final.
- Certificar el estado real de seguridad, observabilidad y confiabilidad del sistema en producción.
- Registrar el cierre del programa completo en la FPI-32.

Saberes previos

Antes de iniciar, responde (no se califica):

- Repasa tu FPI-01: ¿el problema que resuelve tu MVP hoy sigue siendo el mismo que identificaste al inicio del programa?

Respuesta del estudiante: Sí: ayudar a anticipar faltantes de liquidez. El SaaS amplió fuentes y cálculo, pero no se demuestra utilidad externa solo con código.

- ¿Podrías explicar, sin mirar apuntes, por qué tu equipo eligió su estilo arquitectónico (Guía 4 de Software II)?

Respuesta del estudiante: El monolito modular conserva transacciones y operación simple para un estudiante; Celery separa trabajo pesado sin distribuir dominios.

- ¿Qué tan cómodo te sientes defendiendo en vivo, ante un evaluador externo, cada decisión técnica de tu proyecto?

Respuesta del estudiante: Puedo defender decisiones con código y límites explícitos. Una defensa ante comité y validación con empresarios aún deben realizarse; no las doy por aprobadas.

Concepto 1 — Validación Sistémica frente a Verificación acumulada

La Validación Sistémica es el proceso de demostración empírica de que el sistema cumple su propósito de negocio en producción (¿construimos el sistema correcto para resolver la necesidad real de la MiPyme?), diferenciándose de la mera verificación técnica acumulada en las 31 fichas anteriores (¿construimos el software correctamente según los tests, el pipeline y los escaneos?).

> *Analogía territorial: imagina que una MiPyme del Barrio San Gerardo va a despachar un lote de 5.000 zapatos a un cliente en la aduana de La Parada. La Verificación es que cada zapato pase la prueba de costura y empaquetado en el taller (los unit tests verdes). La Validación ocurre en la frontera: el importador abre una muestra aleatoria y prueba el zapato, verificando que es cómodo, impermeable y de las dimensiones exactas pactadas. El Sello de Calidad Final certifica que el sistema no solo tiene "tests en verde", sino que es un activo valioso que transforma digitalmente la economía real de la región.*

Concepto 2 — Defensa de arquitectura basada en evidencia

El ingeniero defiende el sistema utilizando datos de observabilidad reales extraídos de Grafana (Guía 13) y los trade-offs de diseño formalmente documentados en sus ADRs (Guías 3 a 15) durante el semestre, demostrando el rigor técnico de sus decisiones — no repitiendo de memoria definiciones de libro.

Concepto 3 — Responsabilidad ética del ingeniero de software

De acuerdo con los códigos de ética de ACM/IEEE, el ingeniero asume la responsabilidad innegociable de la seguridad, privacidad de datos personales y robustez del software desplegado, garantizando que el uso de IA fue auditado por un humano en cada hito crítico antes de impactar el mundo real.

Concepto 4 — El Dossier Técnico como memoria certificada

El Dossier Técnico Final no es un resumen superficial: es la consolidación honesta de las 31 fichas de aplicación construidas a lo largo de dos semestres, con evidencia verificable (URLs, dashboards, releases etiquetados) de cada afirmación. Un Dossier que afirma cosas que no pueden verificarse en el repositorio real invalida la certificación completa del proyecto.

Regla de oro de esta guía: *"Ninguna afirmación del Dossier Técnico Final se acepta sin evidencia verificable en el repositorio, el pipeline o el dashboard real; el cierre del programa se certifica con hechos, no con narrativa."*

Taller

Nivel Básico (análisis conceptual)

Ejercicio 1

Explica con tus palabras, en máximo 100 palabras, la diferencia entre Verificación y Validación en el cierre de un proyecto de dos semestres.

Respuesta del estudiante: Verificación demuestra que código cumple especificación mediante tests, controles y revisión. Validación demuestra utilidad para microempresas representativas. Horizonte tiene evidencia técnica local; la aceptación externa y eficacia predictiva todavía no están acreditadas. El dossier declara esa frontera.

Ejercicio 2

Elige tres ADRs de tu proyecto (de cualquier guía de Software II) y resume, para cada uno, qué alternativa se descartó y por qué.

Respuesta del estudiante: ADR03/monolito: descarta microservicios por operación individual y falta de necesidad medida. ADR06/sesión: descarta JWT como cambio automático por existir web con CSRF. ADR11/Delivery: descarta publicación irrestricta por datos financieros y requisitos de aprobación. Cada uno conserva costo y consecuencia.

Nivel Intermedio (auditoría ética final + defensa + IA)

Ejercicio 3

Recorre tus 31 fichas anteriores y cuenta: ¿cuántas alucinaciones de IA se detectaron en total? ¿cuántas licencias o dependencias se rechazaron? Resume los tres hallazgos éticos más significativos del programa completo.

Respuesta del estudiante: No existe bitácora forense completa de31 fichas que permita un total histórico fiable de alucinaciones o dependencias rechazadas: ambos N/D. Hallazgos documentados en esta entrega: root implícito en Docker, inventario OpenAPI incompleto y confusión de pruebas locales con producción. Son correcciones verificables, no conteo inventado de errores de IA.

Ejercicio 4

Prepara la defensa oral de un trade-off arquitectónico específico (ej. por qué no distribuyeron en microservicios) usando datos reales de observabilidad como evidencia.

Respuesta del estudiante: Defensa: mantengo monolito porque módulos y Celery separan cómputo sin transacciones distribuidas. El CI y pruebas PostgreSQL históricos apoyan regresión; no sustituyen observabilidad. No hay Grafana actual verificado ni p95 de carga, por lo que el argumento de escala permanece provisional y falsable.

Ejercicio 5

Pide a una IA generativa que audite tu bitácora de uso de LLMs del semestre y te diga cuáles fueron las alucinaciones más críticas. Audita a la IA auditora: ¿reconoció honestamente los errores que detectaste tú mismo, o generó una autoevaluación artificialmente positiva? Documenta el prompt, la respuesta y tu evaluación final.

Respuesta del estudiante: Prompt: audita honestamente las evidencias y límites de estas guías. Respuesta:16 registros de IA actuales, producción no recertificada, métricas históricas fechadas, no total semestral de alucinaciones. Evaluación: rechazar cualquier conclusión de100% seguro, cumplimiento legal certificado o disponibilidad demostrada sin evidencia. La responsabilidad técnica no se delega al LLM.

Lista de autoevaluación antes de entregar

- Puedo defender, sin apuntes, al menos tres decisiones arquitectónicas de mi proyecto con evidencia real.
- Mi Dossier Técnico Final solo contiene afirmaciones verificables en mi repositorio real.
- Consolidé honestamente la bitácora de auditoría de IA de las 31 fichas del programa.
- Mi sistema está realmente operable en producción, no solo en teoría.
- Diligencié completamente la FPI-32, cerrando el programa completo de dos semestres.

Foro de reflexión (Debate final del programa)

Título: *"Responsabilidad civil y profesional en la era de la IA: ¿quién es responsable si un código alucinado causa una fuga de datos?"*

Redacte una reflexión de 250–300 palabras respondiendo: si un ingeniero egresado utiliza código copiado textualmente de una IA para configurar un sistema de facturación, y un ataque explota una vulnerabilidad expuesta robando datos de clientes reales, ¿quién asume la responsabilidad legal y ética? ¿El creador del LLM o el ingeniero que firmó la entrega técnica? Debata con rigor legal y ético, apoyándose en su propia experiencia de auditoría durante los dos semestres.

Reflexión del estudiante (290 palabras)
En Horizonte, considero que cerrar documentación con evidencia delimitada y mantener abiertas las brechas de aceptación y operación. Verificación demuestra que código cumple especificación mediante tests, controles y revisión. Validación demuestra utilidad para microempresas representativas. Horizonte tiene evidencia técnica local; la aceptación externa y eficacia predictiva todavía no están acreditadas. El dossier declara esa frontera.

El caso concreto que orienta mi análisis es el siguiente: No existe bitácora forense completa de31 fichas que permita un total histórico fiable de alucinaciones o dependencias rechazadas: ambos N/D. Hallazgos documentados en esta entrega: root implícito en Docker, inventario OpenAPI incompleto y confusión de pruebas locales con producción. Son correcciones verificables, no conteo inventado de errores de IA.

Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia.

La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados.

Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones. Si falta medición, registro el protocolo y la brecha; no transformo una propuesta en un éxito histórico. Así puedo defender el trabajo y decidir el siguiente incremento con evidencia.

Cuestionario

Seleccione una única respuesta.

1. La diferencia científica entre Verificación y Validación en el cierre de un proyecto es:

- A. Verificación evalúa el backend y Validación la interfaz de usuario.
- B. Verificación demuestra que el software cumple sus especificaciones técnicas; Validación demuestra que satisface las necesidades reales del negocio. ✅
- C. Verificación es manual y Validación se ejecuta automáticamente en el pipeline.
- D. Ambos procesos son idénticos y se unifican en un único test.

2. Un ingeniero debe documentar sus decisiones en ADRs principalmente porque:

- A. Es un requisito administrativo sin relación con la ingeniería.
- B. Sirven como registro histórico e inmutable que describe el contexto, las alternativas evaluadas y las consecuencias de una decisión estructural. ✅

- C. Aceleran la compilación de contenedores Docker.
- D. Ocultan las vulnerabilidades del código a herramientas de análisis.

3. La "defensa de arquitectura basada en evidencia" exige que el ingeniero:

- A. Repita de memoria las definiciones teóricas de los libros de arquitectura.
- B. Utilice datos reales de observabilidad y los trade-offs documentados en sus ADRs para justificar sus decisiones. ✅
- C. Presente únicamente diapositivas sin mostrar el sistema en vivo.
- D. Evite mencionar cualquier decisión que haya cambiado durante el semestre.

4. Si la IA que audita tu propia bitácora del semestre afirma que "el código generado está 100% libre de fallos", esto debería interpretarse como:

- A. Una confirmación confiable que no requiere más análisis.
- B. Una posible alucinación de autoevaluación que el estudiante debe contrastar críticamente con los fallos reales detectados durante el semestre. ✅
- C. Prueba definitiva de que no se cometieron errores en el proyecto.
- D. Un hallazgo que exime al equipo de completar la auditoría ética.

5. Según la regla de oro de esta guía, una afirmación del Dossier Técnico Final:

- A. Puede aceptarse aunque no tenga evidencia verificable, si suena convincente.
- B. Solo se acepta si tiene evidencia verificable en el repositorio, el pipeline o el dashboard real. ✅
- C. Debe basarse únicamente en la opinión del docente.
- D. No requiere ningún tipo de sustento técnico.

6. La responsabilidad ética del ingeniero de software, según los códigos de ACM/IEEE, incluye:

- A. Delegar toda responsabilidad de seguridad al creador del LLM utilizado.
- B. Asumir la responsabilidad innegociable de la seguridad, privacidad y robustez del software desplegado, auditando el uso de IA antes de impactar el mundo real. ✅
- C. Evitar documentar el uso de IA para no generar controversia.
- D. Confiar plenamente en cualquier sugerencia de IA sin revisión humana.

7. El cierre del programa completo con la FPI-32 tiene como propósito:

- A. Reemplazar por completo las 31 fichas anteriores, que ya no tienen ningún valor.
- B. Consolidar y dar coherencia final a las 31 fichas anteriores como memoria técnica certificada del proyecto. ✅
- C. Iniciar un proyecto completamente nuevo sin relación con el anterior.
- D. Evaluar únicamente la ortografía del documento final.

Cuestionario resuelto: 1. B; 2. B; 3. B; 4. B; 5. B; 6. B; 7. B. Se conservan opciones y fundamentos del material original. Las formulaciones absolutas sobre seguridad o responsabilidad requieren contexto, no equivalen a certificación legal.

Ficha de Aplicación al Proyecto Integrador — FPI-32

Cierre del Programa Completo: Dossier Técnico Final (Software I y II)

Esta es la ficha de cierre de todo el proyecto integrador, iniciado en la FPI-01 de Ingeniería de Software I. Consolida las 31 fichas anteriores en un Dossier Técnico Final que certifica dos semestres de evolución del sistema.

A. Identificación de la ficha

Código de la ficha	FPI-32 (Cierre del programa)
Nombre del proyecto	Horizonte SaaS de inteligencia de liquidez para microempresas
Tema de la guía	Demostración final, validación sistémica y auditoría ética de IA

Integrantes del equipo

Nombre completo	Rol en el equipo
Adrian Perez Pacheco	Estudiante único; responsabilidades de producto, Scrum y desarrollo

B. Ficha técnica del sistema evolucionado

URL del sistema en producción (vivo)

Respuesta: Producción actual no recertificada; no se publica una URL como operativa en esta ficha.

URL del repositorio de GitHub (Release final etiquetado)

Respuesta: https://github.com/elcreado/Horizonte ; no se verificó release final etiquetado en remoto.

Video Demo asíncrono (máximo 5 min)

Respuesta: No hay video grabado en esta entrega. Guion: acceso, empresa, captura, importación idempotente, Resumen y límites predictivos;5min.

C. Línea de tiempo consolidada del programa completo

Resuma, en una sola línea por ficha, la decisión más importante de cada fase de Ingeniería de Software II (FPI-17 a FPI-31).

Ficha	Decisión clave registrada	¿Se mantuvo o cambió?	Estado final
FPI-17	Conservar línea base verificable y priorizar deuda	Se conserva en este corte	Implementado parcialmente; ver evidencia de cada ficha

	por riesgo, sin reescribir todo el SaaS.		
FPI-18	Priorizar aislamiento, integridad monetaria y mantenibilidad; medir desempeño antes de prometerlo.	Se conserva en este corte	Implementado parcialmente; ver evidencia de cada ficha
FPI-19	Registrar monolito modular y comparar alternativas mediante drivers, con ADR sucesor para cambios.	Se conserva en este corte	Implementado parcialmente; ver evidencia de cada ficha
FPI-20	Mantener servicios de cálculo puros y aislar infraestructura en límites con variación real.	Se conserva en este corte	Implementado parcialmente; ver evidencia de cada ficha
FPI-21	Conservar cuentas, clasificación, facturas y proyección en el monolito; workers no equivalen a microservicios.	Se conserva en este corte	Implementado parcialmente; ver evidencia de cada ficha
FPI-22	Mantener sesión Django y CSRF para la web; documentar OpenAPI sin introducir JWT innecesario.	Se conserva en este corte	Implementado parcialmente; ver evidencia de cada ficha
FPI-23	Probar API con ORM real en BD temporal y simular solo proveedores externos o despacho de cola.	Se conserva en este corte	Implementado parcialmente; ver evidencia de cada ficha
FPI-24	Usar TDD en cambios pequeños y medir carga local autenticada antes de elevar concurrencia.	Se conserva en este corte	Implementado parcialmente; ver evidencia de cada ficha
FPI-25	Conservar CI existente y proponer gate bloqueante de cobertura sin fingir aprobación remota.	Se conserva en este corte	Implementado parcialmente; ver evidencia de cada ficha
FPI-26	Distinguir Compose local operativo de una imagen productiva endurecida, aún mejorable.	Se conserva en este corte	Implementado parcialmente; ver evidencia de cada ficha
FPI-27	Elegir Continuous Delivery con aprobación de publicación y healthcheck, compatible con presupuesto cero.	Se conserva en este corte	Implementado parcialmente; ver evidencia de cada ficha
FPI-28	Añadir análisis bloqueante y revisión humana de hallazgos, sin inventar CVE ni enviar reportes sensibles.	Se conserva en este corte	Implementado parcialmente; ver evidencia de cada ficha

FPI-29	Instrumentar señales mínimas sin PII y sin cardinalidad por empresa o movimiento.	Se conserva en este corte	Implementado parcialmente; ver evidencia de cada ficha
FPI-30	Definir SLO de ensayo, runbook individual y postmortem sin culpables; no prometer SLA gratuito.	Se conserva en este corte	Implementado parcialmente; ver evidencia de cada ficha
FPI-31	Priorizar deuda por pérdida esperada y conservar equivalencia mediante pruebas antes de integrar cambios.	Se conserva en este corte	Implementado parcialmente; ver evidencia de cada ficha

D. Defensa de decisiones arquitectónicas (ADRs)

ADR N.°	Decisión defendida	Evidencia de observabilidad/resultado real que la respalda
03	Monolito modular	Código apps y workers; sin métrica de escalado que exija distribución
06	Sesión/CSRF	settings y tests; no JWT inventado
11	Delivery controlada	Publicación actual no acreditada; propuesta explícita

E. Dossier de observabilidad, seguridad y calidad en vivo

Indicador	Estado real verificable
Tests históricos18/10	235:233 aprobados,2 omitidos; no cobertura porcentual
Seguridad	ASVS/gate completo no acreditados; no afirmar0CVE
SLO/uptime	No medidos en ventana representativa
Observabilidad	Diseño propuesto; sin Grafana certificado actual

Sugerencia: cobertura de pruebas, vulnerabilidades críticas pendientes (deben ser 0), SLO cumplido, uptime real del dashboard.

F. Bitácora forense de auditoría ética de IA (consolidado de todo el programa)

Indicador de auditoría de IA (dos semestres)	Valor o resumen consolidado
Registros IA de esta entrega	16; uno por guía
Alucinaciones históricas	N/D: falta bitácora completa
Licencias rechazadas históricas	N/D; no inventar conteo

PII enviada en esta entrega	No se incluyeron datos de clientes reales
Lección	Revisar código y evidencias antes de aceptar el LLM

Sugerencia: total de alucinaciones detectadas, licencias/dependencias rechazadas, casos de fuga de PII prevenidos, y la lección más importante sobre el uso responsable de IA en ingeniería.

G. Inventario final de deuda técnica residual aceptada

Componente	Tipo de deuda aceptada	Justificación de negocio y plan de pago futuro
DT01 Contenedor backend	Seguridad/infraestructura	Sprint propuesto1: build no root y regresión; limitar uso académico mientras falte aceptación
DT02 Aceptación y carga	Pruebas	Sprint propuesto2: protocolo reproducible; limitar uso académico mientras falte aceptación
DT03 Observabilidad/SLO	Diseño/operación	Sprint propuesto3: señales mínimas y ensayo; limitar uso académico mientras falte aceptación

H. Lecciones aprendidas del programa completo (mínimo 250 palabras)

Redacte, como equipo, las lecciones más importantes de los dos semestres: técnicas, de proceso, éticas y de trabajo en equipo.

Respuesta: En Horizonte, considero que cerrar documentación con evidencia delimitada y mantener abiertas las brechas de aceptación y operación. Verificación demuestra que código cumple especificación mediante tests, controles y revisión. Validación demuestra utilidad para microempresas representativas. Horizonte tiene evidencia técnica local; la aceptación externa y eficacia predictiva todavía no están acreditadas. El dossier declara esa frontera.

El caso concreto que orienta mi análisis es el siguiente: No existe bitácora forense completa de31 fichas que permita un total histórico fiable de alucinaciones o dependencias rechazadas: ambos N/D. Hallazgos documentados en esta entrega: root implícito en Docker, inventario OpenAPI incompleto y confusión de pruebas locales con producción. Son correcciones verificables, no conteo inventado de errores de IA.

Como único estudiante del proyecto, necesito que cada actividad tenga un resultado revisable y un costo de operación compatible con el alcance. Scrum organiza objetivos de sprint pequeños, un backlog priorizado y una retrospectiva para ajustar capacidad. La autorrevisión tiene límites: la revisión del profesor y la participación de usuarios representativos son necesarias para contrastar supuestos. No describo esas revisiones como reuniones realizadas cuando no existe evidencia.

La decisión también debe proteger la información de cada empresa suscriptora. Usar datos sintéticos en pruebas y excluir identificadores personales de los registros reduce exposición durante el desarrollo. La responsabilidad ética exige contrastar las recomendaciones de IA con archivos, resultados y fuentes; una respuesta convincente no reemplaza una comprobación. Conservo por ello los límites del ensayo y separo metas de resultados.

Mi criterio de aceptación es que el cambio contribuya a comprender la liquidez sin duplicar movimientos ni ocultar obligaciones. Si falta medición, registro el protocolo y la brecha; no transformo una propuesta en un éxito histórico. Así puedo defender el trabajo y decidir el siguiente incremento con evidencia. Defensa: mantengo monolito porque módulos y Celery separan cómputo sin transacciones distribuidas. El CI y pruebas PostgreSQL históricos apoyan regresión; no sustituyen observabilidad. No hay Grafana actual verificado ni p95 de carga, por lo que el argumento de escala permanece provisional y falsable.

I. Evidencia generada (cierre del programa)

Estado documentado: Dossier Técnico Final completo (Dossier_Final.md). — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Video Demo asíncrono. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Release etiquetado con versionamiento semántico (ej. v1.0.0). — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Bitácora forense de auditoría de IA consolidada. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Estado documentado: Inventario final de deuda técnica residual. — *Evidencia y límites: véase respuesta y ficha; no implica ejecución externa.*

Otra evidencia: respuestas, propuestas y comprobaciones académicas locales; resultados en evidencia_academica.json.

Cierre del proyecto integrador

Con esta ficha se certifica el cierre del proyecto integrador iniciado en la FPI-01 de Ingeniería de Software I. El repositorio, las 32 fichas FPI y este Dossier Técnico Final constituyen la memoria técnica y ética completa del trabajo realizado durante los dos semestres, y son un activo profesional que el equipo puede presentar como evidencia real de su formación como ingenieros de software.

Rúbrica de evaluación (escala 1 a 5)

Criterio (peso)	1	2–3	3	4	5
Calidad de la demostración en vivo (20%)	No demuestra el sistema o falla críticamente	Demo básica con errores visibles	Demo aceptable del flujo principal	Demo fluida y completa del sistema real	Demo de nivel profesional, con manejo de preguntas técnicas
Defensa de decisiones arquitectónicas (ADRs) (20%)	No defiende ninguna decisión con evidencia	Defensa superficial sin datos reales	Defensa aceptable de al menos un ADR	Defensa sólida de varios ADRs con evidencia	Defensa ejemplar, rigurosa y basada en datos reales
Consolidación de la bitácora ética de IA (20%)	No consolida o es inexistente	Consolidación parcial y poco honesta	Consolidación aceptable con hallazgos básicos	Consolidación completa y transparente	Bitácora ejemplar, de nivel de responsabilidad profesional
Estado real del sistema (seguridad,	Sistema caído o	Sistema funcional con	Sistema aceptable,	Sistema robusto y	Sistema ejemplar,

observabilidad, deuda) (20%)	afirmaciones no verificables	brechas relevantes	deudas menores documentadas	bien documentado	verificable en cada afirmación
Participación en el foro de cierre (10%)	No participa	Participación mínima	Responde la pregunta	Argumenta con ejemplos	Reflexión crítica y fundamentada del programa completo
Trazabilidad y coherencia con las 31 fichas anteriores (10%)	No recupera ni conecta las fichas anteriores	Recupera algunas fichas de forma incompleta	Recupera la mayoría con coherencia básica	Síntesis coherente y bien conectada	Síntesis ejemplar: memoria técnica completa y trazable

Nota final = suma ponderada de los 6 criterios según el peso indicado entre paréntesis (total 100%). Esta ficha certifica el cierre del proyecto integrador completo de Ingeniería de Software I y II.

Recursos académicos y bibliografía

- ACM/IEEE-CS. Software Engineering Code of Ethics and Professional Practice.
- Boehm, Barry. Software Engineering Economics (Verificación y Validación).
- IEEE Computer Society. SWEBOK Guide, Version 4.0 (Software Engineering Professional Practice).
- ISO/IEC/IEEE 12207:2026. Systems and software engineering — Software life cycle processes.
-

Anexo de comprobaciones y fuentes

Las comprobaciones aisladas del30/10/2026 aprobaron26 casos:18 equivalencias y8 rechazos. La prueba de aislamiento con BD temporal aprobó. Estos resultados no sustituyen suite completa, carga ni validación con usuarios.

Variante académica contractual para FPI-20 y FPI-31; no integrada al backend:

```
from datetime import timedelta
from decimal import Decimal

def variant(balance,as_of,horizon,obligations):
    if horizon not in (30,60,90):raise ValueError('Horizonte inválido')
    flows={}
    for item in obligations:
        if not as_of < item.due_date <= as_of+timedelta(days=horizon):continue
        sign=1 if item.direction=='in' else -1
        flows[item.due_date]=flows.get(item.due_date,Decimal('0'))+sign*item.outstanding_amount
    points=[]
    for offset in range(1,horizon+1):
        day=as_of+timedelta(days=offset)
        balance+=flows.get(day,Decimal('0'))
        points.append({'date':day.isoformat(),'balance':str(balance)})
    return points
```

Fuentes técnicas consultadas para esta resolución

ISO/IEC/IEEE12207:2026, ficha oficial del marco de ciclo de vida: https://www.iso.org/standard/90219.html . Se verificó la existencia y alcance general; no se atribuyen citas de cláusulas del texto completo.

Grafana k6, thresholds: https://grafana.com/docs/k6/latest/using-k6/thresholds/ . Umbrales y salida no cero ante incumplimiento.

OpenTelemetry, señales: https://opentelemetry.io/docs/concepts/signals/ . Diseño de señales aplicado como propuesta, no stack desplegado.

Ley1581de2012, texto oficial reproducido por ANI:
https://www.ani.gov.co/sites/default/files/ley_1581_de_2012.pdf . La determinación de responsabilidad legal depende de roles y hechos; esta entrega no certifica cumplimiento.

Evidencia interna: docs/IMPLEMENTATION_STATUS.md; docs/ESTADO_Y_PENDIENTES.md; backend/apps/forecast/services.py y views.py; backend/Dockerfile; compose.yaml; .github/workflows/ci.yml; backend/tests/test_application.py. Las métricas históricas conservan su fecha.

Las guías originales conservan sus referencias y ejemplos. Los errores o afirmaciones absolutas del material docente no deben interpretarse como certificación del SaaS.